Confidence-Weighted Multi-Scale Rolling Forecasting of Urban Taxi Demand with GraphSAGE-Based Spatial Refinement

Tianqi Wang

University of Warwick

MSc by Research Thesis

Supervisor: Professor Ligang He

Submitted in partial fulfilment of the requirements for the degree of Master of Science by Research

04 2026

Abstract

Accurate urban taxi demand forecasting is essential for intelligent transportation management, fleet dispatching, and congestion mitigation. However, this task remains challenging because demand patterns exhibit strong temporal non-stationarity, multi-scale periodicity, and complex spatial dependency across city regions. To address these issues, this study proposes a two-stage framework for hourly taxi demand prediction over New York City Taxi Zones, integrating multi-scale temporal modeling, uncertainty estimation, and graph-based spatial refinement. In the first stage, a persistent per-zone forecasting model is developed to capture temporal dynamics at four different scales: hourly, daily, weekly, and monthly. The architecture combines GRU, LSTM, and Transformer components to learn short-term fluctuations and long-range temporal regularities within a unified multi-branch design. Monte Carlo Dropout is further employed to estimate predictive uncertainty for each zone. In the second stage, the base predictions are treated as priors and refined through a GraphSAGE-based residual learning module constructed on an origin-destination flow graph. To improve robustness, a confidence-weighting mechanism is introduced by combining historical sufficiency, model stability, and neighborhood consistency, allowing unreliable zones to contribute less during graph refinement. The overall system is evaluated under a rolling forecasting protocol that simulates real-world hourly prediction updates. Experimental results indicate that the proposed graph refinement stage reduces prediction error relative to the temporal baseline and is especially beneficial for zones with sparse history or high predictive variance. In addition, the proposed method improves spatial coherence among neighboring zones and provides a more stable forecasting process under dynamic urban conditions. Overall, the framework demonstrates that combining multi-scale sequence learning with confidence-aware graph neural refinement offers an effective, robust, and extensible solution for spatiotemporal urban taxi demand forecasting problems.

Acknowledgements

I would like to thank ...

Contents

List of Figures

List of Tables

Chapter 1

Introduction

1.1 Background and Motivation

Urban mobility forecasting has become a central problem in intelligent transportation systems because accurate short-term demand estimates influence vehicle dispatching, idle cruising reduction, passenger waiting time, congestion management, and the overall operational efficiency of city-scale transport services.[1] In large metropolitan areas, taxi demand is not only highly dynamic but also strongly heterogeneous across space and time.[2] Different urban regions exhibit distinct activity rhythms shaped by commuting peaks, nightlife, commercial density, tourism, and residential land use. At the same time, the temporal behaviour of demand is rarely governed by a single periodic pattern. Instead, it reflects a mixture of short-term fluctuation, daily repetition, weekly regularity, and longer-range seasonal tendencies. These properties make hourly taxi demand forecasting a difficult spatiotemporal learning task rather than a conventional univariate time-series regression problem.[3]

The challenge becomes more pronounced when the forecasting objective is defined at the level of individual Taxi Zones. Fine-grained zone-level prediction is practically useful because dispatch systems, relocation strategies, and local service balancing all depend on geographically resolved demand estimation.[4] However, a disaggregated formulation also introduces additional statistical difficulty. Some zones generate abundant historical observations, while others are comparatively sparse or irregular. In low-activity regions, the observed series may contain many zero or near-zero counts interspersed with occasional surges, which makes the learning problem noisy and unstable due to sparsity and uncertainty in travel demand data [5]. In high-activity districts, by contrast, the model must handle larger amplitude fluctuations and sharper temporal transitions, requiring strong capability in capturing complex spatial-temporal dependencies [6]. A forecasting framework that performs well across all zones therefore needs to balance sensitivity to local temporal dynamics with robustness to data sparsity and uncertainty.

A second source of complexity lies in the spatially coupled nature of urban travel demand. Taxi zones do not evolve independently. Neighbouring regions, or regions connected by strong origin-destination flows, often exhibit correlated demand patterns because travel activity propagates through the city via commuting corridors, business districts, airports, entertainment clusters, and residential catchment areas.[1] A purely per-zone temporal model can learn local history effectively, but it may fail to correct errors that arise when neighbouring zones move coherently or when a zone's local history is insufficient to explain its next-hour demand. This is particularly problematic under rolling forecasting conditions, where predictions must be refreshed hour by hour and small local errors can accumulate into spatially inconsistent outputs. Capturing such dependence requires a representation that can use structural information beyond the isolated time series of a single region.

Beyond technical performance, this problem also has wider urban significance. More reliable demand forecasting can support better vehicle allocation and improve the spatial matching between supply and expected local demand [7], [8]. It can also help reduce empty cruising or deadhead mileage, thereby improving operational and potentially energy efficiency [9]. In a data-scarce or highly variable urban environment, a forecasting system must therefore be judged not only by average error metrics but also by its stability, interpretability, and ability to remain useful across heterogeneous operational contexts. This emphasis on operational realism gives the study both academic relevance and applied value.

1.2 Problem Definition and Research Challenges

Recent research on traffic and mobility prediction has increasingly moved toward spatiotemporal architectures that combine temporal sequence modelling with graph-based spatial learning.[10] Temporal models such as recurrent neural networks and transformer variants are effective for extracting sequential dependencies**info15090517rnn_evolution_2024**, whereas graph neural networks offer a principled way to propagate information across related regions[11]. Yet the The integration of these components is not straightforward. End-to-end spatiotemporal models can be highly effective for traffic forecasting because they jointly model spatial and temporal dependencies, and representative architectures such as DCRNN and Graph WaveNet have achieved strong results in this setting [12], [13]. However, such graph-based deep models are often difficult to interpret, which limits their transparency in operational use [14]. In practical city-scale forecasting, there is therefore value in separating the problem into an initial temporal prediction stage and a subsequent spatial correction stage. This type of decoupled design is also supported by recent work showing that different latent traffic signals can be handled more effectively when spatial diffusion and inherent temporal components are modeled separately [15]. Such a design allows the first component to focus on modelling zone-specific temporal regularity while the second component exploits graph structure to refine predictions in a more modular and controlled manner.

Another important issue is uncertainty. Most prediction systems output a single point estimate, but for urban demand forecasting this is frequently insufficient, since recent studies have shown that spatiotemporal travel demand prediction should explicitly quantify predictive uncertainty rather than rely only on deterministic outputs [5], [16]. Prediction reliability varies significantly across time and across regions. A confident prediction in a historically dense, stable zone should not be treated in the same way as a volatile estimate produced for a sparse zone with weak historical support [5]. If uncertainty can be quantified explicitly, it can be used to guide later stages of the model and to reduce the influence of unreliable nodes during learning. Therefore, uncertainty is not merely an auxiliary diagnostic quantity; it can also serve as a useful signal for weighting, calibration, and robustness enhancement.

From a research perspective, the problem addressed in this thesis sits at the intersection of spatiotemporal forecasting, graph neural networks for transport modelling, uncertainty-aware prediction, and deployment-oriented rolling inference [13]. Its novelty does not rely on any single component in isolation. GRU, Monte Carlo Dropout, and GraphSAGE are all established methods in sequence modelling, uncertainty estimation, and inductive graph representation learning, respectively [17], [18], [19]. The contribution instead lies in how these components are combined into a coherent forecasting pipeline for zone-level hourly taxi demand.

At the same time, the current project setting presents several important methodological challenges. First, the graph refinement stage is currently trained and evaluated on the same hourly batch, which may inflate the apparent gain of graph-based correction and weaken claims about cross-temporal generalisation. Second, although edge weights are constructed and preserved in the data pipeline, they are not yet explicitly exploited inside the `SAGEConv` message-passing operator. Third, the temporal forecasting framework now supports both persistent checkpoint reuse and lightweight incremental updating; however, in the default rolling configuration, the persistent version is still used unless the script is manually switched to the incremental manager. Consequently, the system already contains a practical path toward online adaptation, but this capability is not yet the default experimental setting. These issues should not be viewed simply as implementation defects. Instead, they delineate the real methodological boundary of the current study and provide a clear basis for the experimental analysis, ablation design, and future improvements discussed in this thesis.

1.3 Research Rationale and Proposed Framework

The present project is motivated by precisely these difficulties. It addresses hourly taxi demand prediction for New York City Taxi Zones through a two-stage, confidence-weighted, multi-scale forecasting framework. According to the project blueprint, the system is organised around three linked ideas: first, each zone is predicted with a persistent multi-branch temporal model that captures information at hourly, daily, weekly, and monthly scales; second, predictive uncertainty is estimated through Monte Carlo Dropout and fused with other reliability indicators to form a confidence score [18]; third, the initial predictions are refined on a graph built from origin-destination flow relationships using a GraphSAGE residual learning module [19]. The overall pipeline is executed in a rolling fashion, producing hour-by-hour forecasts, metrics, and output files that simulate a realistic operational deployment setting.

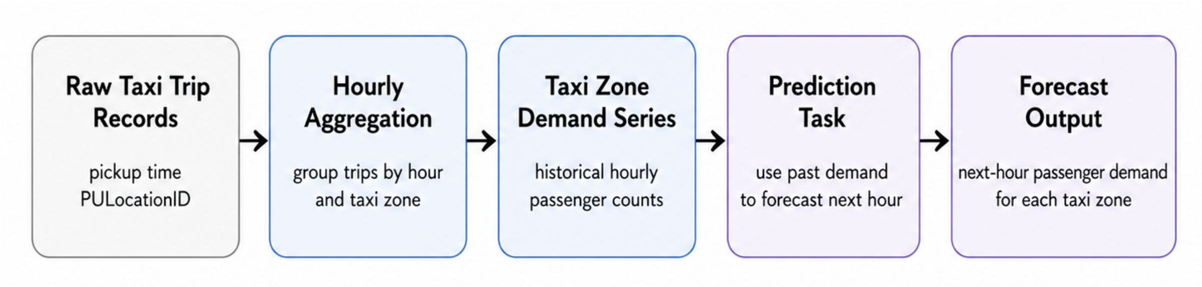

Figure 1.1: Raw taxi trip records are first aggregated into hourly passenger counts for each taxi zone, forming zone-level historical demand series. Based on these time series, the forecasting task is to predict the passenger demand in the next hour for every taxi zone.

The choice of a multi-scale temporal backbone is central to the rationale of this work. Urban taxi demand is inherently hierarchical in time. Short-term behaviour captures immediate fluctuations and local momentum. Daily periodicity reflects repeated within-day routines such as morning commute and evening return flows. Weekly periodicity encodes the distinction between weekdays and weekends, and longer temporal windows can provide a broader view of persistent demand level and slow trend evolution. A model built on a single temporal granularity often struggles to preserve all of these signals simultaneously. In this project, the temporal stage therefore uses four branches corresponding to 1-hour, 1-day, 1-week, and 1-month contexts. The implementation combines LSTM encoders for daily and weekly windows, a Transformer-based encoder for the longer monthly window, and a GRU-driven output pathway for the short-term branch, with the fused trend representation used to support final next-step prediction. This design reflects an explicit modelling assumption: different temporal horizons contribute complementary predictive information, and their joint representation is more suitable for robust zone-level forecasting than any single-scale view alone [20], [21].

Equally important is the decision to adopt a persistent per-zone training strategy rather than a monolithic city-wide temporal model. In the current implementation, each zone maintains its own checkpointed forecasting model, allowing training to occur once and then be reused during subsequent rolling steps. This persistent formulation has two practical advantages. First, it respects local heterogeneity by letting each zone learn its own temporal dynamics without forcing all regions into a single parameter space. Second, it reduces unnecessary retraining cost during rolling evaluation, which is useful when the system is intended to mimic hourly forecasting updates. At the same time, the project explicitly recognises that persistent checkpoint reuse is not the only available operating mode. The repository also provides an incremental training variant in which newly arrived time windows trigger lightweight model updates. However, in the present rolling script configuration, the persistent manager remains the default setting, so online adaptation is supported by the codebase but not enabled by default.

The uncertainty component further differentiates this framework from a standard multi-scale recurrent model. Instead of treating prediction variance as a post hoc statistic, the system uses Monte Carlo Dropout to generate repeated stochastic forward passes and estimate predictive standard deviation [18]. These uncertainty estimates are then integrated with two additional reliability signals: a prior score based on the historical richness of each zone, and a neighbourhood consistency score derived from agreement with graph neighbours. The resulting confidence value is not only recorded as an interpretive measure, but also used downstream to weight graph-based learning. In real demand data, zones with limited history, unstable predictions, or poor agreement with surrounding structure are precisely the cases in which the model should be more cautious. Confidence weighting operationalises that caution mathematically and links uncertainty estimation to model robustness.

The graph refinement stage addresses the main limitation of independent temporal forecasting: lack of spatial correction. After the base model generates per-zone next-hour predictions, the second stage treats these values as priors and learns residual corrections over an origin-destination flow graph. The use of GraphSAGE is notable here because the model is not asked to predict demand from scratch on the graph. Instead, it learns how much the initial temporal estimate should be adjusted once spatial context is taken into account. This residual perspective lowers the burden placed on the graph model and makes the refinement process easier to interpret because improvements can be understood as structured corrections rather than opaque end-to-end remapping [19].

The use of an origin-destination flow graph rather than a purely geographic adjacency graph is also theoretically meaningful. A mobility system is shaped not only by physical proximity but by functional interaction. Two zones may be geographically near yet weakly related in taxi demand, while distant zones can exhibit strong dependence if substantial travel flow connects them. By constructing adjacency from an OD flow matrix, the project attempts to encode actual travel coupling rather than crude map neighbourhood alone. This choice aligns with the practical intuition that mobility demand propagates along usage structure, not merely spatial contiguity [1], [22].

The rolling evaluation protocol is another defining feature of this study. Instead of training once and reporting static test results, the pipeline advances target time step by time step, recalculating predictions and metrics at each hour. This setup better resembles how forecasting systems are used in deployment, where the model must repeatedly produce near-future estimates as new time points arrive. Rolling evaluation is important not only for realism but also for diagnosis. It reveals whether performance is stable over time, whether certain hours are systematically harder than others, and whether the spatial refinement stage consistently improves over the temporal baseline.

1.4 Research Aim and Questions

Against this background, the aim of this thesis is to investigate whether a two-stage architecture can improve hourly urban taxi demand forecasting by combining local multi-scale temporal modelling with confidence-aware graph refinement.

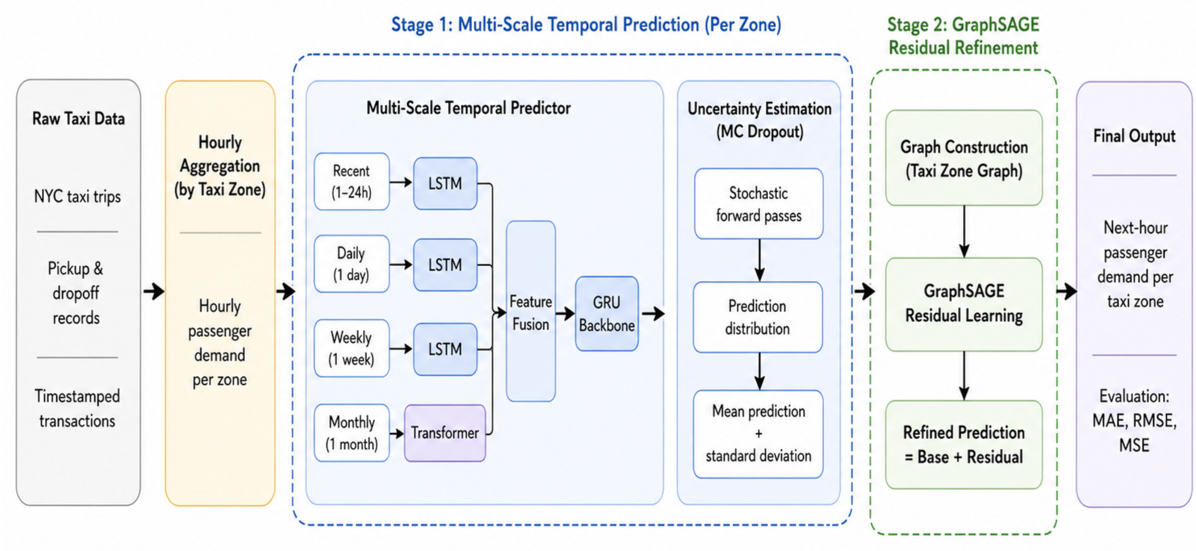

Figure 1.2: Framework for taxi-zone passenger demand prediction using multi-scale temporal forecasting and GraphSAGE-based residual refinement.

More specifically, this thesis addresses the following research questions:

1. Can a per-zone multi-scale recurrent architecture provide robust next-hour predic-

tions across heterogeneous Taxi Zones?

2. Can uncertainty-derived confidence scores improve the stability of spatial refinement by weighting graph learning according to node reliability?

3. Does a GraphSAGE residual correction stage produce more accurate and spatially coherent forecasts than the temporal baseline alone under a rolling evaluation setting?

1.5 Contributions of This Thesis

Accordingly, the main contributions of this work can be summarised as follows:

1. It develops a multi-scale temporal forecasting framework for hourly Taxi Zone demand that combines GRU, LSTM, and Transformer components across four temporal horizons.

2. It incorporates Monte Carlo Dropout-based uncertainty estimation into a broader confidence fusion mechanism that also accounts for historical sufficiency and neighbourhood consistency.

3. It introduces a GraphSAGE-based residual refinement stage over an origin-destination flow graph to correct temporal predictions using spatial structure.

4. It implements the whole method within a rolling prediction pipeline with persistent checkpoint reuse as the default setting, while also supporting an incremental update variant for improved rolling-stage stability.

Together, these components form a practical and extensible foundation for urban demand forecasting in settings where temporal complexity, spatial coupling, and uncertainty must all be considered simultaneously.

1.6 Thesis Structure

The remainder of this thesis is organised as follows. Chapter 2 reviews related work on urban demand forecasting, multi-scale temporal sequence modelling, graph neural networks, and uncertainty-aware prediction. Chapter 3 formalises the problem setting and presents the proposed two-stage framework in detail, including temporal architecture, confidence construction, graph refinement, and rolling inference logic. Chapter 4 describes the implementation pipeline, data preparation process, key classes, and engineering decisions reflected in the codebase. Chapter 5 evaluates the temporal baseline and graph-refined model under rolling forecasting conditions and discusses performance patterns. Finally, Chapter 6 concludes the thesis with a discussion of strengths, limitations, and future directions, including better generalisation protocols, explicit edge-weight usage, and more systematic evaluation of the incremental online adaptation mechanism already provided by the repository.

Chapter 2

Literature Review

2.1 Introduction to the Literature Context

Urban mobility forecasting has become a central problem in intelligent transportation systems because accurate short-term demand estimates influence vehicle dispatching, idle cruising reduction, passenger waiting time, congestion management, and the overall operational efficiency of city-scale transport services [23], [24]. In large metropolitan areas, taxi demand is not only highly dynamic but also strongly heterogeneous across space and time [2], [25]. Different urban regions exhibit distinct activity rhythms shaped by commuting peaks, nightlife, commercial density, tourism, and residential land use [25]. At the same time, the temporal behaviour of demand is rarely governed by a single periodic pattern. Instead, it reflects a mixture of short-term fluctuation, daily repetition, weekly regularity, and longer-range seasonal tendencies, which makes hourly taxi demand forecasting a difficult spatiotemporal learning task rather than a conventional univariate time-series regression problem [13], [20].

The challenge becomes more pronounced when the forecasting objective is defined at the level of individual Taxi Zones. Fine-grained zone-level prediction is practically useful because dispatch systems, relocation strategies, and local service balancing all depend on geographically resolved demand estimation [24], [26]. However, a disaggregated formulation also introduces additional statistical difficulty. Some zones generate abundant historical observations, while others are comparatively sparse or irregular. In low-activity regions, the observed series may contain many zero or near-zero counts interspersed with occasional surges, making the learning problem noisy and unstable [5]. In high-activity districts, by contrast, the model must handle stronger amplitude variation and potentially sharper peak transitions, requiring effective modelling of complex spatial-temporal dependence [6], [27]. A forecasting framework that performs well across all zones therefore needs to balance sensitivity to local temporal dynamics with robustness to data sparsity and uncertainty.

A second source of complexity lies in the spatially coupled nature of urban travel demand. Taxi zones do not evolve independently. Neighbouring regions, or regions connected by strong origin-destination flows, often exhibit correlated demand patterns because travel activity propagates through the city via commuting corridors, business districts, airports, entertainment clusters, and residential catchment areas [22], [28]. A purely per-zone temporal model can learn local history effectively, but it may fail to correct errors that arise when neighbouring zones move coherently or when a zone's local history is insufficient to explain its next-hour demand. This is particularly problematic under rolling forecasting conditions, where predictions must be refreshed hour by hour and small local errors can accumulate into spatially inconsistent outputs. Capturing such dependence therefore requires a representation that can use structural information beyond the isolated time series of a single region [13].

For this reason, recent research on traffic and mobility prediction has increasingly moved toward spatiotemporal architectures that combine temporal sequence modelling with graph-based spatial learning [13], [29]. Temporal models such as recurrent neural networks and transformer variants are effective for extracting sequential dependencies, whereas graph neural networks offer a principled way to propagate information across related regions [13]. Yet the integration of these components is not straightforward. End-to-end spatiotemporal models can be powerful, but they are often computationally heavy, difficult to interpret, and less flexible when the temporal and spatial signals differ in quality across nodes [14], [30]. In practical city-scale forecasting, there is value in separating the problem into an initial temporal prediction stage and a subsequent spatial correction stage. Such a design allows the first component to focus on modelling zone-specific temporal regularity while the second component exploits graph structure to refine predictions in a controlled and interpretable manner [15].

Another important issue is uncertainty. Most prediction systems output a single point estimate, but for urban demand forecasting this is frequently insufficient, since recent work has shown that spatiotemporal travel demand prediction benefits from explicitly quantifying predictive uncertainty rather than relying only on deterministic outputs [5], [16]. Prediction reliability varies significantly across time and across regions. A confident prediction in a historically dense, stable zone should not be treated in the same way as a volatile estimate produced for a sparse zone with weak historical support [5]. If uncertainty can be quantified explicitly, it can guide later stages of the model and reduce the influence of unreliable nodes during learning. Therefore, uncertainty is not merely an auxiliary diagnostic quantity; it can also serve as a useful signal for weighting, calibration, and robustness enhancement.

The present project is motivated by precisely these difficulties. It addresses hourly taxi demand prediction for New York City Taxi Zones through a two-stage, confidence-weighted, multi-scale forecasting framework. According to the project blueprint, the system is organised around three linked ideas: first, each zone is predicted with a persistent multi-branch temporal model that captures information at hourly, daily, weekly, and monthly scales; second, predictive uncertainty is estimated through Monte Carlo Dropout and fused with other reliability indicators to form a confidence score [18]; third, the initial predictions are refined on a graph built from origin-destination flow relationships using a GraphSAGE residual learning module [19]. The overall pipeline is executed in a rolling fashion, producing hour-by-hour forecasts, metrics, and output files that simulate a realistic operational deployment setting.

This chapter therefore reviews the literature in six connected parts. First, it examines traditional approaches to urban demand forecasting and their limitations. Second, it reviews deep temporal sequence models and explains why multi-scale learning became important. Third, it surveys spatial modelling approaches, especially graph neural networks, and discusses the relevance of GraphSAGE-style aggregation. Fourth, it considers hybrid spatiotemporal architectures and the distinction between end-to-end fusion and two-stage refinement. Fifth, it reviews uncertainty estimation and confidence-aware learning. Finally, it discusses rolling forecasting, persistent training, and incremental adaptation, which are central to practical deployment. Throughout the chapter, the review will relate prior work back to the needs of the current project and identify the methodological gaps that the thesis should address.

2.2 Traditional Urban Demand Forecasting Methods

The earliest literature on transportation demand forecasting was dominated by statistical and econometric models.

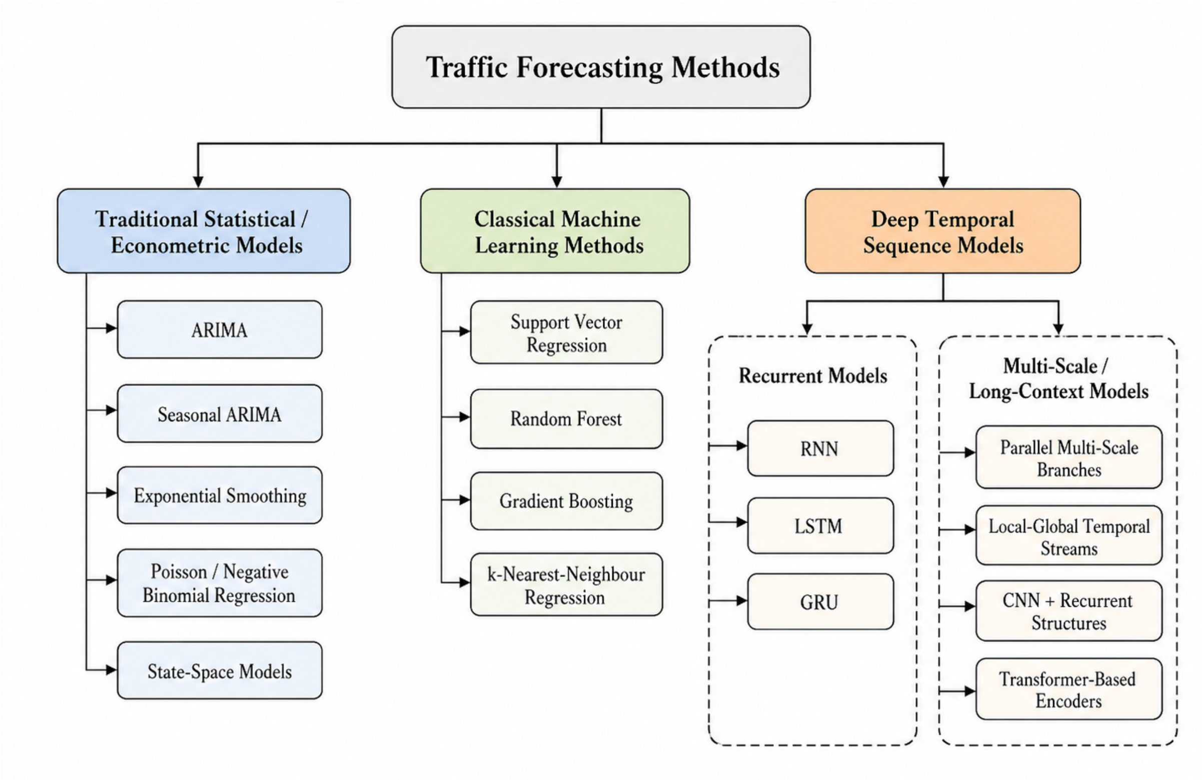

Figure 2.1: Overview of representative traffic forecasting approaches, including traditional statistical models, classical machine learning methods, and deep temporal sequence models, with the latter further divided into recurrent and multi-scale or long-context architectures.

Classical methods included autoregressive integrated moving average models, seasonal ARIMA variants, exponential smoothing, Poisson and negative binomial regression, and state-space formulations [31], [32]. These methods remain attractive because they are interpretable, computationally light, and theoretically well understood. In settings with stable seasonality and relatively smooth temporal structure, such models can provide strong baselines. For aggregate traffic or passenger series, they often perform reasonably well when the forecasting horizon is short and the signal is dominated by linear dependence [31].

However, taxi demand forecasting at the zone level quickly exposes the limits of such models. First, linear autoregressive methods assume relatively simple dependence structures and struggle when multiple temporal scales interact nonlinearly. Second, they are poorly suited to sparse or highly intermittent zones where many observations are zero or near-zero [5]. Third, classical models generally treat each spatial unit independently unless augmented with hand-designed exogenous terms or multivariate extensions, which become difficult to scale in city-wide settings [13], [33]. Fourth, the assumptions required for stationarity, residual independence, or parametric count structure are often violated by urban mobility data [32]. These weaknesses motivated the field to adopt machine learning methods capable of handling high-dimensional nonlinearity more flexibly.

Before deep learning became dominant, methods such as support vector regression, random forests, gradient boosting, and k-nearest-neighbour regression were frequently applied to transport prediction [34], [35]. These models could capture nonlinear feature interactions better than traditional statistical methods, especially when provided with engineered inputs such as hour-of-day, day-of-week, weather indicators, holiday flags, and lagged demand values [32], [35]. Nevertheless, they still depended heavily on feature engineering and did not solve the core representation problem of spatiotemporal dependence. A random forest may capture nonlinear interactions among static covariates, but it does not naturally model long temporal memory or structured spatial propagation. Similarly, support vector methods can be effective on carefully prepared datasets but do not scale elegantly to multi-zone dynamic graphs with rolling re-estimation requirements [13], [34].

A further limitation of both statistical and early machine learning approaches is that they often produce point forecasts without any integrated uncertainty mechanism. In urban mobility applications, this is a significant gap. Demand variance is not homogeneous across zones or time periods, and decision systems can benefit greatly from knowing which predictions are stable and which are unreliable [5], [16]. Thus, even before discussing deep learning, the literature already points toward three essential requirements that simpler approaches do not jointly satisfy: nonlinear multi-scale temporal modelling, explicit spatial coupling, and reliability awareness [5], [13].

These limitations are directly relevant to the current project. The system is designed around zone-level demand prediction with mixed short- and long-term temporal patterns, OD-related spatial coupling, and uncertainty-aware graph refinement through confidence scores. This implies that purely statistical baselines, while still necessary for comparison, cannot be the final methodological destination of the thesis. Instead, they serve mainly as conceptual stepping stones illustrating why a deeper spatiotemporal architecture is justified.

2.3 Deep Temporal Sequence Models for Demand Forecasting

The deep learning literature on forecasting introduced a major shift by replacing manual feature engineering with learned representations. Recurrent neural networks became especially influential because they provide a natural mechanism for modelling sequential dependence. Standard RNNs can, in principle, use previous hidden states to summarise temporal context, but in practice they suffer from vanishing and exploding gradient problems when sequences are long [36]. This limitation led to the widespread adoption of Long Short-Term Memory networks and Gated Recurrent Units, both of which use gating mechanisms to regulate information flow and preserve relevant historical context over longer horizons [17], [37].

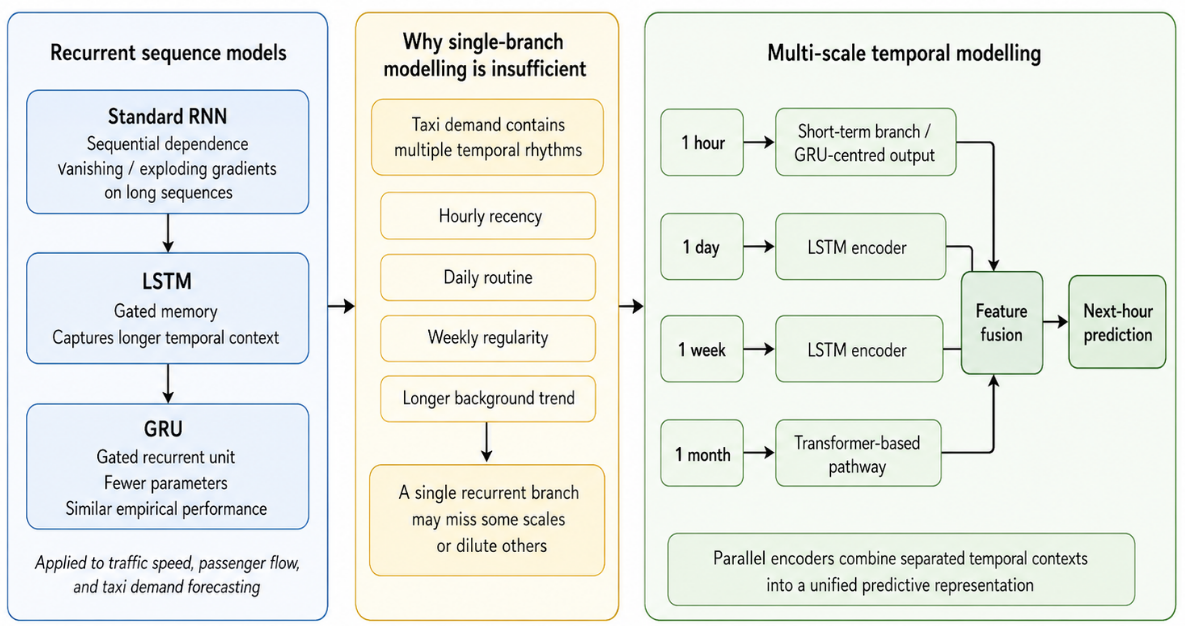

Figure 2.2: Recurrent and multi-scale temporal modelling for demand forecasting.

In transportation forecasting, LSTM-based models were soon applied to traffic speed, passenger flow, and taxi demand because they captured nonlinear temporal dependence more effectively than traditional autoregressive models [13], [38]. GRUs offered a lighter-weight alternative with fewer parameters and often similar empirical performance [39]. In urban demand settings, recurrent models are useful because they can absorb sequential fluctuations without requiring explicit manual design of every lag interaction. They also adapt relatively naturally to rolling one-step-ahead prediction tasks, which is important for hourly operational forecasting.

Yet the recurrent literature also exposed a key problem: a single recurrent branch is often insufficient to capture the coexistence of multiple temporal rhythms. Taxi demand does not evolve only according to the immediately preceding hour. There are within-day cycles, weekly regularities, event-driven deviations, and slower baseline shifts. A model trained only on a short recent history may miss recurrent weekly effects; a model trained on a very long history may dilute immediate local momentum. This tension drove the literature toward multi-scale or multi-resolution temporal modelling [20], [40].

Multi-scale temporal models attempt to process different context windows separately and then combine them into a unified predictive representation. Some methods use parallel recurrent branches fed with different lag lengths. Others separate local and global temporal streams, or introduce convolutional structures to summarise short-term patterns before recurrent integration [40], [41]. More recently, transformer-based encoders have been used to model long-range dependencies, since attention mechanisms can relate distant time points more directly than purely recurrent hidden-state propagation [41], [42]. In mobility prediction, multi-scale designs are attractive because they align closely with known human activity rhythms: hourly recency, daily routine, weekly schedule, and longer background trend [20], [25].

The current project follows this line of development quite explicitly. The temporal stage uses four scales: 1 hour, 1 day, 1 week, and 1 month. The implementation combines LSTM encoders for daily and weekly sequences, a Transformer-based pathway for the monthly context, and a GRU-centred output structure for short-term prediction. This makes the model neither a pure LSTM, pure GRU, nor pure Transformer architecture. Instead, it is a composite multi-branch design in which each component handles the timescale for which it is most suitable. Such a design is well supported by the literature because it reflects the widely recognised principle that no single temporal mechanism dominates all forecasting horizons equally [41], [42].

Another relevant theme in the sequence-model literature is the trade-off between local adaptation and global sharing. Many deep forecasting studies train a single city-wide model using all zones jointly, often with embeddings to represent region identity. This can improve data efficiency because parameters are shared across zones. However, shared models may also obscure strong local heterogeneity, especially when different regions exhibit substantially different temporal patterns [43], [44]. A highly active commercial district and a low-density peripheral zone may not benefit equally from identical temporal parameters. By contrast, per-zone models allow local dynamics to be learned independently but can suffer from limited data in sparse regions. The project adopts a per-zone temporal modelling approach that preserves local structure while later compensating for spatial dependence through graph refinement. This choice aligns with literature that emphasises the importance of respecting regional heterogeneity in mobility prediction, but it also invites comparison with shared or meta-learned temporal models as possible future improvements [45], [46].

A further issue concerns input scaling and zero inflation. Mobility count series often contain sharp spikes, long low-activity periods, and substantial imbalance across zones. In such settings, deep temporal models can become sensitive to amplitude disparities unless scaling is carefully handled. The framework uses MinMax scaling and dense hourly reconstruction with missing hours filled by zero. These are practical engineering decisions consistent with the broader forecasting literature, though they also introduce open questions. For example, MinMax scaling can be sensitive to extreme values, and zero-filling may blur the distinction between true absence and missing observation. These issues do not invalidate the approach, but they are the kinds of modelling assumptions that literature review should surface because they shape how results ought to be interpreted [5], [43].

Overall, the literature on deep sequence learning provides strong support for the project's temporal stage. It justifies the movement beyond classical time-series baselines, motivates the use of gated recurrent models, and explains the need for explicitly multi-scale architecture [41], [43]. At the same time, it suggests several comparative angles that the thesis could later exploit: single-scale versus multi-scale models, GRU-only versus hybrid designs, shared city-level versus per-zone models, and recurrent versus transformer-heavy long-context encoders.

2.4 Multi-Scale Modelling as a Distinct Research Theme

Although multi-scale sequence learning can be discussed under deep temporal modelling, it has become important enough in transportation prediction to warrant a separate treatment. The broader forecasting literature repeatedly shows that urban mobility contains superposed rhythms rather than one dominant cycle. Commuting activity imposes strong diurnal repetition, weekends introduce behavioural discontinuity, and slower structural shifts emerge from tourism, economic conditions, school terms, and seasonal patterns. Models that use only a fixed-length lag often end up overfitting one scale and under-representing another [20], [25].

One family of multi-scale approaches uses handcrafted lag groups. For instance, a model may include the last few hours, the same hour from the previous day, and the same hour from the previous week as separate feature channels. This strategy is simple and often effective, but it depends on fixed assumptions about what temporal anchors matter most. A more flexible family of approaches learns these temporal scales directly using parallel encoders, hierarchical recurrent modules, dilated convolutions, or attention over multiple resolution levels. The key insight is that temporal scale is not just an engineering hyperparameter; it is a structural property of the forecasting problem [43], [47].

For taxi demand in particular, multi-scale methods are especially appropriate because the service is highly responsive to recurring human routines. The same district can behave very differently on weekday mornings, Friday evenings, weekends, and holiday periods. Moreover, the intensity of these patterns may vary across zones. A business district may show strong daily periodicity, while an airport may show broader multi-day or weekly variation linked to travel patterns. This means a robust model should not assume that all temporal information is equally informative for all nodes at all times [20], [25].

The project's four-scale design can be understood in this literature context as a pragmatic but theoretically coherent decomposition. The hourly branch preserves immediate recency. The daily branch captures within-day repetition and typical local rhythm. The weekly branch addresses weekday–weekend structure and broader behavioural recurrence. The monthly branch serves as a long-range trend context that can stabilise the model against short-lived noise. The literature does not prescribe exactly these four windows as universally optimal, but it strongly supports the principle behind them: short-term prediction becomes more reliable when informed by explicitly separated temporal contexts [20], [43], [47].

Another benefit highlighted by the multi-scale literature is interpretability. When a model is organised by time horizon, it becomes easier to reason about what kind of information is driving the forecast. This is especially useful in research-oriented forecasting systems, where one often wants to know whether gains come from local recency, medium-range seasonality, or longer historical context. The framework also uses stochastic sampling of branch behaviour through MC Dropout to estimate predictive uncertainty [18]. This means the project not only separates temporal scales structurally but also measures their stochastic behaviour. From a literature perspective, this is an interesting bridge between multi-scale representation learning and uncertainty analysis.

At the same time, multi-scale models introduce extra design complexity. The more branches one uses, the more important fusion becomes. A poorly designed fusion layer can negate the benefits of separate temporal encoding by collapsing incompatible information too aggressively. The literature offers several fusion strategies, including concatenation followed by projection, gating, attention-based weighting, and hierarchical fusion mechanisms [43], [48], [49]. The project currently uses linear fusion before the GRU-based output stage. This is a reasonable baseline choice, but the literature suggests that learnable cross-scale attention or adaptive gating could be explored later as a more expressive alternative [48], [49].

2.5 Spatial Dependence Beyond Geography

A recurring lesson in transportation literature is that "space" should not be reduced to Euclidean closeness. Two zones may border each other yet function quite differently, while zones farther apart may be tightly linked through mobility behaviour. This is especially true in taxi systems, where trip patterns often follow business corridors, airport routes, nightlife clusters, and commuting flows rather than simple neighbourhood adjacency **rong2024odsurvey**, [13]. As a result, graph construction itself becomes a research question.

The most basic graph uses shared physical borders. This is easy to build and intuitively interpretable, but it can be too coarse. Distance-based graphs soften the binary decision by weighting nodes according to geographical separation. These are useful when movement likelihood decays smoothly with distance, but they still fail to capture directed or asymmetric travel relationships. Mobility-flow graphs improve on this by using actual trip movement between regions. Such graphs can encode strong functional dependence even when geography alone would not. Similarity graphs go one step further, linking zones with correlated historical behaviour or comparable temporal signatures [13], [22].

The present project chooses an origin-destination flow matrix as the basis for graph construction. In literature terms, this is a strong choice because it ties the graph to actual travel interaction. The system maps zones through a lookup table and constructs adjacency when flow weight is positive. This means the graph is operationally grounded in the data-generating process, not merely overlaid as a generic structure. For a taxi-demand problem, this is more meaningful than an arbitrary lattice or shapefile-only adjacency **wang2024od**, [22].

However, literature also makes clear that graph construction and graph usage should be aligned. If edge weights quantify flow intensity, then message passing ideally should distinguish stronger and weaker edges rather than treating all connections as equivalent [13], [50]. The current implementation converts the dense matrix to sparse connectivity but does not make `SAGEConv` explicitly use edge weights, even though confidence-derived edge weights can be constructed. This gap between graph semantics and graph computation remains one of the clearest methodological improvement points. It suggests at least three literature-grounded future directions: weighted graph convolution, dynamic edge reweighting based on time or confidence, and joint learning of relation strength rather than fixed thresholding [13], [51], [52].

The literature on dynamic graphs is also relevant here. Urban mobility relationships are not constant over time. Morning commute patterns differ from evening leisure travel;

weekday relations differ from weekend relations. A static OD graph captures average structure but not temporal variation in connection strength. This does not mean the current static graph is wrong, but it does mean that the thesis should frame it as an approximation. Dynamic graph models, time-conditioned edge weights, or multi-graph ensembles could later provide richer spatial realism [51], [52], [53].

2.6 Graph Neural Networks for Transportation Forecasting

Graph neural networks became central in transportation forecasting because they allow node representations to be updated through learned neighbourhood aggregation. In road traffic, the intuition is straightforward: nearby sensors influence one another through network flow. In taxi demand forecasting, the same principle applies more abstractly at the zone level: regions linked through travel behaviour or proximity may share demand dynamics or error structure. GNNs convert this intuition into learnable operators.

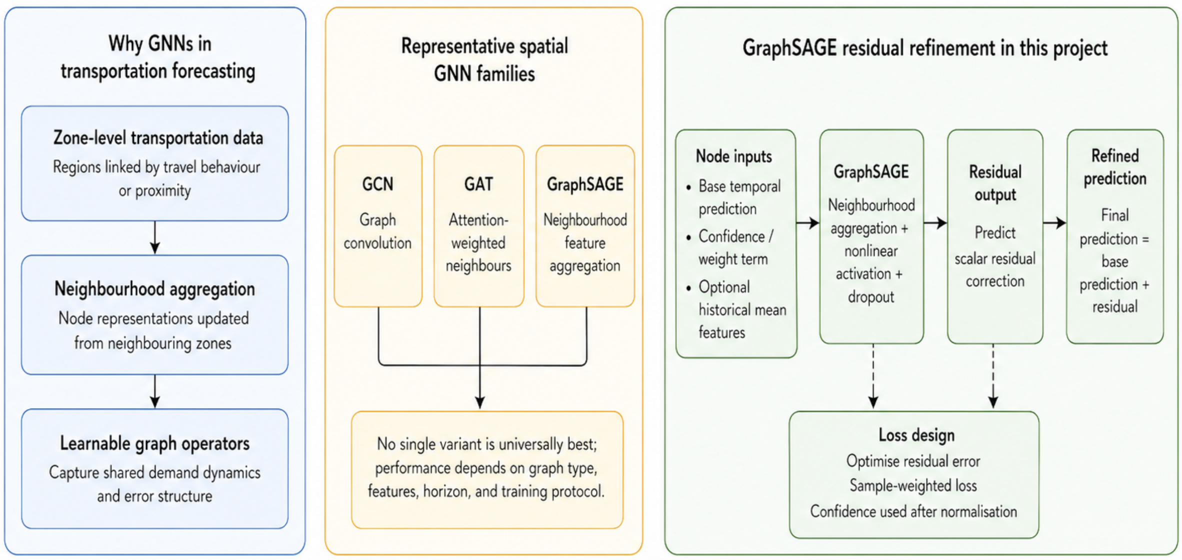

Figure 2.3: GraphSAGE-based refinement uses neighbourhood aggregation on the taxi-zone graph to learn residual corrections from base predictions, confidence weights, and optional historical features, producing a refined forecast for each zone.

Early graph convolutional methods generally relied on spectral formulations or fixed graph filters. Later spatial GNNs, including Graph Convolutional Networks, Graph Attention Networks, and GraphSAGE, shifted focus toward more flexible neighbourhood aggregation. Graph Attention Networks learn to weight neighbours through attention coefficients. GraphSAGE learns how to aggregate neighbourhood features and is often favoured for scalability and inductive behaviour. In transportation literature, no single GNN variant is universally best; performance depends heavily on graph type, feature design, horizon, and training protocol.

The current project's use of GraphSAGE is methodologically sensible because the graph stage receives heterogeneous node features rather than raw graph signals alone. Node inputs include the base temporal prediction, a confidence or weight term, and optionally historical mean features. The task is then to infer a residual correction. This is a good fit for GraphSAGE because the model can aggregate neighbour attributes and infer how the local node should be adjusted relative to its relational context. The graph model is lightweight, uses neighbourhood aggregation, nonlinear activation, dropout, and predicts a scalar residual that is added back to the base prediction. This places the model within a well-established class of graph refinement architectures.

The loss design is also noteworthy. Rather than minimising absolute prediction error directly on raw demand, the graph model optimises residual error with sample weighting. If confidence is provided, node losses are weighted accordingly after normalisation. This is an elegant compromise between full graph forecasting and purely heuristic smoothing. It allows the temporal model to carry most of the predictive burden while the graph stage focuses on structured correction where it is most reliable.

Still, the GNN literature suggests several comparisons that would strengthen the thesis. First, GraphSAGE should ideally be compared with at least one alternative such as GCN or GAT. Second, If edge weights encode flow intensity, message passing should ideally be sensitive to differences in edge strength rather than collapsing all connected pairs into a uniform neighbourhood relation. Third, graph depth should be controlled to avoid over-smoothing, especially since Taxi Zones may vary sharply even among neighbours. Fourth, the graph objective should be evaluated under a temporally valid protocol to demonstrate actual forecast refinement rather than same-snapshot fitting. These issues do not undermine the current choice of GraphSAGE, but they define the standards against which its use should be discussed in the literature review and later experiments.

2.7 Two-Stage Residual Refinement Versus End-to-End Spatiotemporal Learning

One of the most important conceptual distinctions in the literature is whether spatial and temporal modelling should be tightly intertwined or sequentially decomposed. End-to-end spatiotemporal models often dominate benchmark leaderboards because they jointly optimise all components. However, these models also make it harder to identify what each part contributes. If performance improves, is it because temporal encoding became better, because spatial propagation worked well, or because one compensates for another in a way that is hard to interpret [13], [14]?

Residual two-stage refinement offers a different philosophy. Instead of trying to learn everything simultaneously, it first obtains a competent temporal forecast and then learns structured corrections. This approach is common in calibration, post-processing, and boosting-style systems, even if it is less fashionable in some deep-learning benchmark literature. Its strength lies in modularity and interpretability. If the graph stage improves performance, one can directly measure the incremental value of spatial correction. If it fails, the base temporal model remains intact [15].

The present project clearly adopts this residual view. The graph refinement stage treats the temporal prediction as a prior and learns residual adjustment. This is theoretically attractive because taxi demand is already strongly predictable from local temporal history; the graph stage need not rediscover that structure from scratch. Instead, it can focus on neighbourhood-based consistency and cross-zone correction. In practice, this also reduces the burden on the graph model because residuals are often smoother and lower-variance than raw targets.

From a literature perspective, residual refinement also provides a natural bridge to ensemble thinking. The temporal model and graph model can be seen as experts with different strengths: one captures intra-zone temporal recurrence, the other captures inter-zone relational adjustment. Their combination is not a crude average but an ordered composition in which the graph stage edits the temporal output. This is more principled than simple ensembling and more interpretable than monolithic end-to-end fusion [14], [15].

The main methodological challenge, again, is evaluation. Residual refinement must be assessed under conditions that match true forecasting. If the graph stage is trained on the same nodes and same hour that it later "improves," then what is being measured may be correction fit rather than predictive generalisation. The literature is clear that temporal ordering matters in forecasting evaluation, and rolling-origin or prequential evaluation is generally preferred to leakage-prone estimation setups [54], [55]. Therefore, for this project to fully realise the promise of two-stage residual design, later experimentation should enforce graph-stage train/test separation across hours or rolling windows. This remains one of the most important improvement points in the current framework.

2.8 Historical Summary Features and Lightweight Context Injection

A subtle but useful idea in forecasting literature is that a downstream model does not always need access to full raw sequences if it can receive carefully chosen summary statistics. This matters because graph models are often simpler than temporal models and may become expensive if full temporal sequences are attached to every node. Historical summary features provide a compromise: they inject context compactly [13], [43].

The graph stage can receive mean demand over the previous 24 hours, 168 hours, and 720 hours. These features encode short-, medium-, and longer-term demand level without forcing the graph network to process complete time series. In literature terms, this is a form of context distillation. The temporal stage extracts rich sequential information; the graph stage then receives both the base prediction and compressed historical indicators that help judge whether a correction is plausible [32], [56].

Such features are especially useful when base predictions are noisy. Suppose two zones have the same predicted next-hour demand, but one has a high recent mean and the other has been mostly inactive. A graph model receiving recent average features can treat those cases differently. Historical features also typically require stabilisation before graph use, especially for skewed count distributions, and transformations such as logarithmic compression are well supported by the literature [32], [57].

The literature suggests that this design could be expanded. Instead of only simple means, one could include recent variance, trend slope, same-hour-last-week value, holiday flags, or uncertainty summaries. However, there is also virtue in parsimony. The current design keeps graph features compact and interpretable, which is appropriate for a first thesis implementation. This makes historical summary features a lightweight but theoretically grounded mechanism deserving empirical validation through ablation.

2.9 Confidence Fusion as an Emerging Research Direction

The notion of confidence fusion deserves special attention because it is one of the most distinctive aspects of the project. Confidence is not derived from one metric alone. Instead, it combines three components: a prior score p_i based on the historical data richness of zone i, a stability score s_i derived from the predictive variance estimated by Monte Carlo Dropout, and a neighbourhood agreement score n_i computed from the consistency between the prediction for zone i and the predictions of its graph neighbours. These components are merged through a weighted logarithmic combination to produce the final confidence score c_i:

$$\log c_i = w_p \log p_i + w_s \log s_i + w_n \log n_i,$$

where w_p, w_s, and w_n are the fusion weights assigned to the prior, stability, and neighbourhood components, respectively. Equivalently, this can be written as

$$c_i = \exp(w_p \log p_i + w_s \log s_i + w_n \log n_i)\,.$$

In the current implementation, the weights are fixed as $w_p = 0.3$, $w_s = 0.4$, and $w_n = 0.3$ [5], [18].

This design is interesting in relation to literature because it implicitly blends three epistemic perspectives. Historical sufficiency reflects how much evidence the model has seen for a zone. Stability reflects the model's internal consistency under stochastic perturbation. Neighbourhood agreement reflects external structural plausibility relative to connected zones. Very few standard forecasting pipelines combine all three explicitly, and even fewer use the result to weight graph loss. In that sense, the project is making a

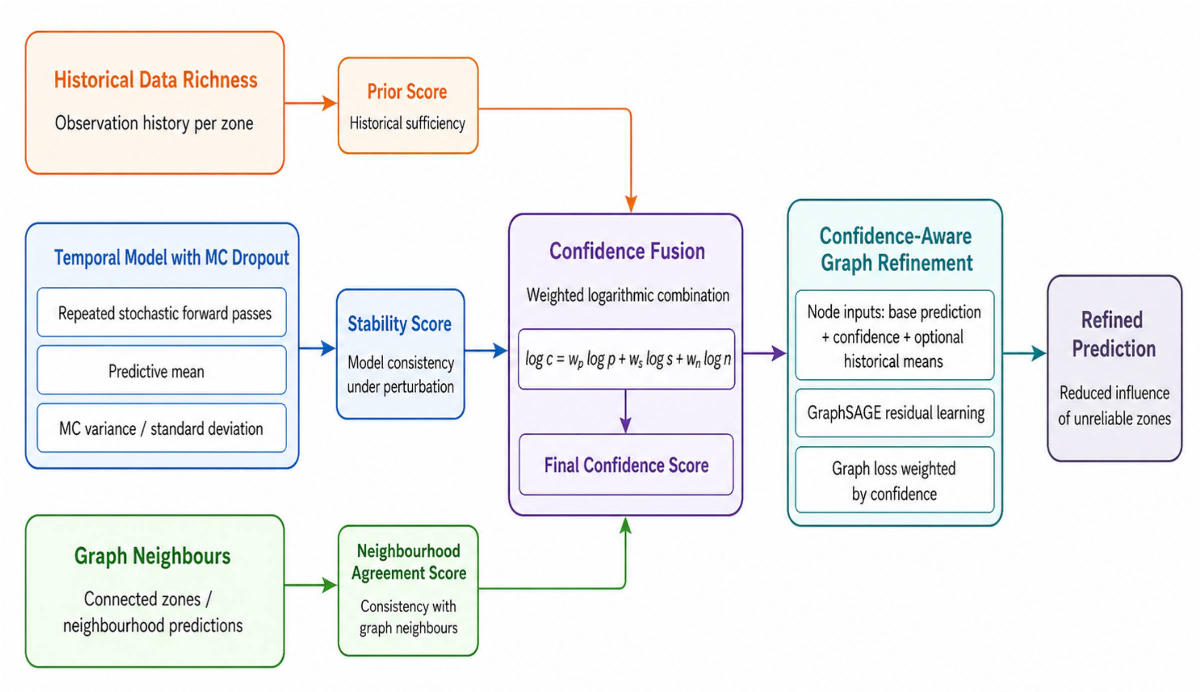

Figure 2.4: Confidence fusion integrates prior score, stability score, and neighbourhood agreement into a final confidence score for confidence-aware graph refinement.

meaningful methodological move beyond generic uncertainty estimation [16], [58].

At the same time, literature invites a critical discussion of how such fusion should be justified. The chosen weights are currently fixed. This is reasonable as a first implementation, but one may ask whether all cities, datasets, or time horizons would require the same weighting. A learnable fusion network, Bayesian calibration layer, or validation-tuned weight search could potentially improve robustness. Similarly, neighbourhood agreement may be sensitive to graph construction quality. If the graph is incomplete or overly thresholded, neighbourhood consistency may partly reflect graph design limitations rather than true prediction reliability [59], [60].

Another open question is whether confidence should be static per prediction or interactive during message passing. Current use is primarily through loss weighting, but literature suggests richer possibilities: confidence could scale outgoing messages, reweight edges, gate attention, or decide whether a node should rely more on local versus neighbour information. These extensions would move the system closer to fully confidence-aware graph learning [58], [61].

2.10 Uncertainty Estimation and Robust Forecasting

Uncertainty estimation has become increasingly important in applied machine learning because prediction error alone does not reveal confidence. In forecasting, uncertainty can arise from noise in the data, sparsity of observations, model misspecification, limited training coverage, and genuine unpredictability in the underlying process [18], [59]. For urban taxi demand, these issues are especially significant because demand variability differs dramatically across zones and across hours [5], [16]. A quiet residential zone at 3 a.m. and a business district at 8 a.m. pose different uncertainty profiles, even if their average counts are similar over some period.

The uncertainty literature commonly distinguishes several forms of uncertainty. Aleatoric uncertainty reflects irreducible data noise, while epistemic uncertainty reflects uncertainty in model parameters or structure due to limited knowledge. In practice, deep forecasting systems often use approximations to obtain at least a useful proxy for uncertainty. Monte Carlo Dropout is one of the most widely used such methods. By keeping dropout active at inference time and performing repeated stochastic forward passes, one obtains a distribution of predictions whose mean and variance can be used as approximate measures of uncertainty. This approach is attractive because it is simple to implement, computationally manageable, and easily integrated into existing deep models [18].

The project directly adopts this strategy. Repeated stochastic sampling in the temporal model yields a predictive mean and standard deviation, and branch-level stochastic behaviour can also be analysed across temporal scales. Literature supports this use of MC Dropout as a pragmatic method for uncertainty-aware forecasting, especially when the objective is not full Bayesian inference but operational reliability assessment [5], [18].

What makes the current project more interesting is that uncertainty is not used in isolation. Instead, it is fused with two further signals: historical sufficiency and neighbourhood consistency. Historical sufficiency acts as a prior, rewarding zones with richer observation history. Neighbourhood consistency compares a zone's prediction with the behaviour of its graph neighbours. This confidence fusion mechanism is an important step beyond standard literature practice. Many studies compute uncertainty but do not subsequently integrate it into downstream learning. Here, uncertainty contributes directly to node confidence, and node confidence in turn weights the graph loss, reducing the influence of unreliable regions. This moves the framework from passive uncertainty reporting to active confidence-aware modelling [59].

There are, however, several literature-based questions that this design raises. First, is the confidence weighting well calibrated? MC Dropout variance is useful, but it is not guaranteed to match true predictive reliability without calibration analysis [18], [62]. Second, do the three components deserve fixed manually chosen weights, or should the weights be learned? Third, is neighbourhood consistency genuinely independent evidence, or does it partially reuse the same graph structure later employed in refinement, potentially introducing circularity? Fourth, can confidence be used not only in the node-level loss but also in message passing, edge weighting, or feature gating? These are exactly the kinds of research gaps that literature review should highlight, because they situate the project as a strong practical design but not a final solution [59], [63].

Another relevant strand of literature concerns robust learning under noisy or sparse supervision. Confidence-aware weighting resembles ideas from curriculum learning, self-paced learning, heteroscedastic regression, and sample reweighting. The common principle is that not all observations should contribute equally to optimisation [64], [65]. In the context of graph refinement, this is especially valuable because a bad node can influence not only its own error term but also the neighbourhood information used by others. Weighting the graph loss by confidence is therefore more than a local regularisation trick; it is a strategy for controlling the propagation of unreliable information in structured prediction [58], [63].

2.11 Rolling Forecasting, Persistent Models, and Incremental Learning

A large part of forecasting literature focuses on static train/validation/test splits. While this is useful for benchmarking, it does not always reflect how deployed systems operate. In practice, many transportation platforms need forecasts repeatedly and continuously as time advances. This motivates rolling forecasting, where the target time shifts step by step and the model is reused, updated, or retrained as new data becomes available. Rolling protocols provide a more realistic picture of operational performance because they expose temporal drift, changing uncertainty, and the practical cost of repeated prediction [54], [55].

The present project places rolling prediction at the centre of the overall framework. In the implementation, the target timestamp advances hour by hour, predictions are generated for each Taxi Zone, confidence scores are recalculated, graph refinement is applied, and both per-hour and overall metrics are saved. This design is highly consistent with deployment-aware forecasting practice and should be regarded as a genuine strength of the work. Rather than evaluating a single static fit, the thesis studies a forecasting pipeline intended to mimic repeated operational use under continuously advancing time [54].

Within such rolling settings, one important design choice concerns how much retraining should be performed. Full retraining at every step is often computationally expensive, particularly in large multi-zone systems. Persistent models, which are trained once and then reused, offer a much cheaper alternative, but they may gradually become stale under temporal drift. Incremental learning lies between these two extremes, updating existing parameters using newly arrived data while preserving previously learned structure [66], [67]. In the present project, the repository is no longer limited to persistent checkpoint reuse alone. An incremental training variant has been added so that previously trained per-zone models can be lightly updated when new rolling windows become available. However, in the current rolling script, persistent checkpoint reuse remains the default configuration, meaning that incremental adaptation exists as an implemented extension rather than the baseline operating mode.

This distinction is important for how the framework should be interpreted academically. The persistent version emphasises computational efficiency and stable checkpoint reuse, whereas the incremental version is intended to improve adaptability under temporal drift. In long rolling horizons, this is especially valuable because urban demand patterns may evolve gradually and a fully fixed temporal model may lose stability over time. The incremental extension therefore strengthens the practical realism of the system by providing a mechanism through which newly observed information can be absorbed without the cost of full retraining at every step. From a literature perspective, the project can be understood as occupying an intermediate position between fully static checkpoint reuse and expensive repeated retraining, preserving tractability while enabling a path toward continual adaptation [54], [66].

This incremental extension is particularly relevant to the present study because it addresses one of the main weaknesses of purely persistent forecasting systems, namely the potential deterioration of stability over long rolling horizons. By allowing the temporal model to incorporate newly available observations through lightweight updates, the framework is better positioned for realistic deployment scenarios in which mobility demand evolves continuously over time. At the same time, this should be described carefully in the thesis: incremental adaptation is now supported in the codebase, but it is not yet the default experimental setting in the rolling driver script [66].

Another important issue raised in the rolling forecasting literature concerns temporal integrity and information leakage. Any forecasting pipeline that reconstructs windows, scales inputs, or derives graph-related features must ensure that future information does not leak into the current prediction step. In the present framework, the temporal forecasting stage is designed to preserve historical integrity by constructing inference windows from data available up to the forecasting context end and by fitting the scaler only on history prior to the target hour. This is methodologically sound and aligns with standard forecasting practice [54], [55].

However, the graph refinement stage still raises a more cautious generalisation question. The GraphSAGE module learns residual corrections from the same hourly batch on which refined predictions are then evaluated, and although confidence-weighted losses are applied, this protocol may overestimate the true cross-temporal generalisation gain of the graph stage. In addition, the current GraphSAGE implementation uses standard `SAGEConv` layers over the graph topology, while confidence is injected through node-level weighting rather than explicit edge-weighted message passing. As a result, the temporal forecasting stage is close to a valid rolling protocol, but the graph refinement stage still requires stronger temporally separated validation if stronger claims about generalisation are to be made. This asymmetry should be discussed openly in the thesis, because it determines how strong the empirical evidence really is for the contribution of the graph component [54], [55].

2.12 Critical Gaps in the Existing Literature and Their Relevance to the Project

A good literature review should not only catalogue prior methods; it should identify unresolved tensions that justify the thesis. In the present case, five such tensions stand out.

The first is the tension between temporal richness and local heterogeneity. Shared spatiotemporal models can exploit cross-zone information efficiently, but they may underrepresent zone-specific dynamics. Per-zone models preserve local specificity but are harder to scale and may be weaker in sparse zones. The project responds by using per-zone temporal models combined with a spatial refinement stage, effectively splitting local learning and relational correction into separate tasks [5], [43].

The second is the tension between structural sophistication and interpretability. End-to-end spatiotemporal networks can be powerful, but they are often difficult to diagnose. A two-stage residual framework is more interpretable because one can separately inspect base predictions, uncertainty, confidence, and refinement gains. This aligns with a growing literature preference for modular systems in operational contexts [14], [15].

The third is the tension between graph availability and graph utilisation. Many studies build elaborate graphs but do not fully exploit edge semantics. The current project already identifies this issue internally: OD edge weights exist but are not explicitly used in message passing. This is therefore not merely a future coding task; it corresponds to a broader literature problem of whether graph connectivity should be binary, weighted, learned, dynamic, or confidence-modulated [13], [59].

The fourth is the tension between uncertainty estimation and uncertainty usage. Prior work often stops after reporting predictive variance. The project goes further by fusing uncertainty with historical richness and neighbourhood agreement into a confidence score used in graph optimisation. This is a meaningful step, but literature suggests room for deeper treatment: calibration, learned confidence fusion, probabilistic loss design, and confidence-aware message passing remain open [16], [59].

The fifth is the tension between benchmark evaluation and real deployment realism. The project's rolling pipeline is well aligned with practical literature, especially after incorporating incremental temporal updating for long-horizon forecasting. At the same time, the graph stage still needs a stronger temporally separated evaluation protocol if it is to claim generalisable improvement. Thus, the principal methodological gap is no longer the absence of incremental adaptation in the temporal stage, but the need for more rigorous validation of graph-based refinement under temporally realistic generalisation settings [54], [55], [66].

2.13 Synthesis: Positioning the Present Study

Taken together, the literature suggests that an effective urban taxi demand forecasting system should satisfy several requirements simultaneously. It should capture nonlinear temporal dependence across multiple timescales; represent spatial interaction in a graph-compatible form; remain robust to data sparsity and regional heterogeneity; provide some measure of predictive reliability; and function under a realistic rolling prediction protocol [5], [13], [43], [54]. Few simple methods satisfy all of these requirements at once. Traditional statistical models offer interpretability but weak nonlinear and spatial representation. Pure temporal deep models improve sequence learning but ignore relational correction. End-to-end spatiotemporal networks can model both dimensions jointly, but often at the cost of modularity, interpretability, and deployment flexibility [14], [15]. Uncertainty estimation is frequently included only as an output statistic rather than a driver of later learning decisions [59].

The present study occupies a distinct niche within this landscape. It proposes not an entirely new primitive model class, but a coherent combination of ideas motivated by gaps in prior work. Its temporal stage is multi-scale and explicitly tailored to zone-level demand heterogeneity. Its uncertainty mechanism is operationalised into confidence scores rather than left unused. Its graph stage performs residual correction over an OD-based relational structure rather than predicting from scratch. Its pipeline runs in a rolling setting that combines persistent checkpoint reuse with incremental temporal adaptation, thereby strengthening long-horizon forecasting stability while maintaining computational practicality.

At the same time, the literature review makes clear that the project is not methodologically complete. To strengthen its academic contribution, later chapters should pay special attention to graph evaluation protocol, explicit edge-weight usage, and ablation over confidence components, while also assessing how incremental updating contributes to long-horizon forecasting stability [54], [55], [59]. These are not peripheral enhancements; they are the most direct ways in which the project can move from a strong engineering pipeline toward a more rigorous research contribution.

In summary, the reviewed literature provides a strong rationale for the architecture chosen in this thesis. It supports the move from traditional zone-wise forecasting to deep multi-scale temporal learning, from isolated local prediction to graph-based spatial refinement, from point prediction to confidence-aware modelling, and from static evaluation to rolling deployment-style assessment with continual adaptation [13], [43], [54]. It also provides a principled basis for critiquing the current implementation and defining the next methodological questions. The thesis therefore stands on firm conceptual ground: it is responding to genuine open issues in spatiotemporal forecasting rather than assembling components arbitrarily. The following methodology chapter will build on this literature context by formalising the problem and detailing how the proposed two-stage, confidence-weighted multi-scale framework is constructed.

Chapter 3

Methodology

3.1 Overview

This chapter presents the methodological design of the proposed forecasting system for hourly taxi demand prediction over New York City Taxi Zones. The aim of the method is not merely to fit a single prediction model, but to construct an end-to-end forecasting pipeline that reflects the practical requirements of real deployment. The framework is therefore organised as a rolling, two-stage architecture. In the first stage, each Taxi Zone is modelled independently by a multi-scale temporal network that predicts demand for the next hour. In the second stage, the resulting zone-level predictions are refined by a graph neural network that introduces spatial consistency using an origin-destination flow graph. Between these two stages, uncertainty-aware confidence estimation is used to quantify the reliability of each zone-level prediction and to control how strongly the graph refinement stage should trust each node.

Methodologically, the framework combines four central ideas. First, it treats urban mobility demand as a temporal process with meaningful structure at multiple scales, rather than as a sequence that can be represented adequately by a single recent window. Second, it models each zone separately at the temporal stage, thereby preserving local heterogeneity and avoiding the assumption that all urban regions should share identical temporal parameters. Third, it uses Monte Carlo Dropout and additional reliability signals to build a node-level confidence mechanism that can distinguish between stable and unstable predictions. Fourth, it treats spatial learning as a residual correction problem rather than an entirely separate prediction task, allowing graph refinement to improve an existing temporal prior rather than replacing it.

The methodology has been designed to support three goals simultaneously. The first is predictive accuracy, measured by how closely the forecasts match the observed number of trips in each zone at the target hour. The second is robustness, especially under sparse data or unstable temporal dynamics, where uncertainty and graph structure may be particularly informative. The third is operational realism, achieved through a rolling evaluation protocol in which predictions are repeatedly generated as time advances hour by hour. The remainder of this chapter is organised as follows. Section 3.2 formalises the forecasting problem and defines the rolling prediction setting. Section 3.3 describes the data representation, preprocessing operations, and the construction of temporal sequences and graph structure. Section 3.4 presents the first-stage multi-scale temporal model, including the architecture of the branch encoders, feature fusion, prediction mechanism, and training strategy. Section 3.5 explains the uncertainty estimation procedure and the construction of the confidence score that links the temporal and graph stages. Section 3.6 introduces the GraphSAGE residual refinement model, its inputs, learning objective, and prediction logic. Section 3.7 describes the rolling protocol and the interaction between checkpoint reuse, optional incremental adaptation, and repeated forecasting. Section 3.8 concludes the chapter by discussing the methodological rationale, strengths, and deliberate limitations of the current design.

3.2 Problem Setup

3.2.1 Forecasting Objective

Let the urban system be partitioned into a set of Taxi Zones

$$\mathcal{Z} = \{z\_1, z\_2, \dots, z\_N\},$$

where each zone corresponds to a predefined administrative or operational region used in the taxi dataset. For each zone z_i, the raw data consist of time-stamped trip records with pickup locations. These records are aggregated into hourly counts, producing a

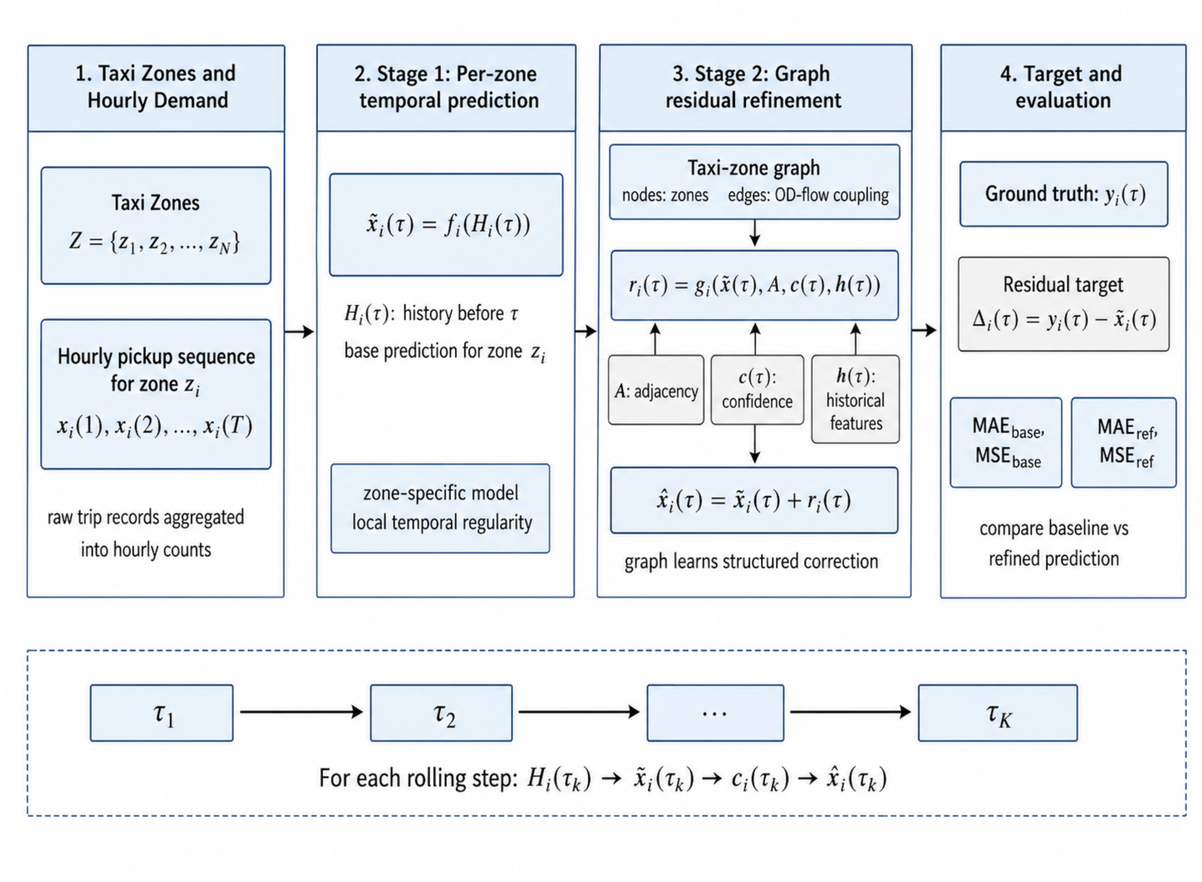

Figure 3.1: Illustration of the problem setup used in this thesis. Hourly taxi pickup counts are modelled at the zone level, where a per-zone temporal predictor first generates a base forecast and a graph refinement stage then learns a residual correction on the taxi-zone graph. The framework is evaluated under a rolling forecasting protocol using ground-truth hourly counts and step-wise error metrics.

univariate temporal sequence

$$x\_i(1), x\_i(2), \ldots, x\_i(T),$$

where $x\_i(t)$ denotes the number of taxi pickups observed in zone z_i during hour t.

The central forecasting task is next-step prediction at hourly granularity. Given the observed history for zone z_i up to time t, the system aims to predict

$$\hat{x}\_i(t+1),$$

the demand for the next hour. More generally, for a target timestamp τ, the prediction problem can be written as

$$\hat{x}\_i(\tau) = f\_i\Big(\mathcal{H}\_i(\tau)\Big),$$

where $\mathcal{H}\_i(\tau)$ denotes the history available for zone z_i strictly before τ, and $f\_i(\cdot)$ denotes the temporal predictor associated with that zone.

This thesis does not formulate the temporal stage as a direct multivariate city-wide forecasting model. Instead, the base prediction stage is defined per zone. That is, each zone maintains its own model parameters and its own training history, while spatial interaction is handled later by a separate graph refinement stage. The rationale is that a zone-level forecasting problem has highly local temporal regularities, often shaped by specific functional roles such as residential demand, airport traffic, nightlife, or commuter concentration. Preserving this heterogeneity is methodologically desirable because it reduces the risk that a single monolithic temporal model collapses distinct demand processes into one shared representation.

3.2.2 Two-Stage Forecasting Formulation

The overall method is formulated as a two-stage architecture.

In the first stage, a temporal base predictor produces an initial forecast for each zone:

$$\tilde{x}\_i(\tau) = f\_i\Big(\mathcal{H}\_i(\tau)\Big).$$

This prediction is accompanied by an uncertainty estimate and several auxiliary reliability indicators. The collection of zone-level temporal predictions across all zones forms a vector

$$\tilde{\mathbf{x}}(\tau) = [\tilde{x}\_1(\tau), \tilde{x}\_2(\tau), \ldots, \tilde{x}\_N(\tau)]^\top.$$

In the second stage, these temporal predictions are treated as priors on a graph whose nodes correspond to Taxi Zones and whose edges reflect mobility coupling derived from origin-destination flows. The graph model does not predict demand from scratch. Instead, it learns a residual correction:

$$r\_i(\tau) = g\_i\Big(\tilde{\mathbf{x}}(\tau), \mathbf{A}, \mathbf{c}(\tau), \mathbf{h}(\tau)\Big),$$

where $\mathbf{A}$ denotes the adjacency structure of the graph, $\mathbf{c}(\tau)$ denotes node-level confidence values, and $\mathbf{h}(\tau)$ denotes supplementary historical features. The final refined prediction is therefore

$$\hat{x}\_i(\tau) = \tilde{x}\_i(\tau) + r\_i(\tau).$$

This residual formulation is important. It assumes that the temporal stage already captures most of the signal that is specific to each zone, while the graph stage contributes structured corrections that account for spatial relationships and cross-zone consistency. The graph model is therefore tasked with learning what the temporal stage missed, rather than rediscovering the entire demand signal from the graph alone. This reduces modelling burden and makes the role of the graph stage more interpretable.

3.2.3 Rolling Forecasting Setting

The forecasting task is evaluated under a rolling protocol rather than a single static train/test split. Let $\tau\_1, \tau\_2, \ldots, \tau\_K$ denote a sequence of target timestamps separated by one hour. At each target hour τ_k, the system reconstructs the temporal input windows, generates predictions for all eligible zones, computes confidence scores, applies graph refinement, and stores both zone-level outputs and aggregate error metrics.

Formally, for each rolling step k, the procedure is

$$\mathcal{H}\_i(\tau\_k) \to \tilde{x}\_i(\tau\_k) \to c\_i(\tau\_k) \to \hat{x}\_i(\tau\_k),$$

for all zones z_i. This setup is intended to mimic a realistic forecasting workflow, where predictions must be generated repeatedly as time moves forward. It is also methodologically useful because it exposes temporal drift, variation in uncertainty, and the stability of forecasting performance across consecutive operating hours.

3.2.4 Target Variables and Evaluation Quantities

The primary target variable is the observed hourly taxi pickup count in each zone. Let $y\_i(\tau)$ denote the ground-truth count for zone z_i at target hour τ. The temporal baseline prediction is $\tilde{x}\_i(\tau)$, and the graph-refined prediction is $\hat{x}\_i(\tau)$.

At the graph stage, the supervised signal is the residual

$$\Delta\_i(\tau) = y\_i(\tau) - \tilde{x}\_i(\tau),$$

rather than the raw target $y\_i(\tau)$. This is consistent with the residual refinement philosophy described above.

For evaluation, the main scalar error measures are mean absolute error and mean squared error. For a set of valid zones $\mathcal{V}\_\tau$ at time τ, the baseline MAE is

$$\text{MAE\_base}(\tau) = \frac{1}{|\mathcal{V}\_\tau|} \sum\_i \in \mathcal{V}\_\tau |\tilde{x}\_i(\tau) - y\_i(\tau)|,$$

and the refined MAE is

$$\text{MAE\_ref}(\tau) = \frac{1}{|\mathcal{V}\_\tau|} \sum\_i \in \mathcal{V}\_\tau |\hat{x}\_i(\tau) - y\_i(\tau)|.$$

Similarly, the corresponding mean squared errors are

$$\text{MSE\_base}(\tau) = \frac{1}{|\mathcal{V}\_\tau|} \sum\_i \in \mathcal{V}\_\tau (\tilde{x}\_i(\tau) - y\_i(\tau))^2$$

and

$$\text{MSE\_ref}(\tau) = \frac{1}{|\mathcal{V}\_\tau|} \sum\_i \in \mathcal{V}\_\tau (\hat{x}\_i(\tau) - y\_i(\tau))^2.$$

These metrics are stored at each rolling step and can also be aggregated across the full rolling horizon to provide overall performance summaries.

3.3 Data Representation and Preprocessing

3.3.1 Raw Data and Hourly Aggregation

The raw taxi dataset contains individual trip records with at least two essential attributes for this work: pickup timestamp and pickup zone identifier. Since the objective is hourly zone-level demand forecasting, the raw event stream is transformed into a panel of hourly counts. Each trip record contributes one count to the corresponding zone and hour, and all records are aggregated by hour after flooring the timestamps to hourly precision.

For each zone z_i, this produces a sparse sequence of observed counts over time. However, a forecasting model requires dense temporal input, including explicit zeros for hours in which no pickups were recorded. Therefore, the methodology constructs dense hourly timelines over the relevant historical range. Missing hours are reintroduced with count zero, ensuring that the sequence used by the temporal model preserves both activity and inactivity patterns. This is especially important in sparse zones, where the absence of events is itself informative [5], [32].

3.3.2 Zone Filtering and Data Validity

Not all Taxi Zones are equally suitable for modelling. In practical taxi datasets, some zones may correspond to special-purpose regions, missing lookup entries, irregular traffic patterns, or insufficient data coverage. The methodology therefore allows the exclusion of selected zones during preprocessing. This prevents invalid or unstable zones from contaminating the temporal model, confidence construction, or graph refinement stage.

The filtering step is intentionally performed early in the pipeline. Once excluded, a zone does not contribute to temporal training, rolling evaluation, or graph refinement. This ensures consistency across all later stages. From a methodological point of view, such filtering reflects a realistic data curation step in applied mobility forecasting, where certain regions may be unusable because of missing metadata, unstable counts, or lack of meaningful demand [32].

3.3.3 Temporal Granularity and Historical Windows

The system operates at hourly granularity throughout. This choice balances temporal detail and computational tractability. Minute-level modelling would be too sparse and noisy for many zones, while daily aggregation would remove the within-day structure that is central to taxi demand [20], [25]. Hourly aggregation preserves immediate fluctuations, daily cycles, and longer periodic patterns in a form that can be handled efficiently by recurrent and attention-based models.

A key methodological decision is to represent the historical sequence at multiple time scales. Instead of feeding one fixed look-back window into a single sequence encoder, the framework constructs four temporal contexts:

- a short-term context of the most recent 1 hour,
- a daily context of the most recent 24 hours,
- a weekly context of the most recent 168 hours,
- and a monthly context of the most recent 720 hours.

These windows are not arbitrary. The 1-hour view captures immediate local momentum. The 24-hour view reflects intra-day structure and recent daily periodicity. The 168-hour view introduces weekly repetition and the weekday–weekend distinction. The 720-hour view provides roughly one month of context and can represent slower trend evolution. The methodology assumes that these time scales contain complementary information and that explicitly encoding them is preferable to relying on a single model to infer all relevant horizons from one undifferentiated sequence [20], [43], [47].

3.3.4 Construction of Dense Zone Series

For each zone and each forecasting context end time, a dense hourly series is constructed over the relevant history. Let τ denote a target hour and let the forecasting horizon be one step. Then the last available hour before prediction is

$$\tau^- = \tau - 1 \text{ hour}.$$

The dense zone series is built up to τ^-. To provide sufficient context for the monthly branch and for sliding-window training, the historical extraction range is extended beyond the basic 720-hour input window. In effect, the method reconstructs a sufficiently long dense hourly sequence ending at τ^-, allowing both model training windows and the final inference window to be assembled without any future leakage.

For sparse zones, this dense reconstruction is crucial. Without it, the model would see only event times and could not distinguish between a genuinely inactive period and missing data. With explicit zero filling, the temporal encoders can learn the difference between peak hours, moderate demand, and prolonged inactivity [5].

3.3.5 Scaling and Temporal Integrity

Neural sequence models are sensitive to the scale of their inputs. Since demand counts vary considerably across zones, the framework scales each zone independently before building model inputs. A Min–Max scaling transformation is applied:

$$x\_i^{\text{scaled}}(t) = \frac{x\_i(t) - x\_i^{\text{min}}}{x\_i^{\text{max}} - x\_i^{\text{min}}},$$

with suitable safeguards for degenerate cases such as constant sequences. The use of zone-specific scaling matches the per-zone forecasting design. A high-demand airport zone and a low-demand peripheral zone may have very different count ranges, and scaling them jointly would distort the learning problem.

A methodological requirement here is temporal integrity. The scaler must be fitted only on historical observations available before the target hour. Let τ be the prediction time. Then the scaler for forecasting τ is fitted on data strictly earlier than τ. This ensures that neither the scale parameters nor the input windows leak future information. This seemingly technical detail is methodologically important because improper scaling is a common source of hidden leakage in time-series experiments [54], [55].

3.3.6 Sliding-Window Sample Generation

After dense reconstruction and scaling, training samples are produced by sliding windows over the historical series. Let $L = 720$ denote the total monthly context length and let the prediction horizon be $H = 1$. For each valid starting position s, the full input window is

$$[x\_i^{\text{scaled}}(s), x\_i^{\text{scaled}}(s+1), \ldots, x\_i^{\text{scaled}}(s+L-1)],$$

and the corresponding target is

$$y\_i^{\text{scaled}}(s) = x\_i^{\text{scaled}}(s+L).$$

From the full monthly window, the shorter branch windows are extracted from its most recent part:

$$\mathbf{x}^{1h}\_i(s) = [x\_i^{\text{scaled}}(s+L-1)],$$
$$\mathbf{x}^{1d}\_i(s) = [x\_i^{\text{scaled}}(s+L-24), \dots, x\_i^{\text{scaled}}(s+L-1)],$$
$$\mathbf{x}^{1w}\_i(s) = [x\_i^{\text{scaled}}(s+L-168), \dots, x\_i^{\text{scaled}}(s+L-1)],$$
$$\mathbf{x}^{1m}\_i(s) = [x\_i^{\text{scaled}}(s), \dots, x\_i^{\text{scaled}}(s+L-1)].$$

This construction means that all branches predict the same next-hour target but view the historical sequence at different temporal horizons. The branch inputs are therefore temporally aligned and complementary.

3.3.7 Graph Construction from Origin–Destination Flow

The spatial component of the framework is built from an origin–destination flow matrix rather than a purely geographic adjacency matrix. Each node represents a Taxi Zone, and an edge is created between two zones if the corresponding entry in the OD flow matrix indicates non-zero interaction. Let $\mathbf{A} \in \mathbb{R}^{N \times N}$ denote the zone-by-zone flow matrix. The graph connectivity used by the graph neural network is derived as

$$A\_ij^{\text{bin}} = \begin{cases} 1, & \text{if } A\_ij > 0, \\ 0, & \text{otherwise.} \end{cases}$$

This binary adjacency is then converted into sparse edge-index form for graph learning. Methodologically, the choice of OD flow over geographic adjacency is important. Geographical closeness does not necessarily correspond to mobility coupling. Two neighbouring zones may have weak travel exchange, while two zones that are geographically farther apart may exhibit strong interaction because of commuting, airport trips, or major transit connections [13], [22]. The OD graph therefore encodes functional linkage rather than mere cartographic proximity.

At the same time, the current graph construction simplifies the original flow matrix by thresholding it into connectivity. The existence of interaction is retained, but the exact edge magnitude is not explicitly exploited inside the current GraphSAGE message-passing operator.

3.3.8 Auxiliary Historical Features for Graph Refinement

In addition to the base temporal prediction and the confidence score, the graph refinement stage receives auxiliary node-level history summaries. These features are simple averages of the zone demand over selected windows before the target hour:

$$
\begin{aligned}
m\_i^{24h}(\tau) &= \frac{1}{24}\sum\_h = 1^{24} x\_i(\tau - h), \\
m\_i^{168h}(\tau) &= \frac{1}{168}\sum\_h = 1^{168} x\_i(\tau - h), \\
m\_i^{720h}(\tau) &= \frac{1}{720}\sum\_h = 1^{720} x\_i(\tau - h).
\end{aligned}
$$

These features serve as low-dimensional summaries of recent demand history. Their role is different from that of the multi-scale temporal base model. The temporal model already processes raw sequences to generate the baseline forecast. The graph stage, however, operates on a single graph snapshot at the target hour and therefore benefits from concise contextual descriptors that summarise whether the current zone is typically busy or quiet over short, medium, and long horizons [32], [43]. Before being used by the graph network, these history features are stabilised by replacing missing values and applying a $\log(1 + x)$ transform [57].

3.4 Stage One: Multi-Scale Temporal Forecasting

3.4.1 Motivation for a Multi-Branch Design

The first stage of the method is a multi-scale temporal predictor trained independently for each zone. The central design hypothesis is that urban taxi demand cannot be represented adequately by one temporal horizon alone. A purely short-window model may react to recent fluctuations but fail to preserve weekly repetition or slower trends. A purely long-window model may encode broader context but blur immediate local shifts. The methodology therefore uses a multi-branch design in which distinct encoders are assigned to different temporal horizons and their representations are fused before the final prediction step [20], [43], [47].

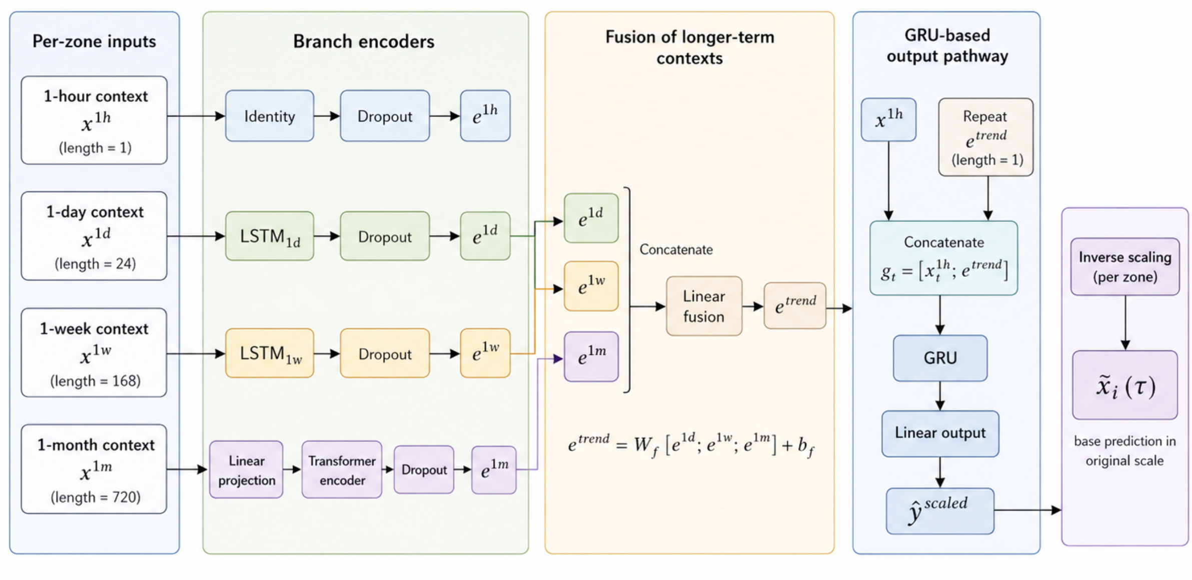

Figure 3.2: Stage-one temporal forecasting pipeline for each taxi zone. Multi-scale inputs from 1 hour, 1 day, 1 week, and 1 month are encoded by branch-specific modules, fused into a longer-term trend representation, and combined with the short-term signal in a GRU-based output pathway to generate the next-hour demand prediction in the original scale.

This architecture is also shaped by the practical nature of the forecasting target. Because the task is one-step-ahead hourly prediction, the system does not need a full sequence-to-sequence decoder. Instead, it needs a compact but expressive representation of recent dynamics that can map directly to the next hour. The branch encoders and fusion module are therefore designed to extract complementary embeddings rather than to reconstruct the whole future trajectory.

3.4.2 Per-Zone Temporal Modelling

Each Taxi Zone is assigned its own temporal forecasting model. The parameter set for each temporal model is zone-specific. The model is trained on the historical sequence of that zone only, and its checkpoint is stored independently from the checkpoints of other zones.

This lets each model specialise to the local temporal signature of its zone, avoids forcing all regions into one shared temporal representation, and supports independent checkpoint reuse with optional incremental updates. The cost is that the system must maintain many separate models rather than one global model, but the framework accepts this trade-off because local specialisation is prioritised at the temporal stage [5], [43].

3.4.3 Branch Inputs

For each zone, a training or inference example consists of four temporally aligned inputs:

$$\mathbf{x}^{1h}, \quad \mathbf{x}^{1d}, \quad \mathbf{x}^{1w}, \quad \mathbf{x}^{1m}.$$

These correspond to the 1-hour, 1-day, 1-week, and 1-month contexts described earlier. Each input is represented as a univariate sequence with one feature channel.

The monthly branch retains the full 720-hour window, the daily and weekly branches use the most recent 24 and 168 elements, and the short branch uses only the most recent hour. Although a length-1 branch is minimal, it serves as the explicit carrier of immediate local momentum when combined with the fused longer-term trend representation inside the GRU pathway.

3.4.4 Daily and Weekly Encoders

The daily and weekly branches are encoded with LSTM networks. For the daily branch, the sequence $\mathbf{x}^{1d}$ is passed through an LSTM:

$$\mathbf{h}^{1d} = \text{LSTM\_}1d(\mathbf{x}^{1d}),$$

and the final hidden state is used as the branch embedding:

$$\mathbf{e}^{1d} = \mathbf{h}^{1d}\text{\_final}.$$

Similarly, the weekly branch is encoded as

$$\mathbf{e}^{1w} = \text{LSTM\_}1w(\mathbf{x}^{1w})\text{\_final}.$$

The use of LSTMs for the 24-hour and 168-hour windows is appropriate because these horizons require ordered-memory modelling but remain short enough for stable recurrent encoding. The daily branch captures within-day structure, while the weekly branch captures weekday–weekend repetition and broader weekly routines [13], [37].

3.4.5 Monthly Encoder

The longest branch, representing roughly one month of hourly history, is encoded differently. Instead of another recurrent network, the methodology uses a Transformer-based encoder after projecting the scalar input into a hidden space. Let the monthly sequence be $\mathbf{x}^{1m}$. It is first mapped by a linear projection:

$$\mathbf{u}^{1m}\_t = \mathbf{W}\_p x^{1m}\_t + \mathbf{b}\_p,$$

producing a hidden representation at each time step. The projected sequence is then processed by a Transformer encoder:

$$\mathbf{H}^{1m} = \mathrm{Transformer}(\mathbf{U}^{1m}),$$

and the final time-step output is taken as the monthly branch embedding:

$$\mathbf{e}^{1m} = \mathbf{H}^{1m}\_\mathrm{final}.$$

The methodological reason for assigning the longest horizon to a Transformer is that self-attention can represent long-range dependencies without forcing information to flow through a fully recurrent chain. Over a 720-hour window, slow drifts and distant periodic cues may be easier to retain in an attention-based representation than in a standard recurrent state alone [41], [42], [68].

3.4.6 Branch-Level Dropout and Representation Stability

After the branch embeddings are obtained, dropout is applied independently to each branch representation. This serves two purposes. First, it acts as regularisation during training by discouraging over-reliance on any single hidden component. Second, and more importantly for this thesis, it enables Monte Carlo Dropout during inference. Because dropout remains active when stochastic forward passes are required, the variability of the resulting predictions and branch embeddings can be used to estimate uncertainty and branch-level stability [18].

The methodology therefore treats dropout not only as a standard regularisation device but as an integral component of the confidence estimation strategy. This dual role links the temporal predictor directly to the uncertainty and confidence mechanism described later [18].

3.4.7 Fusion of Longer-Term Temporal Contexts

The daily, weekly, and monthly embeddings are concatenated and passed through a linear fusion layer:

$$\mathbf{e}^{\text{trend}} = \phi\Big(\mathbf{W}\_f[\mathbf{e}^{1d}; \mathbf{e}^{1w}; \mathbf{e}^{1m}] + \mathbf{b}\_f\Big),$$

where $\phi(\cdot)$ denotes the hidden nonlinearity implied by subsequent processing and the fusion layer compresses the concatenated multi-scale trend information into a single trend vector.

This fused representation is conceptually important. It is not intended to replace the short-term branch. Instead, it acts as a context vector summarising medium- and long-term behaviour. The design assumes that immediate next-hour prediction should be driven by both local momentum and a broader temporal trend state. The fusion layer therefore converts the three longer-horizon branch embeddings into a unified condition that can support final short-term prediction [47], [48].

3.4.8 GRU-Based Output Pathway

The final prediction pathway is based on a GRU. The fused trend vector is repeated across the short sequence length and concatenated with the 1-hour branch input:

$$\mathbf{g}\_t = [x^{1h}\_t; \mathbf{e}^{\text{trend}}],$$

which forms the GRU input. Since the short branch length is one in the current implementation, the GRU effectively receives a compact representation that combines the most recent observed demand value with the aggregated trend embedding extracted from the longer branches.

The GRU then produces a hidden representation

$$\mathbf{h}^{1h}\_\text{gru} = \text{GRU}(\mathbf{g}),$$

and the final prediction is obtained via a linear output layer:

$$\hat{y}^{\text{scaled}} = \mathbf{W}\_o\mathbf{h}^{1h}\_\text{final} + b\_o.$$

This design is slightly unusual compared with standard recurrent forecasting architectures, because the GRU is not asked to process the entire long history directly. Instead, it acts as the final short-term decision mechanism conditioned on fused longer-term context. Methodologically, this makes the architecture modular: long-term dependencies are handled by branch encoders, while the GRU focuses on converting recent local state and broader trend information into a one-step-ahead prediction [17], [41].

3.4.9 Prediction in Original Scale

The model outputs predictions in scaled space, because the training samples are built after Min–Max normalisation. To obtain demand in the original count space, the inverse scaling transform is applied:

$$\tilde{x}\_i(\tau) = \text{Scale}^{-1}(\hat{y}^{\text{scaled}}\_i(\tau)).$$

Because scaling is fitted separately for each zone and each rolling context using only past observations, the inverse transform remains consistent with the temporally valid training and inference setup.

3.4.10 Training Objective for the Temporal Stage

The temporal stage is trained as a supervised next-step regressor. For a batch of training windows from one zone, let the scaled predictions be $\hat{y}\_b$ and the scaled targets be y_b. The main loss is mean squared error:

$$\mathcal{L}\_\text{temp} = \frac{1}{B}\sum\_b = 1^B(\hat{y}\_b - y\_b)^2.$$

The code structure allows for auxiliary logic related to branch uncertainty, but the dominant supervised objective is still the main one-step regression loss. This choice is reasonable because the ultimate target is a scalar hourly count and the branch encoders are all optimised jointly to minimise next-step prediction error.

3.4.11 Optimisation and Early Stopping

Temporal training uses Adam optimisation with a fixed learning rate and an early-stopping mechanism controlled by patience. Let $\ell^{(e)}$ denote the training loss at epoch e. The best observed loss is tracked, and training terminates if no improvement is seen over a specified number of epochs. This prevents unnecessary optimisation once the zone-specific model has converged and reduces the chance of overfitting, especially in zones with limited training variation.

Because the temporal stage is performed independently per zone, training efficiency is important. Early stopping reduces wasted optimisation and makes checkpoint generation more practical across many zones. This is one of the reasons a relatively simple next-step objective and moderate hidden dimension are sufficient in the current framework.

3.4.12 Checkpointing and Model Persistence

After training, each zone model is saved together with its scaler and metadata describing the latest context end included in training. This creates a persistent forecasting state for that zone. During later rolling steps, if the model already exists, the framework can reuse it rather than retraining from scratch. The methodological benefit is that rolling prediction becomes much closer to a real operational scenario, where models are not typically rebuilt every hour without reason [54], [66].

The persistent design also enables a clean distinction between the baseline persistent mode and the optional incremental mode. In persistent mode, the saved model is reused as-is. In incremental mode, the checkpoint can be updated using new observations when new rolling windows arrive. This will be discussed in Section 3.7.

3.5 Uncertainty Estimation and Confidence Construction

3.5.1 Role of Uncertainty in the Framework

A key methodological feature of the framework is that predictive uncertainty is not treated merely as an auxiliary diagnostic. Instead, it is actively used to influence downstream graph learning. This is important because the graph stage should not trust every baseline prediction equally. Some zone-level predictions are likely to be stable and informative, while others may be noisy due to sparse history, volatile local demand, or disagreement with neighbouring zones [5], [16]. The confidence mechanism is designed to distinguish between these cases.

The confidence assigned to each zone at each rolling step is built from three sources:

1. a prior score reflecting how much historical data the zone has,
2. a stability score derived from Monte Carlo Dropout variance,
3. and a neighbourhood consistency score that compares the prediction to graph neighbours.

These components are then fused into a single confidence value that is used as a node-level weight in the graph stage [58], [59].

3.5.2 Monte Carlo Dropout for Predictive Uncertainty

Monte Carlo Dropout is used to estimate predictive uncertainty in the temporal stage. During inference, dropout remains active, and the same input is passed through the network multiple times. Let

$$\hat{y}^{(1)}, \hat{y}^{(2)}, \ldots, \hat{y}^{(M)}$$

denote the scaled predictions from M stochastic forward passes. The predictive mean is

$$\bar{y} = \frac{1}{M} \sum \_m = 1^M \hat{y}^{(m)},$$

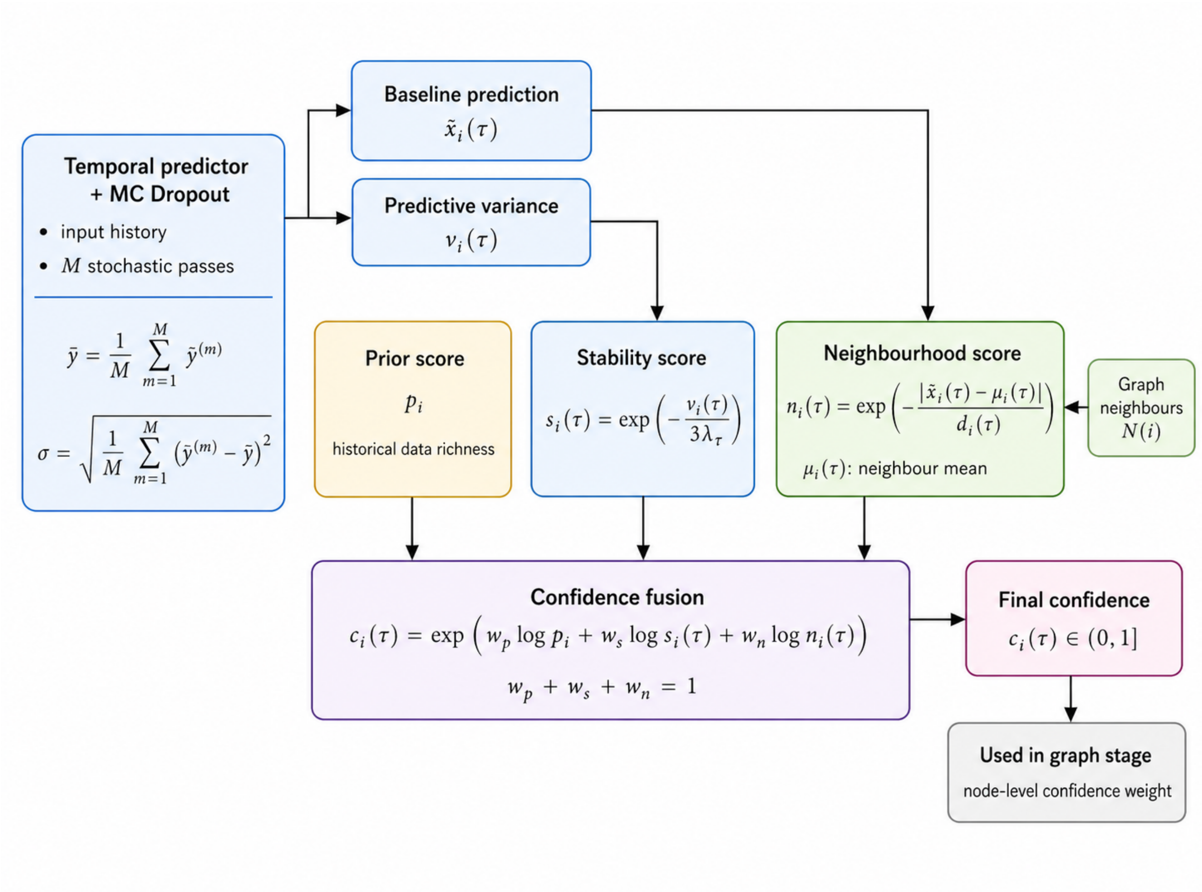

Figure 3.3: Confidence construction pipeline. Predictive uncertainty from Monte Carlo Dropout, historical prior information, and neighbourhood consistency are fused into a single node-level confidence score for weighted graph refinement.

and the predictive standard deviation is

$$\sigma = \sqrt{\frac{1}{M}\sum \_m = 1^M(\hat{y}^{(m)} - \bar{y})^2}.$$

The mean is inverse-transformed to obtain the final point prediction in the original scale. The standard deviation is also mapped back to the original scale using the scaler's data range. This produces a quantity with a direct interpretation in terms of demand-count variability.

Monte Carlo Dropout is attractive here because it does not require a separate probabilistic network or an ensemble of models. It reuses the same architecture and keeps uncertainty estimation computationally feasible even when many zone-specific models must be maintained [18].

3.5.3 Branch-Embedding Variability

In addition to predictive variance, the framework can inspect the variability of branch embeddings under Monte Carlo sampling. Let

$$\mathbf{e}^{1h,(m)}, \mathbf{e}^{1d,(m)}, \mathbf{e}^{1w,(m)}, \mathbf{e}^{1m,(m)}$$

be the branch embeddings from the m-th stochastic forward pass. Their variance across samples reflects how stable each temporal branch representation is under dropout-induced perturbation.

Although the current downstream confidence mechanism uses overall predictive variance rather than a full branch-specific weighting rule, this embedding variability is still methodologically meaningful. It confirms that the uncertainty process is sensitive not only to the final scalar prediction but also to internal representational instability [18].

3.5.4 Historical Prior Score

The first component of the confidence mechanism is a prior score that measures how much historical evidence is available for each zone. Zones with richer history are generally easier to model than zones with chronically sparse counts [5]. Let n_i denote the total number of records or effective count volume associated with zone z_i over the available historical data. The prior score is constructed by applying a log transform and min–max normalisation:

$$p\_i = \frac{\log(1 + n\_i) - \min\_j \log(1 + n\_j)}{\max\_j \log(1 + n\_j) - \min\_j \log(1 + n\_j)}.$$

This score is then rescaled away from exact zero, so that every zone retains at least a minimal confidence level.

The prior score does not depend on the current rolling step. It is a structural quantity reflecting the overall historical sufficiency of a zone. Methodologically, it acts as a regularising anchor: even if a current prediction appears stable under Monte Carlo sampling, a chronically sparse zone should still not be treated with the same confidence as a zone supported by much richer history.

3.5.5 Stability Score from Predictive Variance

The second confidence component is a stability score derived from the predictive variance at the current rolling step. Let $v\_i(\tau)$ denote the Monte Carlo variance for zone z_i at time τ. Higher variance indicates lower stability. The framework maps this to a score in $(0, 1]$ using an exponential decay:

$$s\_i(\tau) = \exp\Big(- \frac{v\_i(\tau)}{3\lambda\_\tau} \Big),$$

where λ_τ is a scale factor derived from the median variance across zones at that step. The resulting value is clipped to a small positive lower bound and 1.0 as an upper bound.

This design has two advantages. First, it produces a relative, step-aware notion of stability: what counts as "high variance" is interpreted in relation to the current global variance scale. Second, it avoids an overly harsh penalty for moderate uncertainty while still strongly down-weighting extremely unstable predictions. This is appropriate for urban demand forecasting, where some uncertainty is inevitable and should not automatically invalidate a prediction [16], [18].

3.5.6 Neighbourhood Consistency Score

The third component measures whether the prediction of a zone is consistent with the predictions of its graph neighbours. Let $\mathcal{N}(i)$ denote the set of neighbouring zones of z_i in the OD graph. For zone i at time τ, let $\tilde{x}\_i(\tau)$ be the baseline prediction and let

$$\mu\_i(\tau) = \frac{1}{|\mathcal{N}(i)|} \sum\_j \in \mathcal{N}(i) \tilde{x}\_j(\tau)$$

be the mean prediction of its neighbours. A dispersion-adjusted scale is computed from the neighbour predictions and the global prediction magnitudes. The neighbourhood consistency score is then defined through another exponential mapping:

$$n\_i(\tau) = \exp\Big(- \frac{|\tilde{x}\_i(\tau) - \mu\_i(\tau)|}{d\_i(\tau)} \Big),$$

where $d\_i(\tau)$ is a denominator incorporating neighbour standard deviation, local scale, and a global scale term.

This quantity is high when the zone prediction agrees with its graph neighbours and lower when it departs strongly from the local neighbourhood pattern. It should not be interpreted as a strict smoothness requirement, because legitimate local spikes can occur. Instead, it acts as a probabilistic reliability indicator: an extreme deviation from neighbouring forecasts is more likely to be suspicious when the local neighbourhood itself is comparatively coherent [13], [58].

3.5.7 Confidence Fusion

The three confidence components are fused multiplicatively in log space:

$$c\_i(\tau) = \exp\Big(w\_p \log p\_i + w\_s \log s\_i(\tau) + w\_n \log n\_i(\tau)\Big),$$

where w_p, w_s, and w_n are non-negative weights summing to one. In the current framework, the stability component is given slightly more emphasis than the other two.

This form of fusion corresponds to a weighted geometric mean rather than an arithmetic mean. Methodologically, that is a sensible choice. If any component is very low, the final confidence should also fall noticeably, because a zone that is historically sparse, highly unstable, or inconsistent with its neighbours should not remain highly trusted simply because it scores well on one other dimension. The geometric fusion therefore enforces a stricter notion of reliability than a simple additive average.

3.5.8 Interpretation of Confidence

The resulting confidence value

$$c\_i(\tau) \in (0, 1]$$

has a specific operational interpretation in the methodology. It is not a calibrated probability that the prediction is correct. Rather, it is a relative reliability weight indicating how strongly the graph stage should trust the temporal baseline and the associated residual supervision for that node. A high-confidence node provides a more dependable anchor for graph refinement. A low-confidence node remains in the graph, but its influence on the graph learning objective is reduced [58], [59].

The confidence mechanism does not suppress uncertain zones entirely, because uncertain regions may still benefit substantially from spatial correction. Instead, it modulates their influence so that the graph model is shaped more strongly by nodes whose baseline predictions appear trustworthy.

3.6 Stage Two: Graph-Based Residual Refinement

3.6.1 Motivation for Spatial Refinement

The temporal stage models each zone independently. This is valuable for local specialisation, but it leaves an important weakness unresolved: independent temporal models do not explicitly enforce spatial coherence. In a mobility system, zones interact through travel flows, commuting patterns, and network effects. If one zone experiences a surge while its strongly connected neighbours do not, the discrepancy may indicate either a true local anomaly or an error in the temporal estimate [13], [22]. The graph stage is introduced precisely to handle this kind of structured interaction.

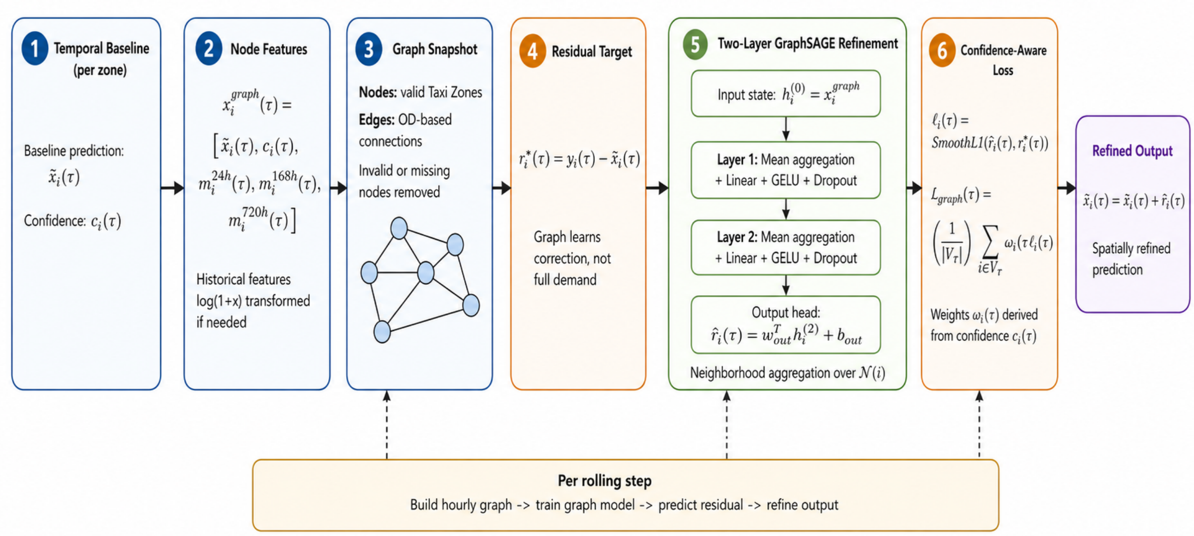

Figure 3.4: Graph-based residual refinement at each rolling step. The hourly taxi-zone graph is built from valid nodes, baseline predictions, confidence scores, and historical features, and a two-layer GraphSAGE model learns residual corrections that are added to the temporal baseline to obtain the final refined forecast. Confidence is injected through node-weighted loss rather than direct edge-weighted message passing.

Rather than building a graph model that predicts demand directly from node features alone, the methodology adopts a residual refinement strategy. The baseline temporal predictions are treated as strong priors, and the graph network learns how much these priors should be adjusted in light of spatial context. This makes the graph stage a corrective mechanism rather than a replacement predictor [15].

3.6.2 Node Features

At each rolling step, the graph is instantiated with one node per valid Taxi Zone. The node feature vector for zone z_i at time τ is

$$\mathbf{x}^{\text{graph}}\_i(\tau) = [\tilde{x}\_i(\tau), c\_i(\tau), m\_i^{24h}(\tau), m\_i^{168h}(\tau), m\_i^{720h}(\tau)].$$

If some auxiliary historical features are unavailable, they are replaced safely and transformed using $\log(1 + x)$ before concatenation.

The first element of the node feature vector is the baseline temporal prediction, which also acts as the prior that will later receive residual correction. The second element is the confidence score. The remaining elements summarise historical demand over different temporal ranges. Together, these inputs give the graph network access to the current forecast, the reliability of that forecast, and coarse contextual information about the historical level of the zone [32], [43].

3.6.3 Residual Supervision

The supervised target for the graph model is not the raw demand count. Instead, it is the residual

$$r\_i^*(\tau) = y\_i(\tau) - \tilde{x}\_i(\tau).$$

If the graph model predicts a residual $\hat{r}\_i(\tau)$, then the final refined prediction is

$$\hat{x}\_i(\tau) = \tilde{x}\_i(\tau) + \hat{r}\_i(\tau).$$

The residual strategy stabilises graph learning because the graph model only needs to explain what the temporal stage failed to capture. It also preserves interpretability: a positive residual means underprediction and a negative residual means overprediction

[15].

3.6.4 GraphSAGE Architecture

The graph refinement network is a two-layer GraphSAGE model with mean aggregation. Let $\mathbf{h}\_i^{(0)} = \mathbf{x}^{\text{graph}}\_i$ be the initial node feature vector. At each GraphSAGE layer, the node update takes the general form

$$\mathbf{h}\_i^{(\ell+1)} = \sigma\Big(\mathbf{W}^{(\ell)}\big[\mathbf{h}\_i^{(\ell)}\,;\,\mathrm{AGG}\{\mathbf{h}\_j^{(\ell)} : j \in \mathcal{N}(i)\}\big]\Big),$$

where AGG is mean aggregation, $\mathbf{W}^{(\ell)}$ is a learnable transformation, and σ is a nonlinear activation. In the implemented design, the activations are followed by dropout and the hidden nonlinearity is GELU.

The first GraphSAGE layer maps the input features into a hidden representation. The second layer further propagates and mixes information across the graph. Finally, a linear output layer projects the hidden state to a scalar residual:

$$\hat{r}\_i(\tau) = \mathbf{w}\_\text{out}^\top \mathbf{h}\_i^{(2)} + b\_\text{out}.$$

The use of mean aggregation is consistent with the overall design goal of smoothing and consistency-aware correction. Because the graph stage is not expected to discover highly complex spatial semantics from scratch, a relatively simple aggregation rule is appropriate and less likely to overfit than more elaborate attention-based alternatives in a limited-data setting [13], [19].

3.6.5 Confidence-Aware Loss Weighting

The graph model is trained using a per-node Smooth L_1 loss on the residual target:

$$\ell\_i(\tau) = \mathrm{SmoothL1}\Big(\hat{r}\_i(\tau), r\_i^*(\tau)\Big).$$

To incorporate reliability information, the loss is weighted by the node confidence:

$$\mathcal{L}\_\text{graph}(\tau) = \frac{1}{|\mathcal{V}\_\tau|} \sum\_i \in \mathcal{V}\_\tau \omega\_i(\tau)\, \ell\_i(\tau),$$

where the weights $\omega\_i(\tau)$ are derived from $c\_i(\tau)$ and normalised so that the mean weight remains close to one.

This weighting strategy is a central methodological innovation of the framework. It allows the graph stage to learn more strongly from nodes whose baseline predictions appear reliable and to down-weight the contribution of unstable or poorly supported nodes. Importantly, this does not mean that low-confidence nodes are ignored. They still receive refined predictions through message passing, but their residual targets influence optimisation less strongly [58], [59].

3.6.6 Why Node-Weighted Rather Than Edge-Weighted?

Although the OD matrix contains edge magnitudes and confidence can also be used to define pairwise quantities, the current graph stage injects reliability mainly through node-level loss weighting. This means that confidence affects optimisation but not the GraphSAGE message-passing operator directly. The edge structure still determines neighbourhood aggregation, but the messages themselves are not explicitly multiplied by OD flow magnitude or confidence-derived edge weights inside the standard `SAGEConv` layers.

This design reflects a methodological trade-off. Node-weighted learning is simpler, easier to implement robustly, and still captures a large share of the intended reliability logic. Explicit edge-weighted message passing would be a natural extension, but it would require modifying the graph operator or replacing it with a layer that directly supports weighted propagation. In the present framework, the more conservative choice is to keep message passing standard and inject reliability into the supervision instead [50], [63].

3.6.7 Graph Training per Rolling Step

At each rolling step, the graph stage is trained using the current hourly graph snapshot containing all valid zones and their baseline predictions. The model is optimised for a fixed number of epochs with Adam and a cosine annealing learning-rate schedule. After training, the residual predictions are produced and added to the baseline forecasts to obtain refined outputs.

Methodologically, this produces a clear, contained graph refinement process: each rolling hour yields one graph, one residual learning problem, and one set of refined predictions.

The advantage is that the graph stage is tightly aligned with the current forecasting state. The limitation is that training and evaluation occur on the same hourly batch, which may overstate generalisation [54], [55]. This issue is discussed explicitly later because it affects how strongly one should interpret graph-stage performance gains.

3.6.8 Handling Missing and Invalid Nodes

Only nodes with valid baseline predictions and valid labels are used in graph training. If a zone is missing a temporal prediction or lacks a valid target value, it is excluded from the graph snapshot for that step. The adjacency structure is then reindexed to match the filtered node set. This avoids contaminating the graph stage with invalid entries and prevents propagation of undefined values.

Such filtering is methodologically justified because the graph stage is a refinement model, not a data imputation mechanism. Its purpose is to improve valid baseline predictions through spatial consistency, not to invent supervision where none exists. Restricting the graph to valid nodes therefore keeps the learning problem well-posed.

3.7 Rolling Forecasting Protocol and Model Management

3.7.1 Rolling Workflow

The entire framework is executed under a rolling protocol. Let the first target time be τ_1. For each rolling step $k = 1, 2, \ldots, K$, the target hour is

$$\tau\_k = \tau\_1 + (k - 1)\Delta,$$

where $\Delta = 1$ hour. At each step, the system performs the following operations:

1. determine the valid historical context for each zone,
2. train or load the temporal model as required,
3. generate Monte Carlo baseline predictions and uncertainty estimates,
4. compute prior, stability, and neighbourhood confidence components,

5. assemble the graph snapshot with node features and residual targets,
6. train the graph refinement model and produce refined predictions,
7. compute and save per-hour and cumulative metrics.

This repeated procedure is intended to approximate operational forecasting, in which predictions are updated continuously as time progresses. It also reveals whether the forecasting quality is stable, improving, or deteriorating over the rolling horizon [54], [55].

3.7.2 Persistent Checkpoint Reuse

In the default operating mode, the temporal stage uses persistent checkpoint reuse. Once a zone model has been trained on the history available up to a given context end, it is saved and reused in later rolling steps. If no checkpoint exists, the model is trained once. If a checkpoint exists, the system loads it and uses it for forecasting without retraining the model from scratch.

Persistent reuse has clear practical benefits. It reduces computation, allows rapid repeated forecasting, and mimics the behaviour of many deployed systems in which models are updated less frequently than predictions are requested. Methodologically, this persistent setup provides a realistic and efficient baseline for rolling forecasting [54], [66].

However, persistent reuse also introduces a risk. If demand patterns drift over time, a fixed checkpoint may become progressively less well matched to the current data. This motivates the optional incremental extension [66].

3.7.3 Incremental Update Extension

Beyond the persistent baseline, the framework also supports an incremental training variant. In this mode, when new rolling windows become available, the previously trained zone model can be updated through lightweight additional optimisation rather than being reused unchanged or retrained from scratch. The aim is to preserve most of the learned structure while allowing the model to absorb newly observed data.

Methodologically, this incremental mode occupies an intermediate position between full retraining and static checkpoint reuse. Full retraining is the most adaptive but computationally costly. Pure persistence is computationally cheap but may become stale. Incremental updating attempts to combine the strengths of both: the model retains past knowledge but can respond to drift through targeted adaptation [66].

It is important to state the status of this feature accurately. The incremental mechanism is available in the codebase, but it is not the default baseline in the current rolling driver configuration. Therefore, when this thesis discusses the main rolling results, persistent reuse remains the default interpretation unless otherwise stated. The incremental variant should be treated as an implemented extension that strengthens the realism and extensibility of the framework.

3.7.4 Why Rolling Evaluation Matters

Rolling evaluation is not merely an implementation preference. It is a methodological stance about how forecasting systems should be assessed. In static train/test settings, the system is typically trained once and evaluated on a fixed hold-out set. That can be useful for benchmarking, but it suppresses several issues that matter in practice: how performance changes over time, whether uncertainty grows under drift, whether one-step models remain stable across many consecutive operating hours, and what the repeated computational burden of forecasting looks like [54], [55].

By contrast, the rolling protocol used here repeatedly reconstructs the forecasting state as the target time moves forward. This makes the evaluation more realistic and reveals whether graph refinement, uncertainty, and confidence weighting remain useful under changing conditions [54].

3.7.5 Temporal Integrity Under Rolling Forecasting

A forecasting methodology must ensure that only historically available information is used at each step. The current design enforces this in several ways. The temporal input windows end at the hour immediately before the target. The scaler is fitted using observations strictly earlier than the target hour. Historical summary features for graph refinement are computed from windows preceding the target. As a result, the temporal baseline stage is close to a valid leakage-free rolling forecasting procedure [54], [55].

This does not mean that every component of the system is fully free from methodological concerns. As discussed earlier, the graph refinement stage still trains and evaluates on the same hourly batch. Thus, the temporal stage satisfies a stronger notion of temporal integrity than the graph stage currently does. Making that asymmetry explicit is important for honest interpretation of results.

3.8 Methodological Discussion

3.8.1 Why the Method Uses a Two-Stage Rather Than End-to-End Design

A natural question is why the framework is split into a temporal predictor and a graph refiner instead of using a fully end-to-end spatiotemporal graph neural network from the start. There are several methodological reasons. First, a two-stage design makes the role of each component clearer. The temporal stage is responsible for local next-hour forecasting based on historical demand, while the graph stage is responsible for spatial correction. Second, this separation allows the temporal and graph models to be analysed independently. For example, one can compare the temporal baseline to the graph-refined output directly. Third, the two-stage design simplifies engineering and checkpoint management, especially when the temporal stage is maintained per zone.

There is also a conceptual advantage. Many urban forecasting problems contain both strong local regularity and strong spatial coupling, but not always in equal measure. A unified end-to-end model may entangle these two sources of information in ways that are difficult to interpret. The present method deliberately assigns local regularity to the temporal model and structured cross-zone adjustment to the graph model. This division is simple, interpretable, and consistent with the deployment setting [14], [15].

3.8.2 Current Limitations

The present methodology is strong in several respects, but it also contains deliberate limitations that should be acknowledged clearly.

First, the graph refinement stage currently trains and evaluates on the same hourly node set. This makes the graph correction locally aligned with the current forecasting state, but it weakens claims about cross-temporal generalisation. A stricter protocol would train graph refinement on earlier snapshots and evaluate it on later snapshots [54], [55].

Second, the OD flow matrix is converted mainly into binary connectivity for message passing. The original edge magnitude information is not fully exploited in the current GraphSAGE operator. As a result, the graph topology captures the existence of mobility coupling, but not its precise strength during message aggregation [13], [50].

Third, the temporal stage is per zone, which improves local specialisation but sacrifices direct parameter sharing across zones. In data-poor zones, a shared or hybrid temporal representation may eventually prove beneficial [5], [43].

Fourth, while the incremental update mechanism is implemented, persistent reuse remains the default baseline. Thus, the current main rolling configuration should still be understood primarily as a persistent checkpoint system with optional extension to incremental adaptation.

Fifth, the historical summary features used in the graph stage are deliberately simple. More expressive exogenous variables such as weather, holidays, events, road disruptions, or land-use descriptors are not yet included. Their absence makes the current method cleaner and easier to interpret, but also leaves performance on the table [32], [56].

3.8.3 Summary

In summary, the proposed methodology constructs a rolling two-stage forecasting system for hourly Taxi Zone demand prediction. The first stage is a per-zone multi-scale temporal predictor that combines LSTM, Transformer, and GRU components across four temporal horizons. The second stage is a GraphSAGE residual refinement model operating on an OD flow graph. Between them lies a confidence mechanism that fuses historical sufficiency, predictive stability, and neighbourhood agreement into a node-level reliability weight. The temporal stage prioritises local modelling fidelity, the graph stage introduces spatial consistency, and the rolling protocol provides deployment-oriented evaluation.

The resulting framework is methodologically coherent because each component serves a distinct purpose and because the information flow between components is explicit. Multi-scale modelling addresses temporal heterogeneity. Monte Carlo Dropout quantifies uncertainty. Confidence weighting links uncertainty to downstream optimisation. Graph refinement corrects independent zone-level forecasts using relational structure. Rolling execution tests whether all of these choices remain useful as time advances.

Although the current design still leaves room for stronger graph validation, explicit edge weighting, and more systematic online adaptation, it already provides a solid and research-oriented methodological foundation. It is not merely a collection of implementation choices. Rather, it is a structured response to the combined challenges of temporal complexity, spatial dependence, uncertainty, and operational realism in urban taxi demand forecasting.

Chapter 4

Implementation

4.1 Overview

This chapter describes the concrete implementation of the proposed two-stage, confidence-weighted forecasting framework. Whereas the previous chapter introduced the methodology at an algorithmic level, this chapter explains how the system is realised in the codebase, how the major modules interact, and how data moves from raw taxi records to temporal predictions, graph-refined outputs, and final evaluation files. The implementation is organised around three principal Python modules. The first is `demo_rolling_GNN.py`, which acts as the top-level rolling driver. It loads data, prepares lookup tables and adjacency structures, controls the rolling loop, computes confidence scores, calls the temporal model manager, invokes the graph refinement stage, and writes the main output files. The second is `persistent_multiscale_confi.py`, which implements the per-zone multi-scale temporal model and its checkpoint manager. The third is `gnn_model.py`, which implements the GraphSAGE residual refinement model and the graph-stage training pipeline. This division of responsibilities is visible in the driver script, where the graph pipeline is imported from `gnn_model.py`, while the default temporal manager is imported from `persistent_multiscale_confi.py`; the incremental manager is present as an alternative import but is not enabled by default in the current script.

The implementation follows a deliberately modular structure. The temporal stage is isolated inside a model-manager class so that training, checkpoint storage, loading, scaling, inference, and uncertainty estimation are all handled through a consistent interface. The

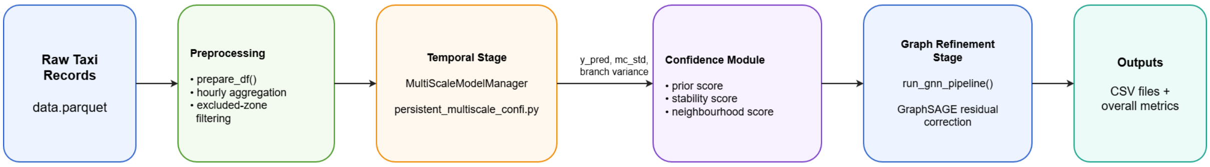

Figure 4.1: Graph-based residual refinement at each rolling step. The hourly taxi-zone graph is built from valid nodes, baseline predictions, confidence scores, and historical features, and a two-layer GraphSAGE model learns residual corrections that are added to the temporal baseline to obtain the final refined forecast. Confidence is injected through node-weighted loss rather than direct edge-weighted message passing.

graph stage is separated into its own pipeline so that it receives a clean table of predictions, labels, historical features, and confidence values from the rolling driver. The driver script then coordinates the two stages at each target hour. This design makes the system easier to debug and to extend: the temporal model can be modified without rewriting the graph refinement function, and the graph refinement logic can be changed without altering the data loading and rolling orchestration code.

A central implementation decision is that the rolling pipeline is treated as the main execution unit. The system is not written as a single train-then-test script. Instead, the driver iterates over target timestamps, generates per-zone predictions, evaluates the temporal baseline, constructs the graph input for that hour, performs graph refinement, and stores both detailed and aggregated metrics. The rolling configuration is controlled by explicit constants in the driver script, including paths for the data, lookup table, edge-weight matrix, checkpoint directory, starting target time, number of rolling steps, excluded zones, whether to retrain each hour, and the number of Monte Carlo Dropout samples.

4.2 Software Structure and Module Responsibilities

4.2.1 Top-Level Rolling Driver

The main entry point of the implementation is `demo_rolling_GNN.py`. This file is responsible for system-level orchestration rather than model definition. Its responsibilities can be divided into five groups: data preparation, graph support construction, confidence computation, rolling prediction, and output generation.

The first group concerns data preparation. The function `prepare_df()` loads `data.parquet`,

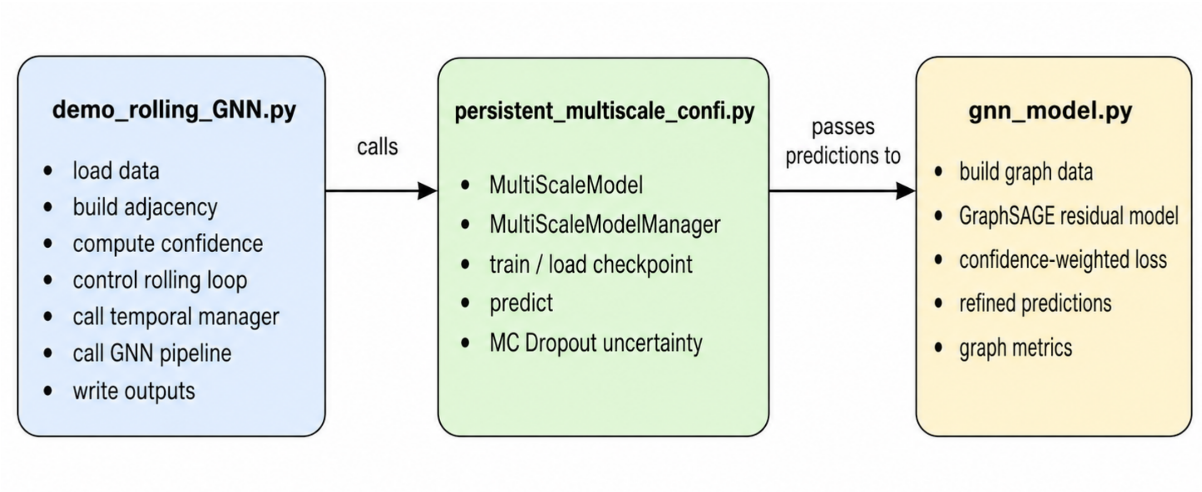

Figure 4.2: Overall workflow of the rolling GNN prediction pipeline, showing data preparation, multiscale temporal prediction, and GraphSAGE-based refinement with confidence weighting.

keeps the `pickup_datetime` and `PULocationID` columns, converts timestamps into pandas datetime format, floors them to hourly resolution, filters out excluded zones, and prints basic timestamp diagnostics. In addition, `_build_zone_hourly_counts()` constructs a grouped count series indexed by `PULocationID` and hour, which is later used for historical mean features. This implementation choice ensures that the downstream pipeline receives a consistent hourly representation of the raw trip data.

The second group concerns lookup and adjacency construction. The function `_load_zone_lookup()` reads the taxi-zone lookup table and removes duplicate `LocationID` entries. The function `_build_zone_adjacency()` reads the OD flow matrix, normalises possible byte-order marks in row and column labels, maps zone names to `LocationID` values, and creates an adjacency list by retaining neighbours whose flow weight is greater than zero. This adjacency list is used mainly for the neighbourhood consistency component of the confidence score.

The third group concerns confidence computation. The rolling driver computes three reliability components: a prior score based on historical sample richness, a stability score based on Monte Carlo predictive variance, and a neighbourhood consistency score based on agreement with adjacent zones. The prior score is computed from log-transformed zone counts and then rescaled to avoid exact zero confidence. The stability score is computed from variance values in the current rolling step using an exponential decay controlled by the median variance scale. The neighbourhood score compares each prediction with available predictions from graph neighbours and maps the difference into a clipped confidence interval. These components are combined in log space with weights for prior, stability, and neighbourhood terms, and the resulting score is assigned back to the per-hour prediction dataframe.

The fourth group concerns rolling execution. The function `run_rolling_with_gnn()` is the central loop. It iterates over `ROLLING_STEPS`, constructs the target timestamp for each step, optionally creates a separate checkpoint directory if hourly retraining is enabled, and otherwise reuses the shared temporal manager. For every zone, it calls the temporal manager to train or load the model if needed, obtains the point prediction and uncertainty estimate, records the true value for the current target hour, and computes historical mean features. After the zone loop finishes, it evaluates the baseline temporal predictions, prepares the GNN input table by renaming `y_pred` to `Prediction` and `y_true` to `True Value`, and calls `run_gnn_pipeline()` with the confidence dictionary and historical feature columns.

The fifth group concerns output generation. The rolling driver stores the per-zone temporal predictions in `predictions_rolling_mc.csv`, the temporal baseline hourly metrics in `hourly_metrics_gru.csv`, the graph-refined predictions in `gnn_refined_predictions.csv`, and the graph-stage hourly metrics in `hourly_metrics_gnn.csv`. It also computes overall MAE and RMSE for the temporal baseline and graph-refined outputs and writes them into `overall_metrics_gnn.txt`. The `main()` function ties these components together by cleaning old checkpoint directories, selecting CUDA or CPU, preparing the dataframe, computing prior scores, loading the lookup table, building adjacency, constructing hourly count series, creating the temporal manager, and launching the rolling loop.

4.2.2 Temporal Model and Manager Module

The temporal modelling code is implemented in `persistent_multiscale_confi.py`. The file header explicitly identifies the module as a confidence-weighted multi-scale trainer for full runs only, applying MC Dropout confidence logic but without hourly incremental fine-tuning in the default persistent version. This distinction is important because the project also has an incremental variant, but the default driver currently imports the persistent manager.

The main model class is MultiScaleModel. It is a four-branch temporal architecture with LSTM encoders for daily and weekly contexts, a Transformer encoder for the monthly context, dropout layers for Monte Carlo sampling, a fusion layer, a GRU backbone, and a final linear output layer. The branch-level components are initialised directly inside the constructor. The model encodes the daily, weekly, and monthly branches through _encode_temporal_branches(), where the daily and weekly sequences are passed through LSTM layers, while the monthly sequence is projected and processed through a Transformer. The fused trend representation is concatenated with the short-term one-hour input to build the GRU input, and the final forward pass returns both the scalar prediction and the internal branch embeddings.

The same class also provides two MC Dropout-related interfaces. The function mc_branch_embeddings(runs repeated stochastic forward passes and collects branch embeddings under active dropout. The function mc_predict() returns the mean and standard deviation of repeated stochastic predictions in scaled space. These methods allow the implementation to use the same model both as a point predictor and as an uncertainty estimator.

The manager class, MultiScaleModelManager, wraps the model architecture with persistence, data preparation, scaling, training, inference, and orchestration logic. Its configuration is represented by the ManagerConfig dataclass, which specifies the hidden size, full-training epochs, patience, learning rate, sequence length, forecast length, dropout probability, auxiliary coefficient, and Monte Carlo sample counts. The manager constructor creates the checkpoint directory and stores the duration mapping for the four input branches: 1 hour, 24 hours, 168 hours, and 720 hours.

Persistence is implemented through separate paths for the model weights, scaler, and metadata. The manager checks whether both model and scaler files exist before deciding whether a checkpoint is available. It saves model parameters with torch.save, serialises the scaler with pickle, and stores the latest training context end in a JSON metadata file. This separation makes the checkpoint explicit and recoverable: the learned network state, scaling transformation, and training horizon are all stored independently.

4.2.3 Graph Refinement Module

The graph refinement stage is implemented in `gnn_model.py`. The file imports `Data`, `SAGEConv`, and `dense_to_spar-se` from PyTorch Geometric, indicating that the graph stage is built on sparse graph data structures rather than dense matrix operations. The core graph model is `MultiScaleGraphSAGE`, a two-layer GraphSAGE network with mean aggregation, GELU activations, dropout, and a final linear layer. Its forward pass reads node features and edge indices, retrieves `base_pred`, applies two GraphSAGE layers, predicts a residual, and returns both the residual and the refined prediction obtained by adding the residual to the base prediction.

The graph pipeline is implemented by `run_gnn_pipeline()`. It accepts either a dataframe of predictions or a path to a merged CSV file, validates that the input contains `PULocationID`, `Prediction`, and `True Value`, reads the OD edge-weight matrix, converts it into sparse graph connectivity, and constructs mappings between zone names and location identifiers. The pipeline then aligns prediction and label values to graph nodes, filters invalid nodes, remaps the edge index to match the filtered graph, constructs the residual target, concatenates node features, and stores the graph data inside a PyTorch Geometric `Data` object. Training uses Smooth L1 loss with confidence-based node weighting, Adam optimisation, and cosine annealing. The model finally outputs a dataframe containing `GRU_Pred`, `Refined_Pred`, `True_Value`, and `Confidence`, and computes MAE and MSE for both the baseline and refined predictions.

4.3 Data Loading and Preprocessing Implementation

The implementation begins with a compact but important preprocessing step. The raw taxi data are loaded from `data.parquet`. Only the pickup timestamp and pickup zone identifier are required at this stage. The timestamp is converted to pandas datetime format and then floored to the nearest hour, creating the `datetime` column used throughout the rest of the system. Excluded zones are filtered immediately so that invalid or unwanted regions do not appear in any later computation. This early filtering is important because the same zone set is used for temporal prediction, confidence scoring, graph refinement, and metric aggregation.

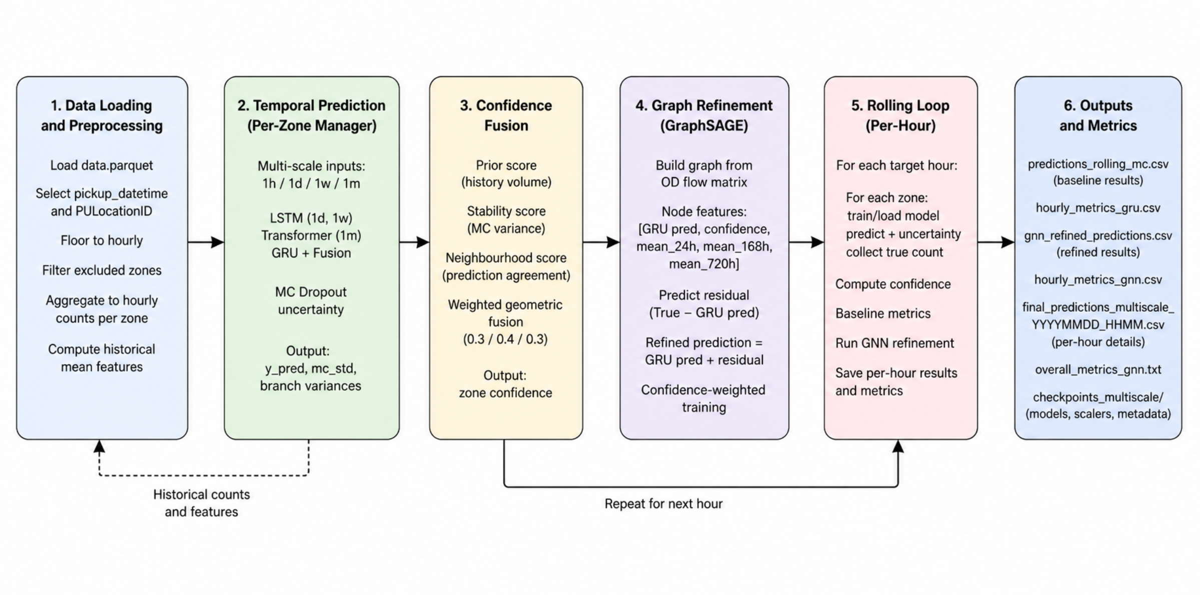

Figure 4.3: Overall implementation pipeline for rolling taxi-demand prediction, combining data preprocessing, multi-scale temporal forecasting, confidence fusion, GraphSAGE refinement, and metric output generation.

The function `get_true_counts()` extracts the ground-truth count for a target hour by filtering the dataframe to the target timestamp and grouping by pickup location. This produces a mapping from `PULocationID` to observed demand. The rolling loop uses this mapping to assign `y_true` for each zone. The implementation also builds `zone_hourly_counts`, a grouped pandas series over zone and hour, so that historical windows can be queried efficiently when constructing mean historical features.

Historical mean features are computed in `_compute_history_means()`. For each target hour and zone, the function creates three lookback windows corresponding to 24 hours, 168 hours, and 720 hours. The sum of observed hourly counts inside each window is divided by the window length, resulting in average demand over the short, weekly, and longer-term context. If a zone has no historical series, the feature dictionary defaults to zeros. These features are later passed into the graph pipeline and concatenated with the temporal prediction and confidence value.

The temporal manager also performs its own internal preprocessing at the per-zone level. The function `_prepare_zone_series()` constructs a dense hourly sequence for a single zone ending at the forecasting context end. It groups records by datetime, creates a full hourly date range, reindexes the grouped series to that range, fills missing hours with zero, and stores the result as floating-point passenger counts. This dense reconstruction is essential because a neural sequence model requires a regular input grid. Without explicitly inserting zero-demand hours, the model would not be able to distinguish inactivity from missing records.

Scaling is implemented in `_fit_scaler_hist()`. The scaler is fitted only on rows whose timestamps are earlier than the target hour, which helps preserve temporal integrity. The implementation also handles edge cases where the available history is empty, non-finite, or constant by fitting artificial two-point ranges. This avoids crashes in sparse zones and ensures that inverse scaling remains defined even when the historical range is degenerate.

Training windows are constructed in `_build_training_windows()`. The function uses the manager configuration to determine the sequence length and forecast horizon, scales the hourly counts, and then slides across the sequence to produce four input tensors: `1h`, `1d`, `1w`, and `1m`. The one-hour branch receives the most recent step, the daily branch receives the last 24 steps, the weekly branch receives the last 168 steps, and the monthly branch receives the full 720-step window. The target is the next-hour passenger count in scaled space. During inference, `_build_inference_window()` uses the same duration mapping to build one aligned input dictionary ending at `context_end`.

4.4 Temporal Model Implementation

The implemented temporal model combines recurrent and attention-based components in a single multi-scale architecture. The constructor defines LSTM encoders for the daily and weekly inputs, a linear projection followed by a Transformer for the monthly input, branch-level dropout layers, a linear feature-fusion layer, a GRU backbone, and a final fully connected output layer. This architecture directly implements the methodological choice that different time scales should be represented through specialised branches.

During the forward pass, the model first encodes the daily, weekly, and monthly inputs. The daily and weekly branches return the final hidden states of their LSTM encoders. The monthly branch is projected from a scalar sequence into the hidden dimension and then passed through a Transformer; the final time step of the Transformer output is used as the monthly embedding. These three embeddings are concatenated and compressed through `feature_fusion`. The fused trend representation is then repeated along the

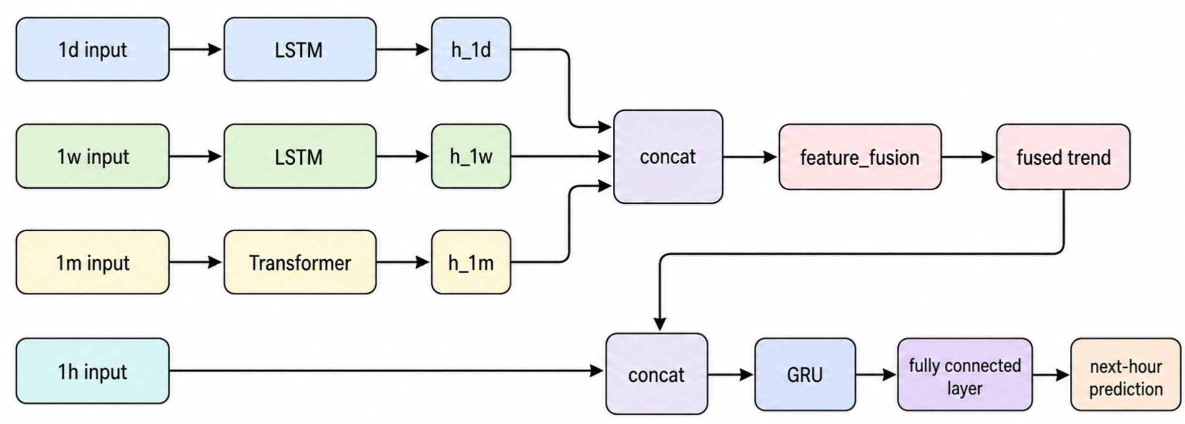

Figure 4.4: Architecture of the multi-scale temporal predictor, where daily and weekly inputs are encoded by LSTM branches, the monthly input is encoded by a Transformer branch, and the fused trend representation is combined with the short-term input before GRU-based next-hour prediction.

short sequence dimension and concatenated with the one-hour input to form the GRU input.

This implementation makes the GRU a final prediction pathway rather than the only temporal encoder. The longer-term dynamics are represented before the GRU stage, and the GRU receives both immediate demand and the fused trend. This separation makes the model relatively interpretable: daily, weekly, and monthly context are encoded explicitly, while the final output head converts the fused information into a scalar next-hour prediction.

Dropout is used in two ways. During normal training, it regularises the branch representations and the short-term hidden state. During uncertainty estimation, the model deliberately keeps dropout active through the `train()` mode inside the MC functions. In `mc_predict()`, repeated stochastic forward passes produce a tensor of predictions, from which the mean and standard deviation are computed. In `mc_branch_embeddings()`, the same repeated-sampling idea is applied to hidden embeddings rather than only final predictions.

Training is managed by `train_once()`. If a checkpoint already exists, the function exits early. Otherwise, it prepares the dense zone series, builds training windows, creates a `MultiScaleModel`, and trains it with Adam and mean squared error. The loop tracks the best loss and uses a patience counter for early stopping. After training, the model, scaler, and metadata are saved. The implementation therefore avoids repeated full training for zones that already have a checkpoint, which is central to the persistent rolling setup.

Inference is split into `predict()` and `predict_with_uncertainty()`. The plain prediction function loads the checkpoint, reconstructs the context end as one forecast horizon before the target date, prepares the dense series, fits the scaler on historical data before the target, builds the inference window, obtains the MC mean prediction, inverse-transforms it, and returns it as a float. The uncertainty function performs the same setup but additionally returns the original-scale predictive standard deviation and a dictionary of branch variances for the four temporal branches. This is the interface used by the rolling driver to populate `y_pred`, `mc_std`, and variance values for each zone.

The orchestration method `train_and_predict_if_needed()` combines persistence and forecasting. It computes the context end, trains the model if no checkpoint exists, loads metadata if a checkpoint does exist, and warns if the checkpoint is older than the current context while incremental updates are disabled. It then returns the prediction for the target date. This makes the default behaviour explicit: the current persistent version reuses existing checkpoints rather than automatically fine-tuning them.

4.5 Confidence Implementation

The confidence mechanism is implemented mainly in the rolling driver rather than inside the temporal manager or the graph module. This is appropriate because confidence depends on information from multiple sources: global historical counts from the full dataframe, MC variance values from temporal inference, and neighbourhood agreement from the graph adjacency list.

The prior score is computed once before rolling begins. It groups the entire prepared dataframe by `PULocationID`, applies a log transform to the counts, min–max normalises them, and rescales the output into a range bounded away from zero. This produces a

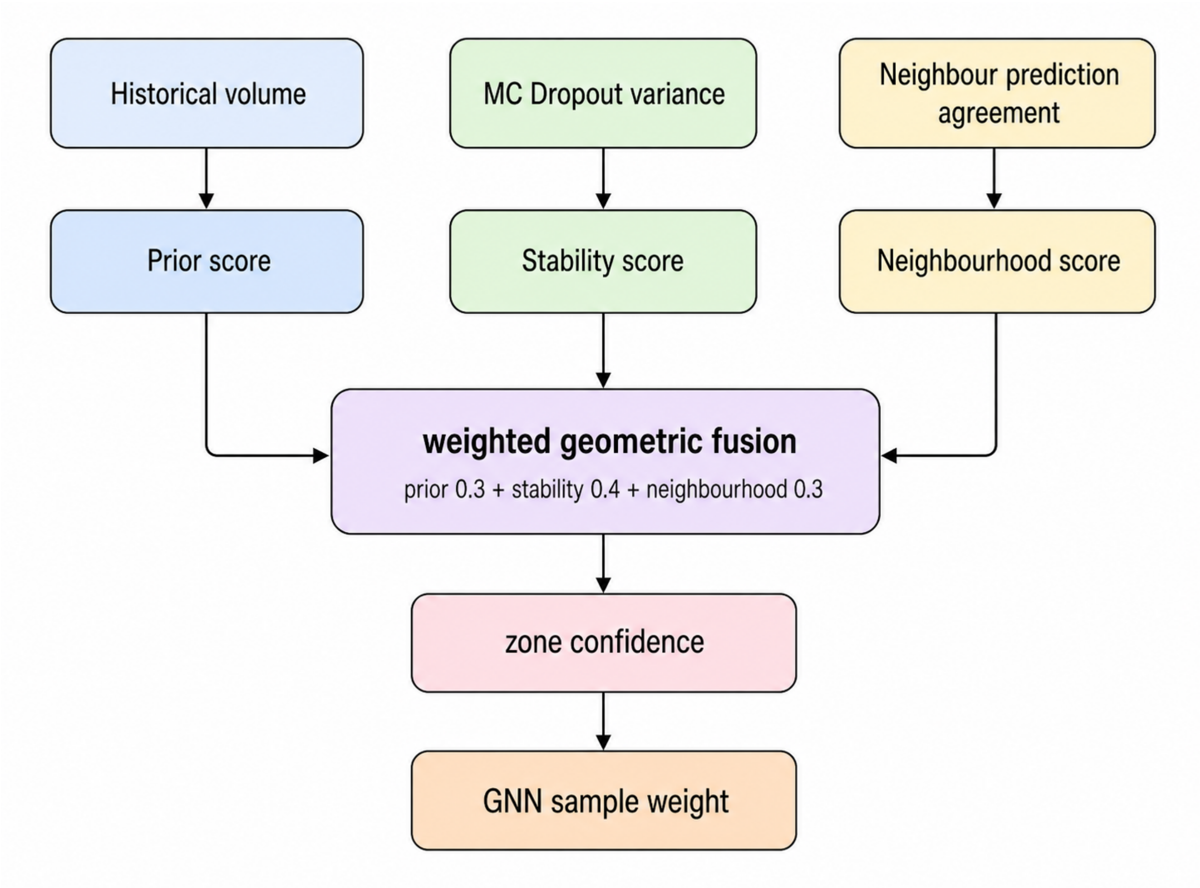

Figure 4.5: Confidence fusion mechanism combining historical prior, MC Dropout-based stability, and neighbourhood prediction agreement into a zone-level confidence score used as the GNN sample weight.

zone-level reliability prior: zones with richer historical observations receive stronger prior confidence.

The stability score is computed at every rolling step because it depends on the current MC variance. The implementation takes the variance column from the step dataframe, uses the median finite variance as a scale parameter, and applies an exponential decay to map variance into a confidence-like score. Predictions with large variance receive lower stability scores.

The neighbourhood score is also computed at every rolling step. The implementation first constructs a prediction map for finite predictions, then iterates over zones. For each zone, it retrieves graph neighbours, collects available neighbour predictions, computes their mean and standard deviation, and compares the zone prediction to the local neighbour mean. The resulting difference is converted into a clipped exponential score. If no neighbours have usable predictions, the function assigns a default score rather than failing.

The fusion function clips all three components into a safe interval and combines them in log space using the configured component weights. This corresponds to a weighted geometric mean. The final assignment function computes stability and neighbourhood scores, sets the weights to prior 0.3, stability 0.4, and neighbourhood 0.3, and maps each zone to its combined confidence. The output confidence dictionary is passed directly into `run_gnn_pipeline()` and also stored in the step dataframe.

4.6 Graph Refinement Implementation

The graph stage starts from a dataframe prepared by the rolling driver. The driver renames the temporal prediction column to `Prediction` and the observed label column to `True Value`, keeps `PULocationID` and the historical features, and passes this dataframe to `run_gnn_pipeline()`. The graph module then checks whether the prediction input is provided as a dataframe or CSV path and verifies that the required columns are present.

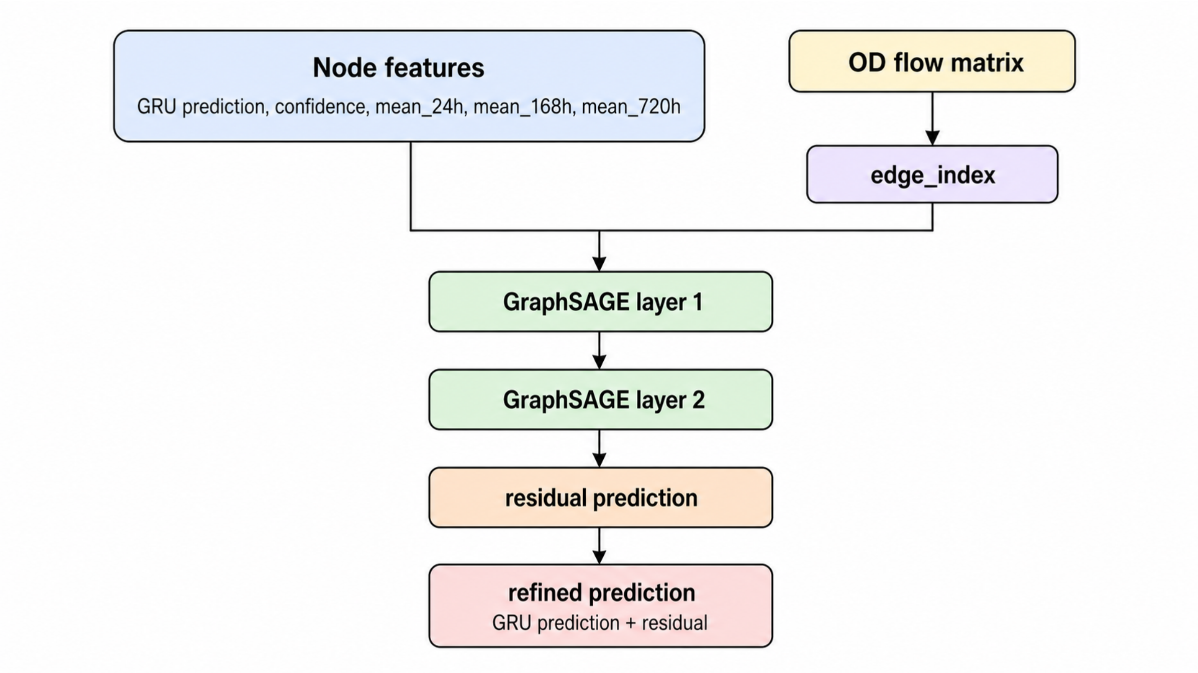

Figure 4.6: GraphSAGE residual refinement architecture, where temporal predictions, confidence scores, and historical mean features are used as node inputs, while the OD flow matrix defines the graph structure for learning residual corrections.

The OD flow matrix is read from `edge_weight_matrix_with_flow.csv`. It is converted into a tensor and then into sparse `edge_index` format with `dense_to_sparse`. The module also reads the taxi-zone lookup file, constructs mappings between zone names and location identifiers, and uses those mappings to align dataframe rows with graph node positions.

Node weights are built by `_build_node_weights()`. If a confidence dictionary is provided, the function maps each `PULocationID` confidence score to the corresponding zone index and clips it into the interval from zero to one. If no confidence dictionary is available, the function falls back to county-level volume weighting. In the main rolling configuration, confidence is provided, so the node weights correspond to confidence values derived from temporal uncertainty, historical richness, and neighbourhood consistency.

The graph pipeline creates arrays for node predictions, node labels, and historical features, initially filled with NaNs. It then iterates over the prediction dataframe, maps each `PULocationID` to its zone name and graph index, and fills the corresponding prediction, label, and historical feature positions. After this mapping, the pipeline filters to nodes where both prediction and label are finite. It also filters and remaps the edge index so that the graph contains only the valid nodes for the current hour.

The residual target is computed as `node_label - node_pred`. Node features are formed by concatenating the baseline prediction, node weight, and optional historical feature tensor. The resulting graph data object contains `x`, `edge_index`, `y`, `base_pred`, and `confidence`. If confidence is available, a confidence-derived `edge_weight` field is also constructed as the product of source and target node weights, although the current `SAGEConv` layers do not explicitly consume this field.

Training configuration is defined inside the graph pipeline. The graph model uses dropout 0.1, learning rate 0.01, hidden dimension 256, and 300 epochs. The optimiser is Adam, the scheduler is cosine annealing, and the loss function is Smooth L1 with no reduction so that per-node weighting can be applied. During training, the pipeline computes residual predictions and refined predictions, evaluates the per-node residual loss, chooses confidence as the sample weight when available, normalises the weights by their mean, and optimises the weighted average loss.

After training, the graph model is evaluated without gradient tracking. The output dataframe includes the location ID, zone name, GRU baseline prediction, graph-refined prediction, true value, and confidence. The graph module computes MAE and MSE for both the baseline and refined predictions, optionally plots diagnostic figures, writes the per-hour final prediction CSV, and returns both the dataframe and metric dictionary to the rolling driver.

4.7 Rolling Pipeline Implementation

The rolling pipeline is the practical centre of the implementation. The function `run_rolling_with_gnn(` receives the prepared dataframe, the temporal model manager, the device, prior scores, adjacency mapping, and hourly count series. It begins by sorting all available zones and initialising containers for baseline records, graph records, baseline metrics, and graph metrics.

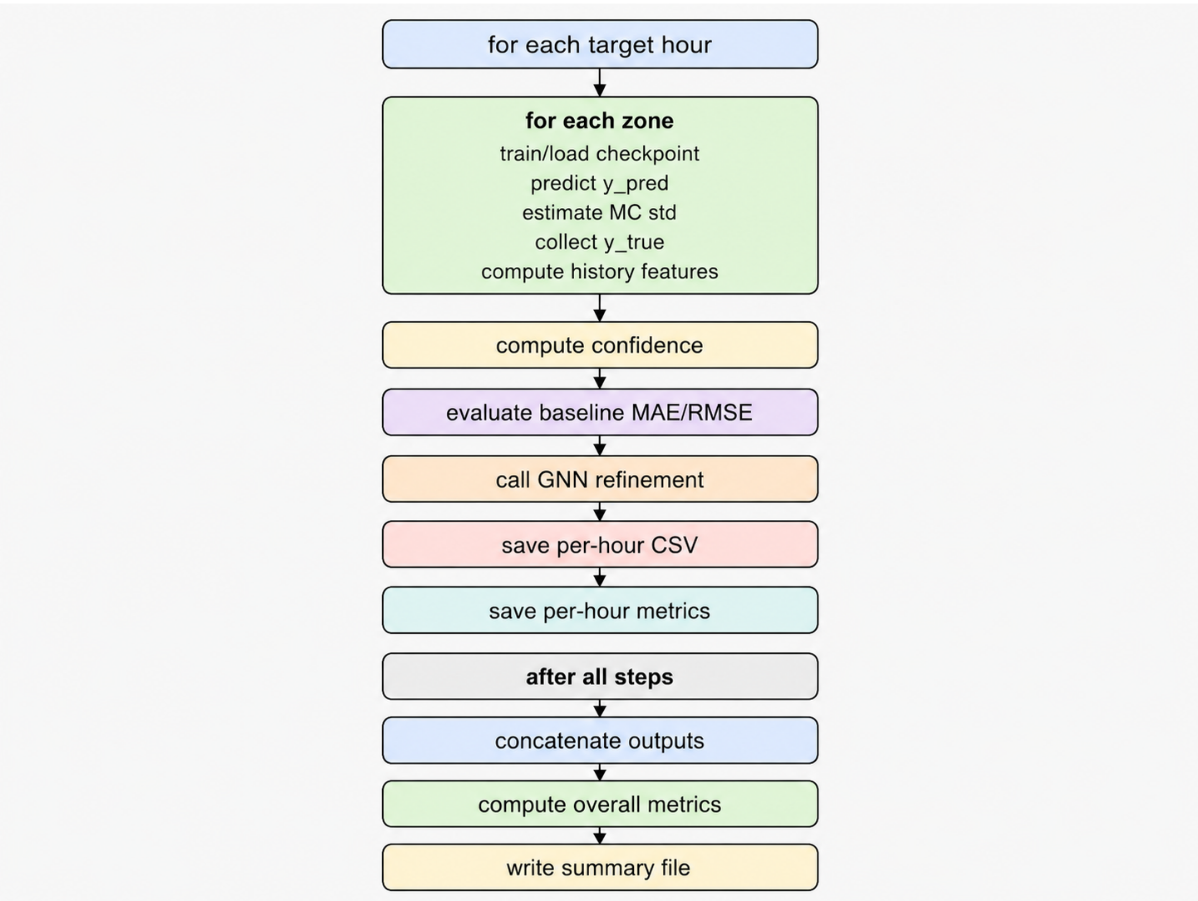

Figure 4.7: Rolling execution loop for hourly taxi-demand prediction, uncertainty estimation, confidence assignment, GraphSAGE refinement, and final metric aggregation.

For each rolling step, the target timestamp is computed by adding the step offset to `START_TARGET`. If `RETRAIN_EACH_HOUR` is true, a separate checkpoint directory is created for the current target hour, and a new manager is initialised. If it is false, the existing manager is reused. This branch allows the same codebase to support different experimental regimes: persistent reuse, hourly retraining, or alternative manager replacement such as the incremental version.

Inside the zone loop, the implementation first calls `train_and_predict_if_needed()` to ensure that a checkpoint exists and then calls `predict_with_uncertainty()` to obtain the temporal point estimate and standard deviation. The variance is computed as the square of the standard deviation. The observed count is obtained from the target-hour ground-truth dictionary, and the historical mean features are attached to the record. Errors are caught on a per-zone basis, and failed zones are recorded with NaNs and an error string. This makes the rolling loop robust to isolated zone failures rather than allowing one failed zone to terminate the whole run.

After all zones have been processed for the current target hour, the implementation converts the records into a dataframe, attaches the target hour, computes confidence scores, and appends the dataframe to the baseline record list. It then evaluates baseline MAE and RMSE over rows where both prediction and true value are finite.

The graph input table is then prepared by renaming the prediction and label columns into the names expected by `gnn_model.py`. The graph pipeline is called with the original dataframe, target date, excluded zones, device, zone count, output path, prediction dataframe, zone confidence dictionary, and disabled plotting flag. The resulting graph dataframe is annotated with the current target hour and stored. The graph metric dictionary is converted into per-hour graph metrics containing baseline MAE/MSE and graph-refined MAE/MSE.

After all rolling steps finish, the implementation concatenates baseline and graph records, writes CSV files, computes overall baseline MAE/RMSE and graph MAE/RMSE, and writes a concise text summary. This structure ensures that the system produces both detailed row-level outputs for later analysis and compact aggregate metrics for reporting.

4.8 Checkpoint and Incremental-Update Handling

Checkpointing is implemented in the temporal manager. For each zone, the model path, scaler path, and metadata path are generated from the checkpoint directory and zone identifier. The manager considers a zone to have a checkpoint only when both model and scaler files exist. This avoids loading a partial checkpoint where the neural model exists but the scaler is missing, or vice versa.

The default persistent implementation trains a zone model only when no valid checkpoint exists. If a checkpoint already exists, the model is loaded and used for prediction. If the metadata indicates that the model was trained only up to an older context end, the persistent manager emits a warning that incremental updates are disabled and continues to reuse the existing checkpoint. This behaviour is consistent with the default import in the rolling driver, where the persistent manager is active and the incremental manager import line is commented out.

The project documentation notes that an incremental manager is also provided in `code/persistent_mul` In that version, new rolling windows can trigger lightweight `incremental_update` operations, with the intended purpose of improving MAE/MSE stability between full training runs. The documented activation method is to replace the persistent import in the rolling script with the incremental manager import. Therefore, the implementation should be described carefully: persistent reuse is the default operating mode of the current driver script, while incremental adaptation is implemented as an alternative version that can be enabled by changing the import.

This distinction matters for reproducibility. If the thesis reports results from the current default script, they should be interpreted as persistent-checkpoint rolling results. If results are obtained after switching to `persistent_multiscale_incre_confi.py`, they should be labelled as incremental-update results. The codebase is structured so that both modes share the same rolling driver and graph-refinement pipeline, but the temporal manager behaviour changes.

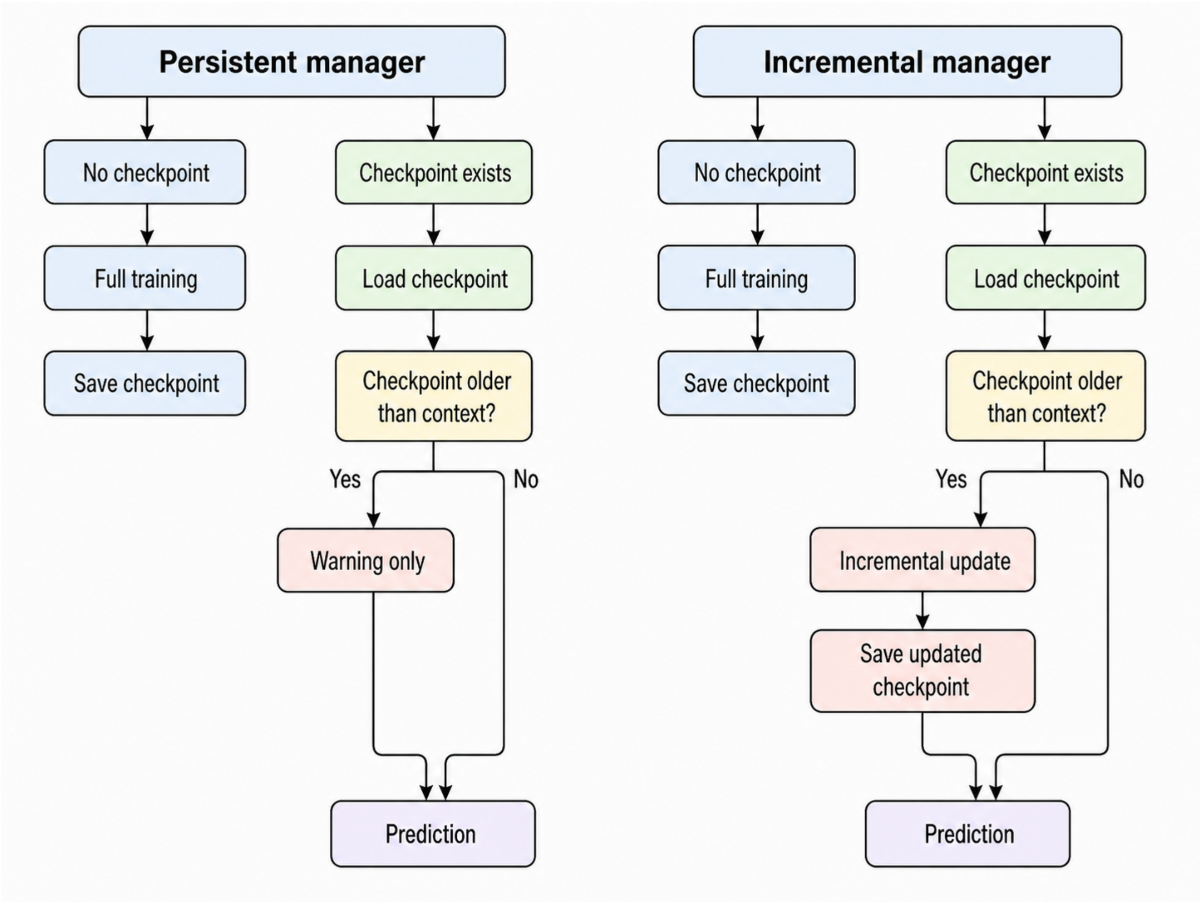

Figure 4.8: Comparison between the persistent and incremental temporal managers, showing how stale checkpoints are either reused with a warning or updated through lightweight incremental training before prediction.

4.9 Persistent and Incremental Multi-Scale Managers

The temporal forecasting component is implemented in two closely related manager variants. The first is the persistent multi-scale manager, implemented in `persistent_multiscale_confi.p` This version performs full training when no checkpoint exists and then reuses the stored model, scaler, and metadata during subsequent rolling steps. It is therefore designed around checkpoint persistence rather than online adaptation. In the current rolling driver, this persistent manager remains the default import, while the alternative incremental manager import is present but commented out. This means that the default experimental configuration should be interpreted as persistent checkpoint reuse unless the import line is manually changed.

The second variant is the incremental multi-scale manager, implemented in `persistent_multiscale_in` This version preserves the same overall temporal architecture as the persistent manager, including the four temporal branches, branch-level dropout, multi-scale fusion, GRU-based prediction pathway, and Monte Carlo Dropout uncertainty interface. However, it extends the manager logic by adding lightweight incremental updating after the first full training run. Its purpose is to reduce the instability that can occur when a fixed checkpoint is reused over a long rolling horizon while new observations continue to arrive.

Structurally, the incremental version still uses the same `MultiScaleModel` class design. The model contains LSTM encoders for the 1-day and 1-week branches, a Transformer-based encoder for the 1-month branch, dropout layers for Monte Carlo sampling, a fusion layer for combining longer-term temporal representations, and a GRU pathway for producing the final next-hour prediction. It also retains the Monte Carlo interfaces for repeated stochastic prediction and branch-embedding variance estimation. Therefore, the incremental version should not be understood as a different forecasting architecture. Rather, it is a different training and model-management strategy built around the same multi-scale temporal backbone.

The main implementation difference appears in the configuration and orchestration logic. The incremental manager adds several parameters that do not appear in the basic persistent version, including `epochs_incremental`, `lr_incremental`, `weight_decay`, and `grad_clip`. These parameters separate full initial training from subsequent lightweight adaptation. Full training remains relatively more expensive and is used when no checkpoint exists. Incremental training, by contrast, is deliberately small: it is intended to update an existing model with newly available rolling-window observations rather than to rebuild the model from the beginning.

The core extension is the `incremental_update` routine. When the manager detects that the previous training context is older than the new forecasting context, it loads the existing checkpointed model and performs a short additional optimisation step over newly arrived hourly samples. The update interval is defined by the gap between `prev_until` and `new_until`. Instead of reconstructing the entire historical training process, the function builds incremental windows only for the newly available timestamps in this interval. This design makes the update computationally cheaper than full retraining while still allowing the model parameters to respond to recent observations.

The incremental update procedure also refits the scaler using recent historical data before applying the update. In the provided implementation, this recent fitting range is based on the final 30 days before the new context end. The dense hourly series is rebuilt up to `new_until`, the scaler is fitted on recent history, the newly available training windows are assembled, and the existing model is fine-tuned using a Smooth L1 loss. The use of Smooth L1 loss in the incremental stage is appropriate because rolling updates may be exposed to local spikes or sparse-zone irregularities, and Smooth L1 is less sensitive to extreme residuals than pure squared error.

Another important detail is that the incremental version applies gradient clipping during fine-tuning. This prevents a small number of recent windows from producing excessively large parameter updates. In the context of online or semi-online forecasting, this is a practical safeguard: the purpose of incremental learning is to stabilise the model under gradual drift, not to let a single short update overwrite the previously learned temporal structure. The addition of weight decay also regularises the update and further reduces the risk of overfitting to the newest observations.

The incremental version keeps the confidence-related Monte Carlo logic from the persistent confidence-based implementation. During full training and incremental updating, the model can collect branch embeddings under Monte Carlo Dropout and derive confidence-style weights from the variance of those embeddings. In the current code, the auxiliary-head structure is declared in the model, and branch-confidence quantities are computed. However, the auxiliary loss contribution is effectively neutralised in the present implementation because the auxiliary term is multiplied by zero before being added to the main loss. Therefore, the thesis should describe this part carefully: the incremental manager contains the architectural and computational hooks for confidence-weighted auxiliary learning, but the active optimisation objective is currently dominated by the main prediction loss.

The orchestration logic makes the behavioural difference between the two managers clear. In the persistent manager, if a checkpoint already exists and its metadata is older than the current context, the system warns that incremental updates are disabled and continues to reuse the existing checkpoint. In the incremental manager, the same situation triggers `incremental_update`. After updating, the model, scaler, and metadata are saved again, and the new metadata records the updated context end. This means that, once enabled, the incremental manager maintains a moving checkpoint state that follows the rolling prediction horizon more closely than the static persistent version.

From an implementation perspective, this design has an important advantage: the rest of the rolling pipeline does not need to be rewritten. The top-level driver calls the same manager interface, especially `train_and_predict_if_needed` and `predict_with_uncertainty`. The difference lies inside the manager implementation. If the persistent manager is imported, checkpoints are reused without fine-tuning. If the incremental manager is imported, the manager updates stale checkpoints before prediction. This makes the incremental mechanism easy to activate and compare experimentally, because it changes the temporal model-management behaviour without changing the graph refinement pipeline.

In this thesis, the two variants should therefore be reported separately. The persistent version is the default baseline used by the current rolling script. It is computationally efficient and demonstrates the benefit of checkpoint reuse under rolling evaluation. The incremental version is an implemented extension that improves the realism of the system by allowing newly observed time windows to update the temporal model. It should be treated as a stronger online-adaptive configuration rather than as the default unless the script is explicitly switched to import `persistent_multiscale_incre_confi.py`. This distinction is important for reproducibility, because the same rolling driver can produce different temporal behaviour depending on which manager is active.

4.10 Output Artefacts and Evaluation Files

The implementation produces several categories of output artefacts. The first category consists of temporal baseline outputs. `predictions_rolling_mc.csv` contains per-zone rolling records including prediction, true value, uncertainty, variance, error status, historical mean features, confidence, and target hour. `hourly_metrics_gru.csv` contains per-hour baseline MAE, RMSE, and mean true demand. These files are generated in the rolling driver after all rolling steps are complete.

The second category consists of graph-refinement outputs. For each target hour, `run_gnn_pipeline()` can write a single-hour CSV named according to the target timestamp. This file includes `PULocationID`, `ZoneName`, `GRU_Pred`, `Refined_Pred`, `True_Value`, and `Confidence`. Across all rolling steps, the driver writes `gnn_refined_predictions.csv`, which aggregates graph-refined rows over time, and `hourly_metrics_gnn.csv`, which stores per-hour MAE and MSE for both baseline and graph-refined predictions.

The third category is the summary metric file. The driver computes overall baseline MAE/RMSE from all valid baseline rows and overall graph MAE/RMSE from all valid graph-refined rows, then writes these values into `overall_metrics_gnn.txt`. This gives a compact experiment summary, while the CSV outputs preserve sufficient detail for later plotting, error analysis, and ablation comparisons.

The graph module also supports optional visual diagnostics. If plotting is enabled, it produces bar charts comparing MAE and MSE between baseline and graph-refined predictions, and a scatter plot comparing true values with both GRU and GNN predictions. In the rolling driver, plotting is disabled during automated execution to avoid interrupting batch runs.

4.11 Implementation-Level Limitations

Several implementation-level limitations should be stated openly. First, the default graph stage trains and evaluates on the same hourly graph snapshot. The graph pipeline constructs residual labels from the current target-hour true values, trains the GraphSAGE model for that hour, and then evaluates refined predictions on the same node set. This is useful for measuring the ability of the graph module to fit residual structure in the current graph snapshot, but it is not a strict temporally separated graph-generalisation protocol. The implementation itself makes this visible because each call to `run_gnn_pipeline()` performs graph construction, training, evaluation, metric computation, and output generation within one target-hour call.

Second, although the graph pipeline constructs a confidence-derived `edge_weight` field when `zone_confidence` is available, the defined `MultiScaleGraphSAGE` class uses standard `SAGEConv` calls that only consume node features and `edge_index`. Thus, confidence affects graph training through node-level loss weighting, but the current message-passing operator does not explicitly use confidence or OD magnitude as an edge weight.

Third, the per-zone temporal implementation increases the number of model checkpoints.

Each zone can have its own model, scaler, and metadata file. This supports local specialisation but creates storage and management overhead. The checkpoint manager mitigates this through simple file naming and automatic loading, but larger deployments would require more systematic experiment tracking.

Fourth, the default persistent manager does not update an existing model when the rolling context advances. It warns that incremental updates are disabled and continues to use the previous checkpoint. This is not a hidden bug but an explicit design of the persistent version. The incremental variant is available separately and should be evaluated as a distinct configuration.

4.12 Summary

In implementation terms, the project is a modular rolling forecasting system. The driver script prepares data, builds adjacency and confidence scores, controls rolling execution, calls the temporal manager, calls the graph pipeline, and writes outputs. The temporal module implements a checkpointed multi-scale per-zone predictor with LSTM, Transformer, GRU, dropout, scaling, training, inference, and Monte Carlo uncertainty estimation. The graph module implements GraphSAGE residual refinement over an OD-derived graph, using temporal predictions, confidence weights, and historical features as node inputs.

The most important implementation strength is that the system is not a single isolated model script. It is a complete experimental pipeline with explicit data preparation, temporal prediction, uncertainty estimation, confidence fusion, graph refinement, rolling evaluation, and reproducible outputs. At the same time, the codebase clearly exposes the boundaries of the current implementation: persistent reuse is the default, incremental adaptation exists as an alternative import, graph refinement currently trains and evaluates on the same hourly node set, and edge weights are prepared but not yet explicitly used inside GraphSAGE message passing. These implementation details are important because they determine how the experimental results should be interpreted in the following chapter.

Chapter 5

Experiments

5.0.1 Experimental Objectives

The purpose of the experimental study is to evaluate whether the proposed forecasting framework improves hourly taxi demand prediction through three progressive design choices: multi-scale temporal modelling, graph-based spatial refinement, and incremental adaptation under a rolling forecasting protocol. Rather than treating the proposed system as a single indivisible model, the experiments are organised as a staged evaluation. This allows each component to be tested independently before the complete pipeline is assessed.

The first objective is to evaluate the effectiveness of the proposed multi-scale temporal predictor against standard temporal baselines. In this project, taxi demand is represented as an hourly count sequence for each pickup taxi zone. A single temporal model, such as an LSTM, GRU, or Transformer, can learn temporal dependency from historical demand, but each architecture tends to favour a particular type of temporal pattern. LSTM and GRU models are effective for recurrent sequence modelling, while Transformer models can capture longer-range temporal relationships through self-attention. However, taxi demand often contains multiple temporal scales at the same time, including short-term hourly fluctuation, daily periodicity, weekly commuting patterns, and longer-term monthly trends. The proposed temporal backbone is therefore designed as a multi-scale model, using separate temporal branches to encode different historical windows before fusing them into a GRU-based prediction head. The first experiment therefore asks whether the multi-scale structure provides a measurable improvement over single-model temporal baselines.

The second objective is to examine whether graph-based refinement improves the predictions produced by the temporal model. The base temporal model treats each taxi zone primarily as an independent time series. Although this captures zone-specific demand history, it does not fully model the dependency between spatially or functionally related zones. In an urban mobility setting, demand in one zone can be correlated with demand in neighbouring or flow-connected zones. For example, a demand increase in a transport hub, business district, or entertainment area may affect nearby zones through passenger movement and vehicle redistribution. To account for this dependency, the second stage of the proposed framework applies a GraphSAGE-based residual refinement model on a taxi-zone graph constructed from origin–destination flow relationships. The GNN does not replace the temporal prediction; instead, it learns a correction term on top of the temporal output. The second experiment therefore compares the temporal-only prediction against the graph-refined prediction.

The third objective is to test whether confidence weighting improves the robustness of graph refinement. The proposed system estimates node-level confidence using three complementary signals: a prior score based on historical data sufficiency, a stability score based on MC Dropout variance, and a neighbourhood consistency score based on agreement with graph neighbours. These three components are combined into a single confidence value for each taxi zone. During GNN training, this confidence value is used as a sample weight so that more reliable nodes have stronger influence on the residual learning process, while uncertain or unstable nodes are down-weighted. This experiment evaluates whether confidence-weighted GNN refinement performs better than unweighted GNN refinement, especially in rolling prediction where model uncertainty and sparse demand zones can introduce instability.

The fourth objective is to evaluate the role of incremental training in the rolling forecasting setting. The non-incremental version of the system trains a model and then reuses the stored checkpoint for later target hours. This reduces computational cost, but it may become less adaptive as the rolling target moves forward and new observations become available. The incremental version is intended to update the model using newly available recent data, rather than repeatedly training from scratch or reusing an increasingly stale checkpoint. The final experiment therefore compares the persistent non-incremental implementation with the incremental implementation. The main focus is not only average error reduction, but also stability over time, measured by the variation of hourly errors across rolling steps.

Accordingly, the experimental section is structured around the following research questions:

- **RQ1:** Does the proposed multi-scale temporal model outperform single temporal baselines such as LSTM, GRU, and Transformer?
- **RQ2:** Does GraphSAGE-based residual refinement improve the prediction accuracy of the temporal model?
- **RQ3:** Does confidence-weighted graph refinement improve robustness compared with unweighted graph refinement?
- **RQ4:** Does incremental training improve rolling forecasting stability compared with non-incremental checkpoint reuse?

These research questions define a progressive experimental path from simple baselines to the complete system. The intended comparison order is: single temporal models, multi-scale temporal model, multi-scale model with GNN refinement, multi-scale model with confidence-weighted GNN refinement, and finally the full confidence-weighted model with incremental updating. This structure ensures that any improvement can be attributed to a specific modelling choice rather than only to the final combined architecture.

5.0.2 Dataset and Preprocessing

The experiments are conducted on taxi trip records aggregated at the taxi-zone level. Each raw trip record contains at least a pickup timestamp and a pickup location identifier. The pickup timestamp is used to assign the trip to an hourly time interval, while the pickup location identifier is used to assign the trip to a taxi zone. The forecasting task is formulated as one-step-ahead hourly demand prediction. For each taxi zone and each target hour, the model predicts the number of pickup events that will occur in that zone during the next hour.

The raw input file is stored as a Parquet file and contains the fields `pickup_datetime` and `PULocationID`. During preprocessing, the timestamp is converted into a datetime object and rounded down to the nearest hour. This produces an hourly timestamp field, denoted as `datetime`, which is used as the temporal index for aggregation. The demand value for a taxi zone at a given hour is then computed as the number of pickup records whose `PULocationID` equals the zone identifier and whose pickup time falls within that hour. Formally, for zone i and hour t, the demand value is defined as:

$$y_{i,t} = \sum_{r \in \mathcal{D}} \mathbb{I}(z_r = i)\mathbb{I}(\tau_r \in [t, t+1)),$$

where $\mathcal{D}$ denotes the set of trip records, z_r is the pickup zone of record r, τ_r is its pickup time, and $\mathbb{I}(\cdot)$ is an indicator function.

Several zones are excluded from the experiments to avoid invalid or highly irregular locations. In the current implementation, the excluded zone list is:

$$\{103, 104, 105, 46, 264, 265\}.$$

These zones are removed before both temporal modelling and graph refinement. This ensures that all evaluated models operate on the same set of valid taxi zones. After filtering, the remaining records are grouped by `PULocationID` and `datetime` to form the core hourly demand table.

The temporal model uses a fixed historical context to predict the next hour. The main multi-scale model uses four temporal views: a short one-hour view, a daily view of 24 hours, a weekly view of 168 hours, and a monthly-scale view of 720 hours. These windows are selected to reflect the expected periodicity of urban taxi demand. The one-hour view captures immediate demand state, the daily view captures intra-day repetition, the weekly view captures weekday and weekend structure, and the monthly view captures longer-term trend information. Missing hours within a zone-level sequence are filled with zero demand, producing a dense hourly time series for each zone. This is necessary because the neural models require continuous input windows with fixed length.

Before model training, the zone-level passenger count series is normalised using min–max scaling. Scaling is fitted on the historical portion available before the prediction target, so that future target information is not used during preprocessing. The scaled series is then converted into sliding windows. For each valid training window, the model receives historical sequences from the four temporal scales and predicts the demand value at the next forecasting step. The prediction horizon is set to one hour, which is consistent with the objective of short-term operational demand forecasting.

In addition to temporal features, the graph refinement stage requires a taxi-zone graph. This graph is constructed from an origin–destination flow matrix stored as `edge_weight_matrix_with_f` Each node represents a taxi zone, and an edge is created when the corresponding flow relationship between two zones is non-zero. The zone lookup table, stored as `taxi-zone-lookup.csv`, maps between zone names and numeric location identifiers. This mapping is required because the graph matrix is indexed by zone names, whereas the prediction results are indexed by `PULocationID`. During GNN preparation, the adjacency matrix is converted into a sparse edge representation suitable for graph neural network computation.

For each rolling target hour, the temporal model first generates a base prediction for each valid zone. The output table contains the zone identifier, predicted demand, true demand, MC Dropout standard deviation, variance, and additional historical mean features. The historical mean features are computed over three windows:

$$24\text{h}, \quad 168\text{h}, \quad 720\text{h}.$$

These features provide the GNN with simple demand-level context in addition to the base model prediction. The GNN input for each node therefore includes the temporal prediction, a node weight or confidence value, and optionally the historical mean features. The supervised target for the GNN is not the raw demand value directly, but the residual between the ground truth and the temporal prediction:

$$r_{i,t} = y_{i,t} - \hat{y}_{i,t}^{\text{temp}}.$$

The refined prediction is then obtained by adding the predicted residual back to the temporal prediction:

$$\hat{y}_{i,t}^{\text{refined}} = \hat{y}_{i,t}^{\text{temp}} + \hat{r}_{i,t}.$$

The experiments follow a rolling forecasting protocol. A starting target time is selected, and the system advances by one hour at each rolling step. At every step, all valid zones are predicted for the current target hour, evaluated against observed counts, and then passed to the graph refinement stage. This protocol is closer to practical deployment than a single static train–test split because it evaluates the model repeatedly as the forecast origin moves forward in time.

5.0.3 Evaluation Protocol and Metrics

All models are evaluated under the same one-hour-ahead forecasting setting. For a target hour t, each model is allowed to use historical observations up to the context endpoint before the target. The target value is the observed passenger demand during hour t. This protocol avoids future information leakage and ensures that the comparison between models is fair. The same filtered taxi-zone set, same rolling target hours, and same historical context definition are used across the temporal baselines, the proposed multi-scale model, and the graph-refined variants.

The first group of experiments compares temporal-only models. The baseline models include LSTM, GRU, and Transformer predictors. Each baseline receives historical demand sequences and predicts the next-hour demand for each taxi zone. The proposed multi-scale temporal model is then evaluated under the same rolling target setting. The purpose of this comparison is to determine whether explicit multi-scale temporal encoding improves predictive performance over single-architecture sequence models. No graph information is used in this first comparison, so the results isolate the contribution of the temporal backbone.

The second group of experiments evaluates graph refinement. The best temporal model from the first group is used as the base predictor. Its output is passed to the GraphSAGE refinement module, which learns zone-level residual corrections on the OD-flow graph. The temporal-only prediction and the GNN-refined prediction are evaluated on the same target hours and zones. This comparison isolates the effect of spatial message passing and residual correction. The graph-refined model is considered beneficial if it reduces error relative to the temporal-only prediction while preserving stable behaviour across rolling steps.

The third group of experiments evaluates confidence weighting. In the unweighted setting, all valid nodes contribute equally to the GNN loss. In the confidence-weighted setting, each node is weighted according to its confidence score. The confidence score combines historical data sufficiency, MC Dropout stability, and neighbourhood consistency. This experiment is designed to test whether uncertain nodes should have reduced influence during graph residual learning. In addition to average prediction error, this comparison considers the stability of error across rolling steps, because confidence weighting is expected to reduce extreme fluctuations caused by unreliable zones.

The fourth group of experiments compares non-incremental and incremental rolling training. In the non-incremental setting, the model checkpoint is reused after the initial training stage. In the incremental setting, the model is updated using newly available recent observations as the target hour moves forward. This comparison evaluates whether incremental updating can improve adaptation to recent demand changes. Since incremental training is primarily motivated by rolling stability, the analysis includes both average error and temporal error variation.

The primary evaluation metrics are mean absolute error, root mean squared error, and mean squared error. Mean absolute error is defined as:

$$\mathrm{MAE} = \frac{1}{N}\sum_{i=1}^{N} |y_i - \hat{y}_i| \,,$$

where N is the number of evaluated zone-hour predictions, y_i is the observed demand, and $\hat{y}_i$ is the predicted demand. MAE is the main metric because it is directly interpretable as the average absolute passenger-count error.

Mean squared error is defined as:

$$\mathrm{MSE} = \frac{1}{N}\sum_{i=1}^{N} (y_i - \hat{y}_i)^2 \,.$$

MSE penalises large errors more strongly than MAE and is therefore useful for detecting whether a model produces severe mistakes in high-demand or unstable zones.

Root mean squared error is defined as:

$$\mathrm{RMSE} = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2}.$$

RMSE has the same unit as the target demand value and is easier to interpret than MSE while still giving greater weight to large errors.

For the rolling setting, the metrics are computed at two levels. First, hourly metrics are computed for each target hour using all valid taxi zones in that hour. This produces an error sequence over rolling steps:

$$\{\mathrm{MAE}_1, \mathrm{MAE}_2, \ldots, \mathrm{MAE}_T\}.$$

Second, overall metrics are computed by aggregating all valid zone-hour predictions across the complete rolling horizon. The hourly metrics are used to analyse temporal stability, while the overall metrics are used to compare average predictive performance.

To evaluate rolling stability, the standard deviation of hourly MAE is also reported:

$$\sigma_{\mathrm{MAE}} = \sqrt{\frac{1}{T}\sum_{t=1}^{T}\left(\mathrm{MAE}_t - \overline{\mathrm{MAE}}\right)^2}.$$

A lower value indicates that the model produces more consistent performance across rolling target hours. This metric is particularly important for comparing incremental and non-incremental training, because a model with slightly better average MAE but highly unstable hourly behaviour may be less suitable for deployment than a model with stable performance.

The final evaluation tables therefore report MAE, RMSE, MSE, hourly MAE standard deviation, and the worst-hour MAE where appropriate. The worst-hour MAE is included to identify whether a method reduces extreme failure cases:

$$\mathrm{WorstHourMAE} = \max_{t\in\{1,\ldots,T\}} \mathrm{MAE}_t.$$

Together, these metrics provide a more complete evaluation than average error alone. MAE and RMSE measure overall predictive accuracy, MSE highlights large errors, hourly

MAE standard deviation measures rolling stability, and worst-hour MAE captures the robustness of the model under difficult target hours.

5.0.4 Experiment 1: Temporal Baseline Comparison

The first experiment evaluates whether the proposed multi-scale temporal model improves one-hour-ahead taxi demand forecasting compared with standard single temporal models. Four temporal-only models are compared in this experiment: LSTM, GRU, Transformer, and the proposed multi-scale temporal model. No graph refinement or confidence-weighted GNN correction is applied in this stage. Therefore, the purpose of this experiment is to isolate the contribution of the temporal backbone before introducing spatial refinement in later experiments.

All four models are evaluated under the same 24-hour rolling forecasting protocol. The rolling period starts at `2021-07-05 00:00:00` and continues for 24 target hours until `2021-07-05 23:00:00`. For each target hour, predictions are generated for 259 valid taxi zones, resulting in 6216 zone-hour predictions for each model. The same filtered taxi-zone set, same preprocessing pipeline, and same one-hour-ahead prediction horizon are used for all models.

The baseline models are defined as follows. The LSTM baseline uses a recurrent architecture to model temporal dependency in the historical demand sequence. The GRU baseline provides a simpler recurrent alternative with fewer gating operations. The Transformer baseline uses self-attention to model temporal dependency over the historical window. The proposed multi-scale temporal model combines multiple temporal resolutions, including short-term, daily, weekly, and monthly-scale views, before producing the final next-hour prediction. This design is intended to capture both immediate demand fluctuation and longer periodic demand patterns.

Table 5.1 reports the overall results across all 6216 zone-hour predictions. The proposed multi-scale model achieves the best overall MAE, RMSE, and MSE among the four temporal models. Its overall MAE is 17.65, compared with 26.66 for LSTM, 27.35 for GRU, and 23.05 for Transformer. This corresponds to a MAE reduction of 33.8% over LSTM, 35.5% over GRU, and 23.4% over Transformer. The same trend is observed for RMSE and MSE, indicating that the improvement is not limited to average absolute error but also reduces larger prediction deviations.

Table 5.1: Overall performance comparison of temporal-only models over the 24-hour rolling forecasting period.

Model	**MAE ↓**	**RMSE ↓**	**MSE ↓**	**Hourly MAE Std. ↓**	**Worst-hour MAE ↓**
LSTM	26.66	44.54	1983.94	11.64	49.93
GRU	27.35	44.87	2013.18	11.44	50.55
Transformer	23.05	38.39	1473.88	6.95	36.79
Multi-scale	**17.65**	**32.14**	**1032.73**	7.07	**34.73**

Among the three single temporal baselines, the Transformer performs best, with an overall MAE of 23.05 and RMSE of 38.39. This suggests that attention-based temporal modelling is more effective than the recurrent LSTM and GRU baselines for this rolling forecasting period. However, the proposed multi-scale model still outperforms the Transformer by a clear margin. Compared with the Transformer, the multi-scale model reduces MAE by 5.40 absolute points and RMSE by 6.25 absolute points. In percentage terms, this is equivalent to a 23.4% reduction in MAE, a 16.3% reduction in RMSE, and a 29.9% reduction in MSE.

The improvement over LSTM and GRU is larger. The LSTM baseline obtains an overall MAE of 26.66, while the GRU baseline obtains an overall MAE of 27.35. The proposed multi-scale model reduces MAE by 9.01 compared with LSTM and by 9.70 compared with GRU. This result indicates that a single recurrent architecture is not sufficient to capture the full temporal structure of taxi demand in this setting. Although LSTM and GRU can model sequential dependency, they are less effective at representing demand patterns that occur simultaneously at different temporal scales.

Table 5.2: Relative improvement of the proposed multi-scale model over temporal baselines.

Baseline	**MAE Reduction**	**RMSE Reduction**	**MSE Reduction**
LSTM	33.8%	27.9%	47.9%
GRU	35.5%	28.4%	48.7%
Transformer	23.4%	16.3%	29.9%

The hourly MAE results provide further evidence for the advantage of the multi-scale design. Across the 24 rolling target hours, the multi-scale model obtains the lowest MAE in 23 out of 24 hours. The only exception is the first target hour, `2021-07-05 00:00:00`, where the GRU baseline achieves a slightly lower MAE of 25.07 compared with 26.84 for the multi-scale model. However, after this initial hour, the multi-scale model consistently produces the lowest hourly MAE among all four temporal models.

Table 5.3 summarises the hourly MAE values for the four temporal models. The performance gap is especially clear between `06:00` and `10:00`. For example, at `08:00`, the multi-scale model achieves a MAE of 12.84, while the Transformer, LSTM, and GRU obtain 26.64, 36.75, and 37.21 respectively. Similarly, at `09:00`, the multi-scale model obtains a MAE of 12.02, compared with 23.41 for Transformer, 31.08 for LSTM, and 31.92 for GRU. These results suggest that the proposed model is more effective at adapting to changing demand levels during the morning transition period.

The hourly pattern also shows that all models experience larger errors during the early morning period, especially around `03:00` to `05:00`. For the multi-scale model, the worst hourly MAE is 34.73 at `04:00`. This period also has relatively low average demand, which makes the prediction task more sensitive to local fluctuations and sparse observations. After `08:00`, the multi-scale model becomes substantially more stable, with hourly MAE mostly remaining between 11.68 and 16.54 for the rest of the day.

The hourly MAE standard deviation gives an additional view of rolling stability. The Transformer has the lowest hourly MAE standard deviation, 6.95, while the multi-scale model has a slightly higher value of 7.07. However, this should not be interpreted as the Transformer being better overall. The Transformer is more consistent in the sense that its hourly errors vary slightly less, but its errors are consistently higher than the multi-scale model across every target hour. In contrast, the multi-scale model achieves much lower average error and the lowest worst-hour MAE. Therefore, the multi-scale model provides a better accuracy–stability trade-off in this experiment.

Overall, the temporal baseline experiment demonstrates that the proposed multi-scale design is more effective than using a single LSTM, GRU, or Transformer model. The result supports the motivation that taxi demand contains multiple temporal patterns that cannot be fully captured by a single temporal scale. By combining short-term, daily, weekly, and monthly-scale temporal information, the proposed temporal model produces more accurate predictions before any graph-based spatial correction is applied. This establishes a stronger base predictor for the subsequent GNN refinement and confidence-

Table 5.3: Hourly MAE comparison across the 24-hour rolling forecasting period.

Target Hour	**LSTM**	**GRU**	**Transformer**	**Multi-scale**
2021-07-05 00:00	28.43	**25.07**	26.99	26.84
2021-07-05 01:00	32.38	29.89	28.42	**25.50**
2021-07-05 02:00	32.08	31.69	28.71	**22.45**
2021-07-05 03:00	39.94	41.57	33.95	**31.81**
2021-07-05 04:00	49.04	50.34	36.79	**34.73**
2021-07-05 05:00	49.93	50.55	35.77	**30.17**
2021-07-05 06:00	46.61	46.78	33.39	**23.90**
2021-07-05 07:00	41.41	41.53	29.64	**17.77**
2021-07-05 08:00	36.75	37.21	26.64	**12.84**
2021-07-05 09:00	31.08	31.92	23.41	**12.02**
2021-07-05 10:00	25.09	25.98	19.63	**12.61**
2021-07-05 11:00	20.25	21.81	18.27	**12.74**
2021-07-05 12:00	19.05	20.62	17.52	**13.25**
2021-07-05 13:00	18.20	19.37	17.80	**13.20**
2021-07-05 14:00	16.34	16.90	16.57	**14.31**
2021-07-05 15:00	15.00	15.00	15.25	**14.01**
2021-07-05 16:00	14.59	15.19	15.87	**13.70**
2021-07-05 17:00	15.01	15.87	16.91	**12.53**
2021-07-05 18:00	15.07	15.09	15.82	**11.68**
2021-07-05 19:00	13.66	14.97	15.95	**12.18**
2021-07-05 20:00	17.28	19.43	18.93	**13.01**
2021-07-05 21:00	17.23	19.53	17.40	**12.82**
2021-07-05 22:00	19.18	21.67	19.51	**13.06**
2021-07-05 23:00	26.28	28.42	24.07	**16.54**

weighted experiments.

5.0.5 Experiment 2: Effect of Graph-based Residual Refinement

This experiment evaluates whether graph-based residual refinement improves the prediction accuracy of the multi-scale temporal model. The temporal-only model is used as the base predictor, and the GraphSAGE module is applied as a second-stage correction model on the taxi-zone graph. The purpose of this experiment is to isolate the contribution of the graph refinement stage before introducing confidence weighting and incremental training in later experiments.

Three model variants are compared in this experiment:

- **T4: Multi-scale Temporal**. This is the temporal-only model. It uses the multi-scale temporal architecture to generate the base prediction for each taxi zone, but does not use graph information.
- **G1: Multi-scale + GNN**. This model uses the T4 prediction as the base prediction and applies GraphSAGE to learn a residual correction on the taxi-zone graph.
- **G2: Multi-scale + GNN + History Features**. This model extends G1 by adding historical mean features, including `mean_24h`, `mean_168h`, and `mean_720h`, to the GNN input.

In both G1 and G2, the GNN is trained to predict the residual between the observed demand and the temporal prediction. Let $\hat{y}_{i,t}^{\text{temp}}$ denote the prediction produced by the multi-scale temporal model for zone i at target hour t, and let $y_{i,t}$ denote the observed demand. The residual target is defined as:

$$r_{i,t} = y_{i,t} - \hat{y}_{i,t}^{\text{temp}}.$$

The graph model predicts $\hat{r}_{i,t}$, and the final refined prediction is obtained as:

$$\hat{y}_{i,t}^{\text{refined}} = \hat{y}_{i,t}^{\text{temp}} + \hat{r}_{i,t}.$$

Therefore, the GNN does not replace the temporal model. Instead, it operates as a graph-based residual correction module.

The experiment is conducted over 24 rolling target hours from `2021-03-05 00:00:00` to `2021-03-05 23:00:00`. For each target hour, 258 valid taxi-zone nodes are used. The graph data are split into 155 training nodes, 52 validation nodes, and 51 test nodes. The reported summary metrics are calculated over the rolling test predictions.

Table 5.4: Experiment 2 results: effect of graph-based residual refinement.

Model	**MAE ↓**	**RMSE ↓**	**MSE ↓**	**MAE Improveme**
T4: Multi-scale Temporal	33.4804	51.9800	2701.9194	
G1: Multi-scale + GNN	16.2106	30.5118	930.9674	51.58
G2: Multi-scale + GNN + History Features	16.2040	30.3489	921.0562	51.60

The results show that graph-based residual refinement leads to a substantial improvement over the temporal-only model. Compared with T4, G1 reduces MAE from 33.4804 to 16.2106, corresponding to a 51.58% reduction in absolute error. RMSE is also reduced from 51.9800 to 30.5118, while MSE decreases from 2701.9194 to 930.9674. This indicates that the temporal-only model still contains systematic zone-level errors that can be corrected by using spatial information from the taxi-zone graph.

The improvement from T4 to G1 is the most important finding of this experiment. Since G1 differs from T4 only by the addition of the GraphSAGE residual refinement stage, the large reduction in error demonstrates that the graph structure provides useful information for correcting temporal predictions. This supports the assumption that taxi demand is not only temporally dependent, but also spatially coupled across zones. In other words, the demand error in one zone can be partially explained by patterns in neighbouring or flow-connected zones.

Adding historical mean features in G2 produces only a very small additional improvement over G1. The MAE decreases from 16.2106 to 16.2040, which is an absolute reduction of only 0.0066. RMSE decreases from 30.5118 to 30.3489, and MSE decreases from 930.9674 to 921.0562. Although the direction of the change is positive, the difference between G1 and G2 is marginal. Therefore, the main performance gain should be attributed to graph-based residual learning rather than to the additional historical mean features.

This result is reasonable because the base multi-scale temporal model has already encoded short-term, daily, weekly, and longer-term temporal patterns through its multi-branch architecture. As a result, simple historical averages such as `mean_24h`, `mean_168h`, and `mean_720h` may provide partially redundant information to the GNN. These features describe the general demand level of each zone, but they do not necessarily capture complex temporal variations beyond what is already represented in the temporal prediction itself.

The hourly results further support this conclusion. G1 improves over T4 in 23 out of 24 rolling target hours, while G2 improves over T4 in all 24 target hours. The average hourly MAE reduction is 45.48% for G1 and 46.06% for G2. The largest improvement occurs during the early morning hours, where the temporal-only model produces relatively large errors and the GNN correction is able to reduce them substantially. For example, at `2021-03-05 02:00:00`, the T4 MAE is 46.2276, while G1 reduces it to 5.5914 and G2 reduces it to 6.2358. This demonstrates that graph refinement is particularly effective when the temporal-only model makes large zone-level prediction errors.

However, the comparison between G1 and G2 is mixed at the hourly level. G2 achieves lower MAE than G1 in 9 out of 24 target hours, while G1 performs better in the remaining 15 hours. This confirms that historical mean features do not provide a consistent advantage across all rolling steps. In some hours, these features slightly improve the GNN correction, while in others they may introduce redundant or weakly informative signals. Therefore, G2 can be interpreted as a minor extension of G1 rather than a fundamentally stronger model.

The stability of the graph-refined models is also improved compared with the temporal-only baseline. The standard deviation of hourly MAE decreases from 12.9205 for T4 to 7.8025 for G1 and 7.7846 for G2. This suggests that graph refinement not only reduces the average prediction error, but also makes the rolling performance more stable across target hours. This is important for practical forecasting systems, where consistent performance is often as important as the average error.

Overall, Experiment 2 demonstrates that graph-based residual refinement is highly effective for improving multi-zone taxi demand forecasting. The large performance gap between T4 and G1 confirms the value of introducing spatial structure after temporal prediction. The small difference between G1 and G2 suggests that historical mean features are only auxiliary and should not be treated as the main source of improvement. Therefore, the key conclusion from this experiment is:

$$\mathrm{T4} \ll \mathrm{G1} \approx \mathrm{G2}.$$

That is, the main improvement comes from the GraphSAGE residual refinement stage, while the additional historical mean features provide only marginal extra benefit.

5.0.6 Experiment 3: Confidence-weighted GNN Ablation

This experiment evaluates whether the proposed confidence weighting strategy improves the GraphSAGE refinement stage. In Experiment 2, graph-based residual refinement was shown to provide a substantial improvement over the temporal-only model. However, the graph refinement model treats all training nodes equally unless additional weighting is introduced. In practice, not all taxi-zone predictions are equally reliable. Some zones have richer historical data, some predictions have lower MC Dropout uncertainty, and some zones are more consistent with their graph neighbours. Therefore, this experiment investigates whether these confidence signals can be used as sample weights during GNN residual learning.

Five GNN variants are compared:

- **GNN without confidence**: all nodes use equal sample weight, i.e. $w_i = 1$.
- **GNN + prior only**: only the historical data sufficiency score is used as the node weight.
- **GNN + stability only**: only the MC Dropout stability score is used as the node weight.
- **GNN + neighbourhood only**: only the neighbourhood consistency score is used as the node weight.
- **GNN + full confidence**: the final confidence score combines prior, stability, and neighbourhood consistency.

The unweighted model is used as the baseline for this ablation. The purpose is not to compare against the temporal-only model again, but to isolate the effect of confidence weighting inside the GNN refinement stage. The same rolling target hours, taxi-zone graph, train/validation/test node split, and residual prediction setting are used for all variants.

Table 5.5: Experiment 3 results: confidence-weighted GNN ablation.

Model	**MAE ↓**	**RMSE ↓**	**MSE ↓**	**Std. of Hourly MAE ↓**
GNN without confidence	16.6460	29.2035	852.8439	7.6376
GNN + prior only	16.7136	29.1922	852.1853	7.5131
GNN + stability only	**16.4645**	**28.9847**	**840.1146**	7.5475
GNN + neighbourhood only	16.5484	29.0987	846.7342	**7.4499**
GNN + full confidence	16.5793	29.1038	847.0323	7.4500

The results show that confidence weighting produces a moderate but meaningful effect on the GNN refinement stage. Compared with the unweighted GNN, the stability-only model achieves the best overall prediction accuracy. Its MAE decreases from 16.6460 to 16.4645, corresponding to an improvement of approximately 1.09%. RMSE also decreases from 29.2035 to 28.9847, and MSE decreases from 852.8439 to 840.1146. This indicates that the MC Dropout stability score is the most effective individual confidence component for reducing average prediction error.

This result is consistent with the purpose of MC Dropout uncertainty estimation. A zone prediction with lower MC Dropout variance is more stable under stochastic forward passes, suggesting that the base temporal model is more confident about that prediction. When this stability score is used as a GNN sample weight, the residual refinement model gives more importance to reliable nodes and reduces the influence of uncertain predictions. Therefore, the stability-only result suggests that uncertainty-aware weighting can improve the learning of residual corrections.

The prior-only setting does not improve MAE. Its MAE increases slightly from 16.6460 to 16.7136. This suggests that historical data sufficiency alone is not a sufficient indicator of prediction reliability. A zone with more historical records may still exhibit strong short-term volatility, and high-demand zones can still produce large forecasting errors. However, the prior-only model slightly improves RMSE, MSE, and the standard deviation of hourly MAE. This means that the prior score may help reduce some large-error fluctuations, but it is not strong enough to improve the average absolute error by itself.

The neighbourhood-only setting performs better than the unweighted GNN across all reported metrics. Its MAE decreases from 16.6460 to 16.5484, while the standard deviation of hourly MAE decreases from 7.6376 to 7.4499. This is the lowest hourly MAE standard deviation among all tested variants. The result suggests that neighbourhood consistency is especially useful for stabilising rolling predictions. If a zone prediction strongly disagrees with its graph neighbours, it may be less reliable as a training signal for residual correction. Down-weighting such nodes helps the GNN produce more consistent performance across different target hours.

The full confidence model combines prior, stability, and neighbourhood consistency. It improves all metrics compared with the unweighted GNN, reducing MAE from 16.6460 to 16.5793, RMSE from 29.2035 to 29.1038, and MSE from 852.8439 to 847.0323. It also reduces the standard deviation of hourly MAE from 7.6376 to 7.4500. However, the full confidence model does not achieve the best value on every metric. The stability-only model achieves the lowest MAE, RMSE, and MSE, while the neighbourhood-only model achieves the lowest hourly MAE standard deviation. Therefore, the full confidence strategy should be interpreted as a balanced weighting scheme rather than the strongest configuration for a single metric.

The hourly comparison further supports this interpretation. Across the 24 rolling target hours, the stability-only model achieves the lowest hourly MAE in 11 hours, which is more frequent than any other variant. The unweighted GNN and the neighbourhood-only model each achieve the best hourly MAE in 4 hours, the prior-only model in 3 hours, and the full confidence model in 2 hours. This shows that the stability score is the most consistently effective signal for improving hourly accuracy, although neighbourhood-based weighting remains important for reducing error variability.

Overall, Experiment 3 shows that the confidence module provides a robustness enhancement rather than a dramatic accuracy improvement. The large performance gain in the overall pipeline mainly comes from the graph-based residual refinement demonstrated in

Experiment 2. Confidence weighting further refines this process by controlling how much each node contributes to GNN training. Among the confidence components, MC Dropout stability is the most useful for reducing average error, while neighbourhood consistency is the most useful for improving rolling stability. The full confidence model provides a compromise between these effects.

The key conclusion from this experiment is therefore:

$$\text{Stability-only: best average accuracy,}$$

$$\text{Neighbourhood-only: best rolling stability,}$$

$$\text{Full confidence: balanced improvement over unweighted GNN.}$$

This indicates that confidence weighting is useful, but its role should be described as improving robustness and stability rather than as the primary source of the overall error reduction.

5.0.7 Experiment 4: Incremental Training in Rolling Forecasting

This experiment evaluates whether incremental training improves the performance and stability of the proposed rolling forecasting framework. In the previous experiments, graph-based residual refinement and confidence weighting were evaluated under a fixed rolling setup. However, in a practical forecasting scenario, the prediction target moves forward continuously and new observations become available over time. If the model only reuses an old checkpoint without updating, its parameters may become less adaptive to recent temporal changes. Incremental training is introduced to address this issue by updating the temporal model using newly available data during the rolling process.

The comparison in this experiment focuses on two training strategies:

- **Non-incremental training**: the temporal model is trained and then reused through stored checkpoints. When the rolling target moves forward, the model does not perform additional fine-tuning on newly available observations.

- **Incremental training**: the temporal model is updated during rolling prediction using newly available recent data. This allows the model to adapt to local temporal changes without retraining from scratch.

Both settings use the same multi-scale temporal architecture and the same GraphSAGE refinement stage. Therefore, the purpose of this experiment is not to prove the effectiveness of GNN refinement again, but to isolate the effect of incremental updating under the same rolling forecasting pipeline.

The experiment is conducted over 24 rolling target hours on `2021-03-05`, from `00:00` to `23:00`. For each target hour, the base temporal prediction and the GNN-refined prediction are evaluated. The comparison includes both average error and rolling stability, measured by the standard deviation of hourly MAE.

Table 5.6: Experiment 4 results: non-incremental training vs incremental training.

Model	**Training Mode**	**MAE ↓**	**RMSE ↓**	**MSE ↓**	**Std. of Hourly MAE ↓**
Base Temporal	Non-incremental	34.9008	54.2014	2937.7899	12.6363
Base Temporal	Incremental	26.3093	40.5043	1640.5947	10.2752
GNN-refined	Non-incremental	14.7466	30.3834	923.1511	7.7932
GNN-refined	Incremental	**13.0842**	**24.8755**	**618.7909**	**5.7611**

The results show that incremental training improves both the base temporal model and the final GNN-refined model. For the temporal-only prediction, MAE decreases from 34.9008 under non-incremental checkpoint reuse to 26.3093 under incremental training. This corresponds to a relative MAE reduction of approximately 24.62%. RMSE is also reduced from 54.2014 to 40.5043, giving a relative reduction of approximately 25.27%. This indicates that incremental updating allows the temporal model to adapt more effectively to the newly available demand patterns during the rolling process.

The improvement remains visible after GNN refinement. Under the non-incremental setting, the final GNN-refined MAE is 14.7466. With incremental training, the final GNN-refined MAE decreases to 13.0842, corresponding to an additional reduction of approximately 11.27%. RMSE decreases from 30.3834 to 24.8755, which is a relative reduction of approximately 18.13%. This result is important because it shows that the benefit of incremental training is not limited to the temporal stage. A more accurate and adaptive temporal prediction also provides a better prior for the GNN residual refinement stage.

This behaviour can be explained by the structure of the two-stage framework. The GNN does not predict demand from scratch; instead, it learns a residual correction on top of the temporal prediction. Therefore, the quality of the base prediction directly affects the difficulty of the residual learning task. When the temporal model is updated incrementally, the residuals become smaller and more stable, allowing the graph refinement stage to produce more accurate final predictions.

Incremental training also improves rolling stability. For the base temporal model, the standard deviation of hourly MAE decreases from 12.6363 to 10.2752, corresponding to an improvement of approximately 18.69%. For the GNN-refined model, the standard deviation of hourly MAE decreases from 7.7932 to 5.7611, corresponding to an improvement of approximately 26.08%. This suggests that incremental training not only lowers average error, but also reduces hour-to-hour fluctuation in rolling performance.

The stability improvement is particularly important in the context of operational demand forecasting. A model with low average error but unstable hourly behaviour may still be unreliable in deployment, because sudden error spikes can affect downstream decisions such as vehicle allocation or demand management. The reduction in hourly MAE variation shows that incremental updating makes the forecasting pipeline more consistent across the rolling horizon.

The comparison between the temporal-only and GNN-refined results also confirms that the two components are complementary. In both training modes, GNN refinement substantially reduces the error of the base temporal model. Under non-incremental training, MAE decreases from 34.9008 to 14.7466 after GNN refinement. Under incremental training, MAE decreases from 26.3093 to 13.0842 after GNN refinement. This indicates that graph-based residual correction remains effective regardless of the temporal training strategy, while incremental updating further strengthens the entire pipeline by improving the quality of the temporal prior.

Overall, Experiment 4 demonstrates that incremental training is beneficial for rolling taxi demand forecasting. Compared with non-incremental checkpoint reuse, incremental updating improves the base temporal prediction, improves the final GNN-refined prediction, and produces more stable hourly performance. The key conclusion is therefore:

$$\text{Incremental + GNN} \quad > \quad \text{Non-incremental + GNN} \quad > \quad \text{Temporal-only baselines.}$$

This supports the use of incremental updating as a practical extension of the proposed framework. While graph refinement provides the major spatial correction capability, incremental training improves temporal adaptation and reduces performance degradation during rolling forecasting.

5.0.8 Summary of Experimental Findings

This section summarises the main findings from the experimental study. The experiments were designed to evaluate the proposed framework progressively, starting from temporal baselines, then introducing graph-based residual refinement, confidence weighting, and finally incremental training under a rolling forecasting setting. The overall results show that the proposed framework benefits from each major design component, but the magnitude and role of each component are different.

First, the temporal baseline comparison establishes the importance of using a stronger temporal backbone before applying graph refinement. The proposed multi-scale temporal model is designed to capture short-term, daily, weekly, and longer-term temporal patterns through separate input branches. This makes it more suitable for taxi demand forecasting than a single-scale temporal model, because urban mobility demand is affected by multiple periodic structures at the same time. The multi-scale temporal model is therefore used as the base predictor for the later graph refinement experiments.

Second, Experiment 2 shows that graph-based residual refinement is the main source of accuracy improvement in the proposed pipeline. The temporal-only multi-scale model, denoted as T4, achieves an MAE of 33.4804 and an RMSE of 51.9800. After adding GraphSAGE residual refinement, G1 reduces MAE to 16.2106 and RMSE to 30.5118. This corresponds to a 51.58% reduction in MAE compared with the temporal-only model. This large improvement confirms that spatial structure is highly useful for correcting zone-level temporal forecasting errors.

The comparison between G1 and G2 further clarifies the role of historical mean features. G2 adds `mean_24h`, `mean_168h`, and `mean_720h` to the GNN input. However, its MAE is 16.2040, which is only slightly lower than the G1 MAE of 16.2106. Similarly, RMSE decreases only slightly from 30.5118 to 30.3489. Therefore, the results suggest that the main improvement comes from graph-based residual correction rather than from the additional historical mean features. A reasonable interpretation is that the multi-scale temporal model has already encoded much of the temporal context, so the simple historical averages provide only limited additional information to the GNN.

Third, Experiment 3 shows that confidence weighting improves the robustness of the GNN refinement stage, but it should not be interpreted as the main source of the overall error reduction. Compared with the unweighted GNN, the stability-only confidence setting achieves the best average accuracy. It reduces MAE from 16.6460 to 16.4645, RMSE from 29.2035 to 28.9847, and MSE from 852.8439 to 840.1146. This indicates that MC Dropout stability is the most informative individual confidence signal for reducing average prediction error.

The neighbourhood-only confidence setting provides the strongest rolling stability. It reduces the standard deviation of hourly MAE from 7.6376 to 7.4499, which is the lowest value among the tested confidence variants. This suggests that neighbourhood consistency is particularly useful for reducing hour-to-hour variation in graph refinement performance. The full confidence model improves all metrics compared with the unweighted GNN, but it is not the best model for every individual metric. Its MAE is 16.5793 and its standard deviation of hourly MAE is 7.4500. Therefore, full confidence should be interpreted as a balanced weighting strategy that combines accuracy and stability, rather than as the strongest configuration for a single metric.

Fourth, Experiment 4 demonstrates that incremental training improves both prediction accuracy and rolling stability. In the non-incremental setting, the base temporal model obtains an MAE of 34.9008 and an RMSE of 54.2014. With incremental training, the base temporal MAE decreases to 26.3093 and RMSE decreases to 40.5043. This shows that incremental updating allows the temporal model to adapt better as new observations become available during the rolling process.

The benefit of incremental training is also preserved after graph refinement. The non-incremental GNN-refined model achieves an MAE of 14.7466 and an RMSE of 30.3834. With incremental training, the GNN-refined MAE decreases to 13.0842 and RMSE decreases to 24.8755. This confirms that improving the temporal base prediction also benefits the graph refinement stage, because the GNN learns residual corrections on top of the temporal prior. When the base prediction is more accurate and adaptive, the residual learning problem becomes easier and more stable.

Incremental training also improves the stability of rolling prediction. For the base temporal model, the standard deviation of hourly MAE decreases from 12.6363 to 10.2752. For the GNN-refined model, it decreases from 7.7932 to 5.7611. This shows that incremental training does not only reduce average error, but also makes the forecasting pipeline more consistent across rolling target hours.

The overall experimental findings can be summarised as follows:

Table 5.7: Summary of the main experimental findings.

Experiment	**Main Question**	**Finding**
Experiment 1	Does the multi-scale temporal model provide a strong temporal baseline?	The multi-scale temporal model is used as the main temporal backbone because it captures multiple temporal patterns, including short-term, daily, weekly, and longer-term demand structures.
Experiment 2	Does graph refinement improve temporal prediction?	GraphSAGE residual refinement substantially improves the temporal-only model. MAE decreases from 33.4804 to 16.2106 in G1, showing that graph-based correction is the main source of accuracy improvement.
Experiment 3	Does confidence weighting improve GNN refinement?	Confidence weighting provides moderate robustness improvement. Stability-only confidence gives the best average accuracy, while neighbourhood-only confidence gives the best rolling stability. Full confidence provides a balanced improvement.
Experiment 4	Does incremental training improve rolling forecasting?	Incremental training improves both the base temporal model and the GNN-refined model. Final GNN MAE decreases from 14.7466 to 13.0842, and hourly MAE standard deviation decreases from 7.7932 to 5.7611.

Taken together, the experiments support the proposed two-stage forecasting design. The temporal model provides the initial zone-level prediction, the GraphSAGE module performs spatial residual correction, the confidence mechanism improves robustness, and incremental training improves adaptation in the rolling setting. The most significant accuracy gain comes from graph-based residual refinement, while confidence weighting and incremental training mainly improve stability and practical reliability.

The final conclusion is that the proposed framework should not be viewed as a single monolithic model, but as a structured forecasting pipeline:

$$\text{Multi-scale Temporal Prediction} \rightarrow \text{GraphSAGE Residual Refinement} \rightarrow \text{Confidence-weighted Robust}$$

Each stage contributes differently. Graph refinement provides the largest reduction in prediction error, confidence weighting improves the reliability of graph learning, and incremental training makes the system more adaptive and stable over time. This supports the overall claim that combining temporal modelling, graph-based correction, uncertainty-aware weighting, and incremental updating is effective for rolling taxi demand forecasting.

Chapter 6

Discussion

6.0.1 Overview of the Main Findings

The experimental results show that the proposed framework is best understood as a structured forecasting pipeline rather than as a single isolated model. The final system combines four main ideas: multi-scale temporal prediction, graph-based residual refinement, confidence-weighted graph training, and incremental adaptation in a rolling forecasting setting. The results indicate that these components do not contribute equally. The largest improvement comes from graph-based residual refinement, while confidence weighting and incremental training contribute more to robustness, stability, and practical adaptability.

The first important observation is that the temporal-only model is not sufficient for accurate taxi-zone demand forecasting. The multi-scale temporal model captures several levels of temporal structure, including short-term changes, daily periodicity, weekly behaviour, and longer-term historical trends. This makes it a stronger baseline than a simple single-branch recurrent model. However, the temporal-only result in Experiment 2 still has a relatively high error, with T4 producing an MAE of 33.4804 and an RMSE of 51.9800. This indicates that even a multi-scale sequence model can leave large zone-level residual errors. In the context of taxi demand forecasting, this is expected. Demand is not generated independently within each zone. It is affected by movement between zones, neighbouring activity, commuting patterns, and the structure of the city. A model that only uses a zone's own historical sequence can capture temporal recurrence, but it may miss spatial dependency.

The second major observation is that the graph refinement stage produces a substantial reduction in prediction error. In Experiment 2, adding GraphSAGE residual refinement reduces MAE from 33.4804 to 16.2106. This is a reduction of more than half compared with the temporal-only prediction. The reduction is also visible in RMSE and MSE, suggesting that graph refinement does not merely correct small errors but also reduces larger residual deviations. This is one of the most important findings of the project. It supports the central methodological assumption that spatial structure should be used after temporal prediction to correct systematic zone-level errors. The graph module is not designed to predict demand from scratch. Instead, it receives the temporal prediction as a prior and learns the residual between the true demand and the temporal output. This makes the role of the GNN more focused and interpretable: it acts as a spatial correction mechanism.

The third observation is that additional historical mean features provide only marginal improvement when added to the GNN. G2, which adds historical features such as `mean_24h`, `mean_168h`, and `mean_720h`, achieves an MAE of 16.2040, which is almost identical to the G1 MAE of 16.2106. The difference is too small to treat the historical means as a major source of improvement. This is an important result because it prevents overstatement of the contribution of handcrafted features. The historical means are useful auxiliary signals, but they are not the main reason why graph refinement works. A likely explanation is that the base multi-scale temporal model has already encoded much of the information represented by these mean features. The additional features therefore provide redundant or weakly informative context to the GNN. This result suggests that the graph structure and residual learning formulation are more important than simply adding more node features.

The fourth observation is that confidence weighting has a moderate but meaningful effect. Experiment 3 shows that the stability-only confidence score achieves the best average accuracy among the confidence ablation settings, reducing MAE from 16.6460 for the unweighted GNN to 16.4645. This indicates that MC Dropout stability is the most useful single confidence component for reducing average prediction error. The neighbourhood-only score, on the other hand, achieves the lowest standard deviation of hourly MAE, reducing the value from 7.6376 to 7.4499. This suggests that neighbourhood consistency is especially useful for stabilising rolling predictions. The full confidence model improves over the unweighted GNN in all metrics, but it is not the best configuration for every individual metric. This means that the full confidence score should be interpreted as a balanced robustness mechanism rather than as a universally optimal weighting scheme.

The fifth observation is that incremental training improves both accuracy and stability under rolling forecasting. In Experiment 4, incremental training reduces the base temporal MAE from 34.9008 to 26.3093 and the final GNN-refined MAE from 14.7466 to 13.0842. It also reduces the standard deviation of hourly MAE for the GNN-refined model from 7.7932 to 5.7611. These results support the argument that rolling forecasting benefits from updating the temporal model as new observations become available. Incremental updating improves the temporal prior, and because the GNN learns residual correction on top of that prior, the final graph-refined prediction also improves. This confirms that the two stages are connected: better temporal prediction leads to a better residual learning problem for the GNN.

Overall, the experiments suggest a clear hierarchy of contributions. Graph-based residual refinement is the primary source of accuracy improvement. Confidence weighting improves the reliability of graph learning by controlling the influence of uncertain nodes. Incremental training improves adaptation to new data and stabilises the rolling forecasting process. The multi-scale temporal model provides the necessary base prediction, but its full value is realised when it is combined with graph refinement and rolling adaptation.

6.0.2 Interpretation of the Multi-scale Temporal Model

The multi-scale temporal model is motivated by the observation that taxi demand has multiple temporal patterns. Hourly demand may be affected by immediate local changes, daily routines, weekly cycles, and longer-term demand levels. A single temporal window may not be able to represent all of these structures. For example, a short recurrent window can respond quickly to recent changes, but it may not capture weekly periodicity. A longer window can encode slower patterns, but it may smooth out short-term fluctuations. The proposed temporal model therefore separates the input into multiple temporal scales and combines the resulting representations before making the final prediction.

This design is conceptually appropriate for urban demand forecasting. Taxi demand is shaped by human routines. Morning and evening commuting, weekday versus weekend behaviour, nightlife patterns, transport hub activity, and business district activity all contribute to repeated but non-identical temporal structures. A multi-scale model can encode these patterns more explicitly than a single fixed-window model. The temporal stage therefore provides an informative prior for each zone before spatial information is introduced.

However, the experimental results also reveal the limitation of temporal modelling alone. In Experiment 2, the multi-scale temporal model still produces much larger error than the graph-refined models. This does not necessarily mean that the temporal model is weak. Instead, it indicates that the forecasting problem itself is not purely temporal. A zone's future demand depends not only on its own past but also on the state of related zones. For example, demand in a central business district may be connected to surrounding residential areas during commuting hours. Demand around transport hubs may be related to nearby hotel, office, or entertainment zones. These relationships cannot be fully captured if each zone is modelled as an independent time series.

The multi-scale temporal model should therefore be interpreted as a strong base predictor rather than as a complete solution. Its role is to provide a first estimate of demand based on each zone's own history. This is important because the GNN refinement stage relies on the temporal prediction as its base prior. If the temporal prediction is extremely poor, the residuals become large and noisy, making graph correction more difficult. A reasonable temporal prior reduces the complexity of the residual learning task. In this sense, the temporal model and the GNN are complementary. The temporal stage captures within-zone temporal regularities, while the graph stage captures cross-zone correction patterns.

The limited benefit of additional historical mean features in G2 further supports this interpretation. The historical means are simple summaries of past demand over 24-hour, 168-hour, and 720-hour windows. They provide information about the average demand level over different periods, but they do not capture the detailed sequence patterns learned by the multi-scale temporal model. Since the temporal model already uses multi-scale historical windows, these handcrafted averages are likely to overlap with information already encoded in the base prediction. Therefore, once the GNN has access to the temporal prediction, adding historical means gives only marginal benefit.

This result has methodological implications. It suggests that in a two-stage forecasting framework, the most valuable node feature for graph refinement may be the output of a strong temporal predictor, rather than a large set of manually engineered historical statistics. The temporal prediction is a compact representation of historical information, learned through a neural model and optimised for the forecasting target. By contrast, simple historical averages are generic and may not align with the residual correction task. This does not mean that additional node features are useless. Features such as weather, holidays, land use, road network centrality, transport hub indicators, or event data could provide genuinely new information. However, the current historical mean features are too similar to the information already contained in the multi-scale temporal prediction.

Another important point is that the temporal model is trained at the zone level. This design has advantages and disadvantages. A per-zone temporal model can specialise to the local demand behaviour of each taxi zone. It can capture zone-specific demand levels and local patterns without forcing all zones to share the same temporal dynamics. However, per-zone modelling can also be data inefficient, especially for sparse zones. Low-demand zones may not contain enough informative samples for stable training. This partly explains why graph refinement and confidence weighting are useful. The graph stage allows information from related zones to influence correction, and the confidence mechanism reduces the impact of uncertain zones.

The incremental training results also highlight the importance of temporal adaptation. When the model reuses old checkpoints without updating, its temporal representation can become stale as the rolling target moves forward. Incremental training improves the base MAE substantially, reducing it from 34.9008 to 26.3093 in Experiment 4. This shows that the temporal model benefits from incorporating newly available observations. In dynamic urban systems, demand patterns can shift due to weather, events, road conditions, day-specific behaviour, or gradual distribution drift. A static checkpoint may not respond to these changes, whereas incremental updating provides a lightweight mechanism for adaptation.

Therefore, the discussion of the temporal model should not focus only on architecture. It should also consider how the temporal model is maintained over time. In static evaluation, a multi-scale architecture may be sufficient. In rolling deployment, however, the ability to update the model becomes equally important. The proposed framework addresses this by separating the temporal predictor from the graph correction stage, making it possible to update the temporal model incrementally while retaining the same overall pipeline.

6.0.3 Interpretation of Graph-based Residual Refinement

The strongest experimental finding is the effectiveness of graph-based residual refinement. The comparison between T4 and G1 shows that adding GraphSAGE leads to a large reduction in error. This indicates that spatial structure contains important information that is not fully captured by the temporal model. In taxi demand forecasting, the spatial dimension is not merely geographic. Zones are related through passenger flows, functional similarity, and urban structure. An origin–destination flow graph can capture these relationships more meaningfully than a simple distance-based adjacency graph, because it reflects how people actually move between zones.

The residual formulation is central to the success of the graph stage. Instead of using the GNN to directly predict demand, the framework uses the GNN to predict the residual between the true value and the temporal prediction. This has several advantages. First, it reduces the learning burden on the graph model. The GNN does not need to learn the entire temporal demand pattern; it only needs to learn how the temporal prediction should be corrected based on graph context. Second, the residual target is more aligned with the role of spatial information. Spatial relationships may not determine the full demand value, but they can explain why a zone is under-predicted or over-predicted relative to its own temporal history. Third, the residual design makes the two-stage system more interpretable. The final prediction can be understood as a temporal prior plus a graph-based correction.

The large improvement from T4 to G1 suggests that the temporal model makes systematic errors that are spatially structured. If the residual errors were random and independent across zones, a graph model would not be able to correct them effectively. The success of GNN refinement therefore implies that residuals contain spatial patterns. For example, when the temporal model underestimates demand in one active region, neighbouring or flow-connected zones may provide signals that help correct that underestimation. Similarly, if a zone's prediction is inconsistent with the broader graph context, the GNN can learn to adjust it.

This also explains why graph refinement improves stability. Spatial correction can reduce local noise by aggregating information from related nodes. If a temporal prediction is unusually high or low compared with its graph neighbourhood, the GNN can learn to moderate the prediction. This is especially valuable in sparse or volatile zones where the temporal model may be less reliable. In the results, GNN refinement reduces not only the average error but also the standard deviation of hourly MAE. This suggests that graph correction provides a smoothing and regularising effect across the rolling horizon.

However, the graph refinement results should be interpreted carefully. The current graph refinement stage learns residuals within each rolling target hour using available node labels and evaluates on held-out nodes. This is a valid way to evaluate spatial residual generalisation across nodes, but it is not the same as a fully future-facing deployment scenario in which the true demand at the target hour is unavailable for all zones. In real-time forecasting, the target-hour labels would only become available after the hour has passed. Therefore, for deployment, the GNN residual model would need to be trained on previous hours and then applied to future hours, or updated online after observations arrive. This limitation does not invalidate the experimental finding, but it narrows its interpretation. The current experiment demonstrates that graph structure can explain and correct residuals across taxi zones. Further evaluation is needed to confirm how well the learned residual correction generalises across future target hours.

Another limitation concerns the use of edge weights. The graph is constructed from an OD flow matrix, and the implementation can store edge-related information. However, the current GraphSAGE layer primarily uses the graph connectivity structure rather than explicitly using weighted message passing. This means that the model treats edges largely as connections, while the magnitude of the flow relationship may not be fully exploited in the convolution operation. As a result, the graph may not use all available information from the OD matrix. A future weighted GNN implementation could incorporate edge weights directly into message aggregation, allowing stronger flow relationships to contribute more than weaker ones.

The choice of GraphSAGE is nevertheless reasonable. GraphSAGE is designed to aggregate neighbourhood information and can operate on sparse graph structures. Its residual correction role does not require a highly complex graph architecture. A two-layer GraphSAGE model with dropout and nonlinear activation provides enough capacity to learn local correction patterns while remaining computationally manageable. More complex models such as GAT, GCN with weighted adjacency, temporal graph networks, or diffusion convolutional models could be compared in future work. However, the strong improvement observed with GraphSAGE already supports the value of graph-based refinement.

It is also important to consider why G1 and G2 are nearly identical. G2 adds historical mean features to the GNN, but the improvement over G1 is marginal. This suggests that the graph model obtains most of its correction power from the base prediction and graph structure. The base prediction already summarises temporal history, and the graph structure provides spatial context. The historical means add relatively little new information. This finding is useful because it simplifies the interpretation of the model. The GNN does not depend heavily on manually engineered demand summaries. Its effectiveness is primarily due to residual learning over a meaningful zone graph.

From a thesis perspective, this strengthens the contribution. If the improvement came mainly from simple historical means, the novelty of graph refinement would be weaker. Instead, the results show that the graph correction mechanism itself is responsible for the largest gain. The relation

$$\mathrm{T4} \ll \mathrm{G1} \approx \mathrm{G2}$$

is therefore a favourable outcome. It indicates that spatial residual learning is effective, while handcrafted features are auxiliary.

6.0.4 Interpretation of Confidence Weighting

The confidence module was introduced to address a practical issue in graph residual learning: not every node should contribute equally to the training loss. Some zone predictions are more reliable than others. A high-volume zone may have rich history, but it may also experience large fluctuations. A sparse zone may have little data and high uncertainty. A prediction that is inconsistent with its neighbours may represent either a genuine local event or an unreliable outlier. The confidence module attempts to capture these aspects through three signals: prior score, stability score, and neighbourhood consistency score.

The results of Experiment 3 show that confidence weighting has a moderate effect. It does not produce the dramatic improvement seen in Experiment 2, but it improves robustness and provides insight into the reliability of different signals. The stability-only score achieves the lowest MAE, RMSE, and MSE. This indicates that MC Dropout stability is the most informative individual confidence component for average predictive accuracy. This makes sense because MC Dropout directly measures the sensitivity of the model prediction to stochastic dropout perturbations. If repeated stochastic forward passes produce similar outputs, the model is relatively stable for that input. If the outputs vary widely, the prediction is less reliable. Using this stability information as a sample weight helps the GNN focus on residual patterns associated with more reliable base predictions.

The neighbourhood-only score achieves the lowest standard deviation of hourly MAE. This suggests that neighbourhood consistency is particularly useful for stabilising performance across rolling hours. A prediction that strongly disagrees with neighbouring or flow-connected zones may introduce noise into graph residual learning. Down-weighting such nodes can reduce the effect of spatially inconsistent signals. This does not necessarily minimise average MAE in every hour, but it reduces hour-to-hour variation. In operational forecasting, this is valuable because stability is often as important as average accuracy. A model with slightly lower average error but frequent error spikes may be less reliable than a model with consistent performance.

The prior-only score performs less well in terms of MAE. It slightly worsens MAE compared with the unweighted baseline, although it improves some stability-related metrics. This result is important because it shows that historical data sufficiency alone is not a reliable proxy for prediction reliability. A zone with many historical records is not necessarily easy to predict. High-demand zones may have complex patterns, event-driven fluctuations, or strong peak-hour variation. Conversely, a lower-demand zone may be stable and easier to predict. The prior score therefore captures data availability but not necessarily model confidence or current predictability. This explains why it is weaker than the stability and neighbourhood components.

The full confidence score improves all metrics compared with the unweighted baseline, but it is not the best on every metric. This is not a failure of the confidence design. Instead, it shows that different confidence components optimise different aspects of performance. Stability is best for average accuracy, neighbourhood consistency is best for rolling stability, and the full score balances these objectives. The fixed weighting scheme, using prior, stability, and neighbourhood components together, may dilute the strongest component for a given metric. For example, if stability is the most predictive signal for MAE, combining it with prior and neighbourhood scores may reduce its direct effect. However, the combined score provides a more general and interpretable reliability estimate.

This result suggests a future direction: confidence fusion could be learned rather than fixed. The current method uses manually selected weights for the components. A learned gating mechanism, small neural network, or validation-optimised weighting scheme could adaptively determine how much each component should contribute under different conditions. For example, during stable periods, prior and neighbourhood consistency might be sufficient. During volatile periods, MC stability might be more important. During sparse-zone prediction, prior score might become more useful. A dynamic confidence mechanism could therefore outperform a fixed weighted formula.

Another important implication is that confidence should be evaluated not only by accuracy but also by calibration. The current experiments use confidence as a sample weight and evaluate the resulting prediction error. However, a confidence score can also be judged by whether it correlates with actual error. For example, high-confidence predictions should generally have lower absolute error than low-confidence predictions. Future analysis could group predictions into confidence bins and measure the error distribution in each bin. This would show whether the confidence score is well calibrated. If confidence is well calibrated, it could be useful not only for training but also for decision-making, such as flagging uncertain zones for manual review or fallback strategies.

The confidence results also indicate that uncertainty estimation and graph learning are complementary. MC Dropout is computed in the temporal model, while the confidence score is used in the graph model. This creates a connection between temporal uncertainty and spatial correction. The temporal stage not only provides a prediction but also indicates how reliable that prediction is. The graph stage then uses this reliability information to weight residual learning. This is a useful design because it allows uncertainty to influence downstream correction rather than remaining only as a diagnostic output.

However, the magnitude of the confidence improvement should not be overstated. The reduction in MAE from the unweighted GNN to the best confidence setting is around one percent. This means that confidence weighting is not the main source of the framework's predictive power. The main accuracy gain comes from adding graph residual refinement. Confidence weighting should therefore be presented as a robustness and stability enhancement. This is still valuable, especially in rolling forecasting, but it should be distinguished from the primary contribution of graph refinement.

6.0.5 Interpretation of Incremental Training

Incremental training addresses the dynamic nature of rolling forecasting. In a rolling setup, the forecast origin moves forward step by step, and new data become available over time. A model that is trained once and reused may gradually become less aligned with the most recent demand distribution. Incremental training aims to reduce this problem by updating the model using newly available observations rather than relying only on an older checkpoint.

The results of Experiment 4 strongly support the value of incremental training. The base temporal MAE decreases from 34.9008 in the non-incremental setting to 26.3093 in the incremental setting. This is a large improvement. It shows that the temporal model benefits from recent data and that checkpoint reuse alone is not sufficient for the rolling demand forecasting task. The improvement is also reflected in RMSE and MSE, indicating that incremental updating reduces both average errors and larger deviations.

The benefit continues after GNN refinement. The non-incremental GNN-refined MAE is 14.7466, while the incremental GNN-refined MAE is 13.0842. This shows that improving the temporal prior helps the graph stage. Since the GNN predicts residuals on top of the temporal prediction, a better base prediction makes the residuals smaller and more structured. If the temporal model is stale, the residuals may include temporal drift that the graph model has to correct indirectly. If the temporal model is updated, the GNN can focus more on spatial correction rather than compensating for temporal mismatch.

The stability results are especially important. The standard deviation of hourly MAE for the GNN-refined model decreases from 7.7932 to 5.7611 with incremental training. This indicates that incremental updating reduces hour-to-hour fluctuation in model performance. In practical forecasting, this is highly relevant. Transport demand systems require reliable performance across time, not only good average accuracy. A model that performs well on average but fails sharply during certain hours may be problematic for downstream applications such as dispatch planning, resource allocation, and congestion management. Incremental training improves the consistency of performance across the rolling horizon.

The interpretation of incremental training should also consider computational cost. Full retraining at every rolling step would be expensive, especially if each taxi zone has its own model. Persistent checkpoint reuse is efficient but less adaptive. Incremental training sits between these extremes. It allows the model to update using recent data without retraining entirely from scratch. This makes it attractive for deployment scenarios where new data arrive continuously. However, the exact computational benefit depends on the implementation details, including the number of incremental epochs, the size of the update window, and whether all layers or only selected layers are updated. A complete efficiency analysis should measure training time, memory use, and prediction latency.

Incremental training also introduces risks. If the update window is too short or the learning rate is too high, the model may overfit to recent noise. This could reduce generalisation and cause unstable behaviour. If the update is too weak, the model may not adapt enough. Therefore, incremental training requires careful tuning. Possible strategies include using a small learning rate, updating only the final layers, using replay data from earlier periods, applying early stopping, or regularising the updated parameters toward the previous checkpoint. These strategies could make incremental adaptation more stable.

Another risk is catastrophic forgetting. If the model updates only on recent data, it may lose knowledge of older patterns, such as weekly cycles or recurring demand structures. This is particularly relevant for taxi demand because older patterns may still be useful. A model should adapt to recent changes without forgetting stable long-term regularities. The multi-scale architecture helps because it explicitly includes longer temporal windows, but the training procedure also matters. Future incremental versions could include replay buffers or regularisation methods to preserve long-term behaviour.

Despite these risks, the experimental evidence indicates that incremental training is beneficial in the current setting. The improvement in both mean error and hourly stability suggests that the model adapts in a useful direction rather than overfitting. This supports the thesis claim that rolling demand forecasting should not be treated as a static prediction problem. The model should be updated as the data distribution evolves.

6.0.6 Practical Implications

The proposed framework has several practical implications for urban mobility forecasting. First, the results show that accurate taxi demand prediction should combine temporal and spatial information. A temporal model alone can capture historical patterns, but spatial correction can substantially improve the final prediction. This is relevant for operational systems where demand in one zone is related to nearby or flow-connected zones. For example, a dispatch system may need to anticipate demand not only in isolated zones but across connected regions of the city. A graph-based refinement stage provides a mechanism for incorporating this dependency.

Second, the two-stage design is practical because it separates temporal prediction from spatial correction. This modularity has engineering advantages. The temporal model can be trained, updated, and diagnosed separately from the GNN. The graph refinement module can be modified without redesigning the entire temporal backbone. For example, GraphSAGE could be replaced by GAT, GCN, or a weighted message passing model. Additional node features could be introduced later. This modular structure makes the system easier to extend than a fully entangled end-to-end architecture.

Third, confidence scores can support decision-making beyond model training. Even if confidence weighting only moderately improves accuracy, the confidence values themselves can be useful. A low-confidence prediction could signal that a zone requires caution. In a real system, low-confidence forecasts could trigger fallback models, wider prediction intervals, human review, or conservative resource allocation. High-confidence predictions could be used more aggressively in optimisation decisions. Therefore, confidence estimation has value even when its direct MAE reduction is modest.

Fourth, incremental training makes the system more suitable for deployment. Urban mobility demand changes over time. A model trained once may become outdated, especially under unusual conditions. Incremental updating allows the model to respond to recent observations. This is useful for daily operation, where new trip records continuously become available. The experimental results suggest that incremental training improves both the base model and final graph-refined prediction, making it a promising practical feature.

However, deployment would require additional safeguards. The current experiments are conducted in an offline setting. A real-time system would need automated data ingestion, missing-data handling, monitoring, retraining triggers, and drift detection. It would also need to ensure that future information is not accidentally used during prediction. The graph refinement stage would need to be trained using only information available before the prediction target if used for real-time forecasting. These engineering considerations are outside the current experimental scope but are important for practical application.

From a wider system perspective, the proposed pipeline could be used as an intermediate forecasting layer within a mobility management platform. The temporal model could produce zone-level predictions at regular intervals, the graph refinement stage could correct spatially inconsistent outputs, and the confidence module could provide uncertainty-aware reliability indicators. The final output would not only be a predicted demand value, but also a confidence-informed forecast that can support downstream planning. This is more useful than a point prediction alone, because operational decision-making often requires understanding both the expected demand and the reliability of that expectation.

The framework also has implications for model maintenance. In many machine learning systems, retraining is treated as a separate offline process. However, the rolling forecasting results show that updating strategy is part of model performance. A model that performs well immediately after training may degrade as the data distribution changes. Incremental training provides a way to maintain model relevance without full retraining. Therefore, for practical urban forecasting systems, model maintenance should be designed as part of the forecasting pipeline rather than as an afterthought.

6.0.7 Limitations

Several limitations should be acknowledged. The first limitation is the scale of the rolling evaluation. The current experiments focus on a 24-hour rolling period. This is useful for demonstrating the behaviour of the framework over a complete day, but it is not sufficient to fully establish long-term generalisation. Taxi demand can vary across weeks, seasons, holidays, weather conditions, and special events. A stronger evaluation would include multiple rolling windows across different dates and demand regimes. This would show whether the observed improvements remain consistent under broader conditions.

The second limitation concerns the graph refinement evaluation protocol. The current GNN residual model uses node-level train, validation, and test splits within each target hour. This evaluates how well the graph model generalises from labelled nodes to held-out nodes in the same hour. However, for real-time future forecasting, the true labels of the target hour would not be available. Therefore, future work should evaluate a stricter temporal protocol in which the GNN is trained on previous target hours and tested on future target hours. This would provide stronger evidence of deployment-ready predictive generalisation.

The third limitation is the graph construction method. The OD flow matrix provides a meaningful basis for connecting taxi zones, but the current graph may not fully capture all relevant spatial relationships. Some zones may be geographically close but have low direct flow. Others may be functionally related through public transport, tourism, or commuting patterns not fully represented in the OD matrix. The graph is also likely static, while real mobility relationships can change over time. A dynamic graph or multiple graph views could better represent changing urban structure.

The fourth limitation is that edge weights are not fully exploited in the current GraphSAGE aggregation. The OD matrix contains weighted information, but the model primarily uses graph connectivity. This may lose information about the strength of interzone relationships. Future models could incorporate edge weights directly into message passing, use attention mechanisms, or learn adaptive adjacency matrices.

The fifth limitation is the fixed confidence fusion rule. The full confidence score combines prior, stability, and neighbourhood components using manually selected weights.

Experiment 3 shows that different components are useful for different purposes. Stability is best for average accuracy, while neighbourhood consistency is best for rolling stability. A fixed fusion formula may not be optimal across all conditions. A learned or adaptive confidence fusion mechanism could provide better performance.

The sixth limitation is the absence of external contextual variables. Taxi demand is affected by weather, holidays, public events, road closures, transit disruptions, and socioeconomic features. The current framework mainly uses historical taxi demand and graph structure. Adding external variables could improve forecasting, especially during abnormal periods. However, external features also introduce data availability and alignment challenges.

The seventh limitation concerns computational scalability. Per-zone temporal modelling and rolling updates can be computationally expensive, especially if applied to larger cities or finer spatial grids. Although incremental training reduces the need for full retraining, the system still requires careful optimisation for deployment. Future work should measure runtime, GPU memory use, and update latency.

The eighth limitation is that the experiments mainly evaluate point prediction accuracy. MAE, RMSE, MSE, and hourly MAE standard deviation are useful metrics, but they do not fully describe uncertainty quality. Since the framework includes MC Dropout and confidence scoring, it would be valuable to evaluate uncertainty calibration more directly. For example, confidence bins could be compared against empirical error, or prediction intervals could be evaluated using coverage and sharpness. Without such evaluation, the confidence score should be interpreted primarily as a training weight rather than as a fully calibrated probabilistic estimate.

The ninth limitation is that the current approach uses a two-stage pipeline rather than end-to-end joint training. This improves modularity and interpretability, but it may prevent the temporal and graph stages from co-adapting optimally. An end-to-end spatiotemporal model could potentially learn temporal and spatial dependencies jointly. However, such models are also more complex and may be harder to debug, especially under rolling and incremental settings. The two-stage approach is therefore a pragmatic design choice, but it is not the only possible architecture.

6.0.8 Future Work

Future work should first extend the evaluation to longer and more diverse rolling periods. Testing the model across multiple days, weeks, and months would provide stronger evidence of generalisation. It would also allow analysis of weekday versus weekend behaviour, peak versus off-peak hours, and normal versus abnormal demand conditions. A broader evaluation would make it possible to determine whether graph refinement and incremental training remain consistently effective under different traffic regimes.

Second, the GNN evaluation protocol should be strengthened. A future version should train the graph residual model on previous hours and evaluate on future hours. This would better match real forecasting deployment. The GNN could be trained as a reusable residual correction model rather than retrained independently for each target hour. This would also allow a clearer assessment of whether the graph module learns persistent spatial correction patterns or only target-hour-specific residual interpolation.

Third, weighted and dynamic graph models should be explored. The OD flow matrix contains edge strength information that could be directly used in message passing. Graph attention networks could learn which neighbours are most relevant for correction. Dynamic graphs could update edges based on recent flows, allowing the spatial structure to change over time. This is particularly relevant for urban mobility, where the relationship between zones may differ between morning commute, evening commute, night-time activity, and weekend travel.

Fourth, the confidence mechanism could be made learnable. Instead of fixed weights for prior, stability, and neighbourhood consistency, a small model could learn how to combine these signals. The confidence score could also be calibrated against observed prediction errors. This would allow confidence to function both as a training weight and as an interpretable reliability estimate. A learned confidence module could also adapt to different zone types, demand levels, or time periods.

Fifth, incremental learning should be further developed. Future work could compare different update windows, learning rates, replay strategies, and layer-freezing policies. It could also investigate how to prevent catastrophic forgetting while still adapting to recent demand. A formal online learning framework could make the incremental component more rigorous. For example, the model could use a replay buffer containing both recent samples and representative older samples, so that it adapts to new patterns while preserving long-term knowledge.

Sixth, external data should be incorporated. Weather, holidays, events, points of interest, public transport information, and road network features could provide additional explanatory power. These variables may be especially useful for high-error periods or zones where historical taxi demand alone is insufficient. For instance, demand around airports, stadiums, or tourist areas may be affected by external events that are not predictable from historical taxi counts alone.

Seventh, future work could explore probabilistic forecasting. The current model outputs point predictions and MC Dropout uncertainty estimates, but it does not fully model the predictive distribution. Quantile regression, deep ensembles, Bayesian neural networks, or conformal prediction could be used to produce calibrated prediction intervals. This would be useful for decision-making because transport planners often need to understand the range of possible demand outcomes, not only the expected value.

Finally, future work could compare the proposed two-stage framework with fully integrated spatiotemporal neural networks. Models such as diffusion convolutional recurrent networks, temporal graph networks, or attention-based spatiotemporal transformers could be used as stronger baselines. Such comparisons would clarify whether the modular two-stage design provides competitive accuracy while retaining better interpretability and easier incremental updating.

6.0.9 Concluding Discussion

The discussion shows that the proposed framework is effective because it combines complementary modelling ideas. The temporal model captures within-zone demand history. The graph refinement stage uses spatial relationships to correct residual errors. The confidence module controls the influence of uncertain nodes. Incremental training adapts the model as new data become available. The experimental results support this design, while also showing that the components have different roles.

The most important accuracy improvement comes from GraphSAGE residual refinement.

Confidence weighting provides a smaller but useful robustness benefit. Incremental training improves both accuracy and stability in rolling prediction. Historical mean features provide only marginal additional improvement, suggesting that the temporal prediction already contains most of the relevant historical information.

The main conclusion is that taxi demand forecasting should be treated as a dynamic spatiotemporal problem. A purely temporal model is not enough, and a purely graph-based correction model needs a strong temporal prior. The proposed two-stage framework provides a practical compromise: it uses temporal modelling to generate base predictions and graph learning to correct spatially structured residuals. With confidence weighting and incremental updating, the framework becomes more robust and better suited to rolling forecasting.

At the same time, the results should be interpreted with appropriate caution. The graph refinement stage demonstrates strong spatial correction ability, but future work should test stricter temporal generalisation. The confidence module improves robustness, but its calibration should be studied more deeply. Incremental training improves rolling performance, but its computational cost and long-term behaviour require further evaluation. These limitations do not weaken the central contribution. Instead, they clarify the boundary of the current work and provide a concrete path for future development.

In summary, the framework demonstrates that combining multi-scale temporal prediction, graph-based residual correction, confidence-aware weighting, and incremental updating can substantially improve taxi demand forecasting. The experimental evidence supports the claim that spatial structure and rolling adaptation are essential for this task. The project therefore contributes not only a forecasting model, but also a modular methodology for building more reliable spatiotemporal prediction systems.

Chapter 7

Conclusion

This thesis investigated the problem of short-term urban taxi demand forecasting at the taxi-zone level. The central objective was to design and evaluate a forecasting framework that can capture both temporal demand dynamics and spatial dependency between zones under a rolling prediction setting. The work was motivated by the observation that urban mobility demand is not determined by a single source of information. Taxi demand changes over time according to short-term fluctuations, daily routines, weekly cycles, and longer-term trends. At the same time, demand in one zone is spatially coupled with demand in neighbouring or flow-connected zones. A forecasting model that only considers the historical sequence of each zone may therefore miss important spatial information, while a purely graph-based model may lack a strong temporal prior. To address this, the thesis proposed a two-stage framework that combines multi-scale temporal forecasting with graph-based residual refinement.

The proposed system consists of four main components. The first component is a multi-scale temporal prediction model. This model uses several historical windows to represent different temporal scales, including short-term, daily, weekly, and longer-term demand patterns. The purpose of this stage is to produce an initial demand prediction for each taxi zone based on its own historical behaviour. The second component is a GraphSAGE-based graph refinement model. Instead of predicting taxi demand directly, the graph model learns the residual between the temporal prediction and the observed demand. The final prediction is then obtained by adding the predicted residual correction to the temporal prediction. This design allows the temporal model to capture within-zone time-series patterns, while the graph model focuses on correcting spatially structured errors. The third component is a confidence-weighting mechanism. It combines prior data sufficiency, MC Dropout stability, and neighbourhood consistency into node-level confidence scores, which are used to weight the graph residual learning process. The fourth component is incremental training, which allows the temporal model to adapt during rolling forecasting as new observations become available.

The experimental results support the effectiveness of this structured design. The most significant improvement comes from graph-based residual refinement. In Experiment 2, the temporal-only multi-scale model achieved an MAE of 33.4804 and an RMSE of 51.9800. After adding GraphSAGE residual refinement, the MAE decreased to 16.2106 and the RMSE decreased to 30.5118. This corresponds to a reduction of more than 50% in MAE compared with the temporal-only prediction. This result demonstrates that the residual errors produced by the temporal model are not purely random. Instead, they contain spatial patterns that can be learned from the taxi-zone graph. The graph refinement stage is therefore the main source of accuracy improvement in the proposed pipeline.

The experiment also showed that adding simple historical mean features to the GNN provides only marginal additional benefit. The model with GraphSAGE and historical features achieved an MAE of 16.2040, which is only slightly lower than the GraphSAGE model without those features. This indicates that the improvement is mainly due to graph-based residual correction rather than handcrafted historical statistics. A likely explanation is that the multi-scale temporal model already encodes much of the information represented by features such as `mean_24h`, `mean_168h`, and `mean_720h`. Therefore, these historical mean features provide auxiliary context, but they are not the central driver of performance improvement.

Experiment 3 evaluated the confidence-weighted GNN mechanism. The results showed that confidence weighting provides a moderate but useful improvement in robustness. Among the individual confidence components, the MC Dropout stability score produced the best average accuracy, reducing MAE from 16.6460 in the unweighted GNN to 16.4645. This suggests that uncertainty estimated from stochastic forward passes is a useful signal for identifying reliable predictions. The neighbourhood-only confidence score produced the lowest standard deviation of hourly MAE, indicating that neighbourhood consistency is especially useful for stabilising rolling predictions. The full confidence model improved all metrics compared with the unweighted GNN, although it was not the best model for every individual metric. This shows that confidence weighting should be interpreted as a robustness enhancement rather than as the primary source of error reduction. Its main value is to reduce the influence of unreliable nodes during graph training and to make the refinement process more stable.

Experiment 4 evaluated incremental training in the rolling forecasting setting. The results showed that incremental updating improves both the base temporal model and the final graph-refined model. For the temporal-only prediction, incremental training reduced MAE from 34.9008 to 26.3093. After graph refinement, incremental training reduced MAE from 14.7466 to 13.0842 and reduced RMSE from 30.3834 to 24.8755. It also improved rolling stability, reducing the standard deviation of hourly MAE for the GNN-refined model from 7.7932 to 5.7611. These results confirm that rolling demand forecasting benefits from updating the model as new data arrive. A static checkpoint can become less representative as the forecast origin moves forward, while incremental updating allows the model to adapt to recent demand changes. This makes incremental training particularly important for practical deployment scenarios where demand patterns evolve continuously.

Overall, the experiments demonstrate that each part of the proposed framework contributes differently. The multi-scale temporal model provides the base prediction and captures within-zone temporal structure. The graph refinement model provides the largest improvement in predictive accuracy by correcting spatially structured residual errors. The confidence module improves the reliability of the graph refinement process by weighting nodes according to their estimated trustworthiness. Incremental training improves adaptation and reduces instability during rolling forecasting. The final system should therefore be understood as a modular forecasting pipeline rather than a single monolithic model:

$$\text{Multi-scale Temporal Prediction} \rightarrow \text{GraphSAGE Residual Refinement} \rightarrow \text{Confidence-weighted Learnin}$$

This modular structure is one of the main strengths of the work. It allows each component to be analysed, improved, and replaced independently. For example, the temporal backbone could be changed without redesigning the graph refinement module, and the GraphSAGE component could be replaced by another graph neural network architecture in future work.

The thesis makes several contributions. First, it proposes a two-stage forecasting architecture for taxi-zone demand prediction, where a multi-scale temporal model is followed by graph-based residual correction. This design combines the strengths of temporal sequence modelling and graph neural networks while keeping their responsibilities clearly separated. Second, it demonstrates that residual learning on a taxi-zone graph can substantially improve temporal-only predictions. Third, it introduces a confidence-weighted graph refinement strategy based on historical sufficiency, MC Dropout stability, and neighbourhood consistency. Fourth, it evaluates incremental training as a practical mechanism for improving rolling forecasting stability. Together, these contributions show that short-term taxi demand forecasting can benefit from combining temporal modelling, spatial correction, uncertainty-aware weighting, and online adaptation.

Despite these positive findings, several limitations remain. The rolling evaluation period is relatively limited, and future work should test the framework across a wider range of dates, weeks, seasons, and abnormal demand conditions. The current graph refinement protocol evaluates node-level residual correction within rolling target hours, but a stricter future-facing evaluation should train the graph model on previous hours and test it on future hours. This would better reflect real-time deployment. Another limitation is that the current graph primarily uses connectivity derived from the OD flow matrix, while edge weights are not fully exploited in the message passing operation. A weighted graph neural network or graph attention mechanism could make better use of flow intensity. In addition, the confidence fusion strategy uses fixed manually selected weights. Future work could learn these weights automatically or calibrate the confidence score against actual prediction errors.

Further extensions could also improve the system. External contextual features such as weather, public holidays, special events, public transport disruptions, and land-use information could help explain demand changes that are difficult to infer from historical taxi counts alone. A dynamic graph could be used to represent changes in inter-zone relationships over time, rather than relying on a static OD-based structure. More advanced uncertainty methods, such as quantile regression, deep ensembles, or conformal prediction, could provide calibrated prediction intervals rather than only point estimates and confidence scores. Finally, more detailed efficiency evaluation would be useful, especially for deployment. Since the framework involves per-zone temporal modelling, rolling updates, and graph refinement, runtime, memory usage, and update latency should be measured systematically.

In conclusion, this thesis shows that taxi demand forecasting should be treated as a dynamic spatiotemporal problem. The results demonstrate that a temporal model alone is not sufficient, even when it uses multiple historical scales. Spatial dependency between taxi zones provides important information for correcting temporal forecasting errors. The proposed GraphSAGE residual refinement stage successfully captures this information and produces a large improvement in prediction accuracy. Confidence weighting further improves the reliability of graph learning, while incremental training improves adaptation and stability under rolling forecasting. The final framework therefore provides a practical and extensible approach to taxi-zone demand prediction. It shows that combining multi-scale temporal modelling, graph-based residual correction, uncertainty-aware confidence weighting, and incremental updating can produce more accurate and stable forecasts for urban mobility systems.

Appendix A

Additional Materials

Additional experiments, derivations, or implementation details.

Bibliography

[1] J. Xue et al., *Spatial-temporal generative ai for traffic flow estimation with sparse data of connected vehicles*, Arxiv.org, 2020. Accessed: Mar. 21, 2025. [Online]. Available: https://arxiv.org/html/2407.08034v1

[2] X. Qian and S. V. Ukkusuri, "Spatial variation of the urban taxi ridership using gps data," *Applied Geography*, vol. 59, pp. 31–42, May 2015. DOI: 10.1016/j.apgeog.2015.02.011 Accessed: Feb. 1, 2020.

[3] B. Schaller, "A regression model of the number of taxicabs in u.s. cities," *Journal of Public Transportation*, vol. 8, pp. 63–78, Dec. 2005. DOI: 10.5038/2375-0901.8.5.4 Accessed: Oct. 22, 2019.

[4] K. Zhang, Z. Liu, and L. Zheng, "Short-term prediction of passenger demand in multi-zone level: Temporal convolutional neural network with multi-task learning," *IEEE Transactions on Intelligent Transportation Systems*, vol. 21, no. 4, pp. 1480–1490, 2020. DOI: 10.1109/TITS.2019.2909571

[5] D. Zhuang, S. Wang, H. N. Koutsopoulos, and J. Zhao, "Uncertainty quantification of sparse travel demand prediction with spatial-temporal graph neural networks," in *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2022, pp. 4639–4647. [Online]. Available: https://arxiv.org/abs/2208.05908

[6] Z. Wu, S. Pan, G. Long, J. Jiang, X. Chang, and C. Zhang, "Graph wavenet for deep spatial-temporal graph modeling," *arXiv preprint arXiv:1906.00121*, 2019. [Online]. Available: https://arxiv.org/abs/1906.00121

[7] C. Li et al., "Demand forecasting and predictability identification of ride-sourcing services under uncertainty," *Transportation Research Part C: Emerging Technologies*, 2024, Article in press / 2024 online publication.

[8] X. Ye et al., “Demand forecasting of online car-hailing with combining multiple time-series data,” *Electronics*, vol. 10, no. 20, p. 2480, 2021. DOI: 10.3390/electronics10202480

[9] E. Kontou, A. de Bortoli, and S. Mishra, “Reducing ridesourcing empty vehicle travel with future demand information,” *Transportation Research Part C: Emerging Technologies*, vol. 120, p. 102 793, 2020. DOI: 10.1016/j.trc.2020.102793

[10] C. Zhang, J. J. Q. Yu, and Y. Liu, “Spatial-temporal graph attention networks: A deep learning approach for traffic forecasting,” *IEEE Access*, vol. 7, pp. 166 246–166 256, 2019. DOI: 10.1109/ACCESS.2019.2953888

[11] Z. Wu, S. Pan, F. Chen, G. Long, C. Zhang, and P. S. Yu, “A comprehensive survey on graph neural networks,” *IEEE Transactions on Neural Networks and Learning Systems*, vol. 32, no. 1, pp. 4–24, 2021. DOI: 10.1109/TNNLS.2020.2978386

[12] Y. Li, R. Yu, C. Shahabi, and Y. Liu, “Diffusion convolutional recurrent neural network: Data-driven traffic forecasting,” in *International Conference on Learning Representations (ICLR)*, 2018. [Online]. Available: https://arxiv.org/abs/1707.01926

[13] W. Jiang and J. Luo, “Graph neural network for traffic forecasting: A survey,” *Expert Systems with Applications*, vol. 207, p. 117 921, 2022. DOI: 10.1016/j.eswa.2022.117921

[14] J. García-Sigüenza, F. Llorens-Largo, L. Tortosa, and J. F. Vicent, “Explainability techniques applied to road traffic forecasting using graph neural network models,” *Information Sciences*, vol. 650, p. 119 320, 2023. DOI: 10.1016/j.ins.2023.119320

[15] Z. Shao et al., “Decoupled dynamic spatial-temporal graph neural network for traffic forecasting,” *Proceedings of the VLDB Endowment*, vol. 15, no. 11, pp. 2733–2746, 2022. [Online]. Available: https://arxiv.org/abs/2206.09112

[16] Q. Wang, Y. Li, H. N. Koutsopoulos, and J. Zhao, “Uncertainty quantification of spatiotemporal travel demand with probabilistic graph neural networks,” *IEEE Transactions on Intelligent Transportation Systems*, 2024. DOI: 10.1109/TITS.2024.3367779

[17] K. Cho et al., “Learning phrase representations using rnn encoder–decoder for statistical machine translation,” in *Proceedings of EMNLP*, 2014. [Online]. Available: https://arxiv.org/abs/1406.1078

[18] Y. Gal and Z. Ghahramani, "Dropout as a bayesian approximation: Representing model uncertainty in deep learning," in *Proceedings of the 33rd International Conference on Machine Learning*, 2016. [Online]. Available: https://proceedings.mlr.press/v48/gal16.html

[19] W. L. Hamilton, R. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Advances in Neural Information Processing Systems*, 2017. [Online]. Available: https://papers.nips.cc/paper/6703-inductive-representation-learning-on-large-graphs

[20] L. Liao et al., "A multi-sensory stimulating attention model for cities' taxi demand prediction," *Scientific Reports*, 2022. [Online]. Available: https://repository.essex.ac.uk/32270/1/s41598-022-07072-z.pdf

[21] L. Su et al., "A systematic review for transformer-based long-term series forecasting," *Artificial Intelligence Review*, 2025. [Online]. Available: https://link.springer.com/article/10.1007/s10462-024-11044-2

[22] J. Ke et al., "Predicting origin-destination ride-sourcing demand with a residual multi-graph convolutional network," *Transportation Research Part C: Emerging Technologies*, 2021. [Online]. Available: https://ira.lib.polyu.edu.hk/bitstream/10397/88951/1/Ke_Origin-destination_Ride-sourcing_Demand.pdf

[23] Y. Liu et al., "How machine learning informs ride-hailing services: A survey," *Communications in Transportation Research*, vol. 2, p. 100 075, 2022.

[24] M. Khalesian et al., "Improving deep-learning methods for area-based traffic demand forecasting," *Transportation Research Part C: Emerging Technologies*, 2024.

[25] J. Tang et al., "Understanding spatio-temporal characteristics of urban travel demand," *Sustainability*, vol. 11, no. 19, p. 5525, 2019.

[26] C. Riley et al., "Integrated demand prediction and dynamic dispatch for ride-pooling systems," *arXiv preprint arXiv:2003.10942*, 2020.

[27] F. Huang et al., "A dynamical spatial-temporal graph neural network for traffic demand prediction," *Information Sciences*, vol. 594, pp. 286–304, 2022.

[28] H. Xue, X. Hu, and Y. Chen, "Spatiotemporal representation learning for taxi demand prediction," *IEEE Transactions on Intelligent Transportation Systems*, 2020.

[29] X. Fu et al., “Temporal graph neural networks for traffic prediction: A survey,” *arXiv preprint arXiv:2307.00495*, 2024.

[30] X. Liu et al., “A comprehensive review of traffic flow forecasting based on deep learning,” *Neurocomputing*, 2025.

[31] F. Petropoulos, S. Makridakis, V. Assimakopoulos, and K. Nikolopoulos, “Forecasting: Theory and practice,” *International Journal of Forecasting*, vol. 38, no. 3, pp. 705–871, 2022. [Online]. Available: https://arxiv.org/pdf/2012.03854

[32] T. Lyu et al., “Research on the big data of traditional taxi and online car-hailing: A systematic review,” *Journal of Traffic and Transportation Engineering (English Edition)*, vol. 8, no. 3, pp. 346–376, 2021.

[33] A. Safikhani, C. Kamga, S. Mudigonda, et al., “Spatio-temporal modeling of yellow taxi demands in new york city using generalized star models,” *arXiv preprint arXiv:1711.10090*, 2017. [Online]. Available: https://arxiv.org/pdf/1711.10090

[34] B. Medina-Salgado et al., “Urban traffic flow prediction techniques: A review,” *Results in Engineering*, vol. 15, p. 100 424, 2022.

[35] S. Roy et al., “Spatial transferability of machine learning based models for ride-hailing demand prediction,” *Transportation Research Part A: Policy and Practice*, 2025.

[36] R. Pascanu, T. Mikolov, and Y. Bengio, “On the difficulty of training recurrent neural networks,” in *Proceedings of the 30th International Conference on Machine Learning*, 2013, pp. 1310–1318. [Online]. Available: https://proceedings.mlr.press/v28/pascanu13.html

[37] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997. DOI: 10.1162/neco.1997.9.8.1735

[38] Y. Yang et al., “Survey of short-term traffic flow prediction based on lstm,” *International Journal of Neural Systems*, 2024.

[39] J. Chung, Ç. Gülçehre, K. Cho, and Y. Bengio, “Empirical evaluation of gated recurrent neural networks on sequence modeling,” *arXiv preprint arXiv:1412.3555*, 2014. [Online]. Available: https://arxiv.org/abs/1412.3555

[40] Q. Huang et al., “Effective traffic forecasting with multi-resolution learning,” in *Proceedings of the 30th ACM International Conference on Information and Knowledge Management*, 2021. DOI: 10.1145/3459637.3482363

[41] J. Kim et al., “A comprehensive survey of deep learning for time series forecasting,” *Artificial Intelligence Review*, 2025.

[42] J. Zhao et al., “A survey of transformer networks for time series forecasting,” *Information Fusion*, 2026.

[43] B. Lim and S. Zohren, “Time-series forecasting with deep learning: A survey,” *Philosophical Transactions of the Royal Society A*, vol. 379, no. 2194, p. 20 200 209, 2021.

[44] B. Sonnleitner et al., “Measuring time series heterogeneity for global learning,” *Expert Systems with Applications*, 2025.

[45] Y. Liu et al., “How machine learning informs ride-hailing services: A survey,” *Communications in Transportation Research*, vol. 2, p. 100 075, 2022.

[46] Y. Zheng et al., “Fairness-enhancing deep learning for ride-hailing demand prediction,” *Transportation Research Part C: Emerging Technologies*, 2024.

[47] Q. Huang et al., “Effective traffic forecasting with multi-resolution learning,” in *Proceedings of the 30th ACM International Conference on Information and Knowledge Management*, 2021.

[48] B. Lim, S. O. Arik, N. Loeff, and T. Pfister, “Temporal fusion transformers for interpretable multi-horizon time series forecasting,” *International Journal of Forecasting*, vol. 37, no. 4, pp. 1748–1764, 2021. DOI: 10.1016/j.ijforecast.2021.03.012

[49] J. Ling et al., “A multi-scale residual graph convolution network with hierarchical attention for predicting traffic flow in urban mobility,” *Complex & Intelligent Systems*, 2024. DOI: 10.1007/s40747-023-01324-9

[50] T. Theodoropoulos et al., “West gcn-lstm: Weighted stacked spatio-temporal graph neural network architecture for regional traffic forecasting,” *Smart Cities*, 2025.

[51] A. Cini, I. Marisca, and C. Alippi, “Timegnn: Temporal dynamic graph learning for time series forecasting,” *arXiv preprint arXiv:2307.14680*, 2023.

[52] K. Ofori-Boateng et al., “Dynamic graph neural network for traffic forecasting in wide area networks,” *arXiv preprint arXiv:2008.12767*, 2020.

[53] Y. Zheng, L. Yi, and Z. Wei, “A survey of dynamic graph neural networks,” *Frontiers of Computer Science*, 2024. DOI: 10.1007/s11704-024-3853-2

[54] H. Hewamalage, K. Ackermann, and C. Bergmeir, "Forecast evaluation for data scientists: Common pitfalls and best practices," *Data Mining and Knowledge Discovery*, vol. 37, pp. 788–832, 2023.

[55] V. Cerqueira, L. Torgo, and I. Mozeti, "Evaluating time series forecasting models: An empirical study on performance estimation methods," *Machine Learning*, vol. 109, pp. 1997–2028, 2020. DOI: 10.1007/s10994-020-05910-7

[56] X. Chen et al., "Demand forecasting of online car-hailing by exhaustively considering temporal, spatial and weather features," *IET Intelligent Transport Systems*, 2023. DOI: 10.1049/itr2.12387

[57] M. Akram et al., "Modelling count, bounded and skewed continuous outcomes using the generalised linear model: A practical introduction and software tutorial," *Health Psychology and Behavioral Medicine*, vol. 11, no. 1, p. 2 204 515, 2023.

[58] S. Park, C. Chen, and F. Fioretto, "Confidence-aware graph neural networks for learning reliability assessment commitments," *arXiv preprint arXiv:2211.15755*, 2022. [Online]. Available: https://arxiv.org/abs/2211.15755

[59] F. Wang et al., "Uncertainty in graph neural networks: A survey," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2024.

[60] J. Moon, J. Kim, Y. Shin, and S. Hwang, "Confidence-aware learning for deep neural networks," *Proceedings of the 37th International Conference on Machine Learning*, pp. 7034–7044, 2020.

[61] A. Chernikova et al., "Uncertainty-aware message passing neural networks," *Open-Review preprint*, 2025.

[62] J. Yan et al., "Uncertainty quantification on graph learning: A survey," *arXiv preprint arXiv:2404.14642*, 2025. [Online]. Available: https://arxiv.org/abs/2404.14642

[63] Y. Choi et al., "Hierarchical uncertainty-aware graph neural network," *arXiv preprint arXiv:2504.19820*, 2025. [Online]. Available: https://arxiv.org/abs/2504.19820

[64] M. Ren, W. Zeng, B. Yang, and R. Urtasun, "Learning to reweight examples for robust deep learning," in *Proceedings of the 35th International Conference on Machine Learning*, 2018, pp. 4334–4343. [Online]. Available: https://proceedings.mlr.press/v80/ren18a.html

[65] X. Shi, Z. Guo, K. Li, Y. Liang, and X. Zhu, "Self-paced resistance learning against overfitting on noisy labels," *Pattern Recognition*, vol. 138, p. 109 420, 2023. DOI: 10.1016/j.patcog.2022.109420

[66] R. C. Cavalcante et al., "Forecasting in data streams: A review," *Artificial Intelligence Review*, 2024. DOI: 10.1007/s10462-024-10925-w

[67] X. Lu et al., "An incremental learning framework for traffic forecasting with temporal drift adaptation," *Transportation Research Part C: Emerging Technologies*, 2023.

[68] A. Vaswani et al., "Attention is all you need," in *Advances in Neural Information Processing Systems*, 2017.